

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 04.09.2026, 22:36:31
Файлов исходного кода: 79
Суммарный объём кода: 1021.8 KB
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

```
22. src/data/tickets/ticket1_5.ts
23. src/data/tickets/ticket14_20.ts
24. src/data/tickets/ticket21_24.ts
25. src/data/tickets/ticket6_13.ts
```

Интерактивные компоненты и симуляторы

```
26. src/components/ArticulationPointsViz.tsx
27. src/components/BellmanFordViz.tsx
28. src/components/BfsViz.tsx
29. src/components/ChapterImage.tsx
30. src/components/ChapterNav.tsx
31. src/components/ChapterView.tsx
32. src/components/DfsBridgesSimulator.tsx
33. src/components/DijkstraViz.tsx
34. src/components/DPVisualizer.tsx
35. src/components/EulerSimulator.tsx
36. src/components/ExpressQuiz.tsx
37. src/components/FloydViz.tsx
38. src/components/GraphTraversalViz.tsx
39. src/components/HeapViz.tsx
40. src/components/hints/demoKit.tsx
41. src/components/hints/demosExtra.tsx
42. src/components/hints/demosGraph.tsx
43. src/components/hints/demosStruct.tsx
44. src/components/hints/MiniDemo.tsx
45. src/components/hints/SimulatorModal.tsx
46. src/components/hints/TermPopover.tsx
47. src/components/JohnsonViz.tsx
48. src/components/KosarajuViz.tsx
49. src/components/KruskalSimulator.tsx
50. src/components/MnemonicCards.tsx
51. src/components/MobileToc.tsx
52. src/components/Navbar.tsx
53. src/components/PlanarityDemo.tsx
54. src/components/QueueViz.tsx
55. src/components/SalmonAutomatonWidget.tsx
56. src/components/SegmentTreeVisualizer.tsx
57. src/components/Sidebar.tsx
```

Универсальное пособие • полный исходный код

ФАЙЛ 1/79: AGENTS.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7

AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

****CRITICAL DIRECTIVE****:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML elements.

ФАЙЛ 2/79: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2

Gemini App Canvas Instructions

Ты – AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas). При запросе пользователя создать или сгенерировать новую страницу, ты ****строго**** придерживайся готовых шаблонов разметки, стилей и структуры контента.

Правила для малых страниц (Билетов / Глав)

- **Единый стиль:**** Всегда используй Tailwind CSS и HTML-шаблоны.
- **Структура JSON/TS:**** Каждая новая малая страница должна быть сгенерирована в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.json``.
- **Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-500``).

```
}
```

ФАЙЛ 5/79: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 41 | РАЗМЕР: 1

```
{
  "name": "algv0-exam-guide",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "audit:content": "node scripts/audit-coverage.mjs",
    "audit:terms": "node scripts/audit-terms.mjs",
    "export:txt": "node scripts/export/collect-code.mjs",
    "export:png": "node scripts/export/render-pages.mjs",
    "export:pdf": "node scripts/export/build-pdf.mjs",
    "export": "npm run export:txt && npm run export:png",
  },
  "dependencies": {
    "clsx": "2.1.1",
    "lucide-react": "^1.21.0",
    "motion": "^12.40.0",
    "pdfkit": "^0.20.1",
    "react": "19.2.6",
    "react-dom": "19.2.6",
    "tailwind-merge": "3.4.0"
  },
  "devDependencies": {
    "@tailwindcss/vite": "4.1.17",
    "@types/node": "22.19.17",
    "@types/pdfkit": "^0.17.6",
    "@types/react": "19.2.7",
  }
}
```

Универсальное пособие • полный исходный код

```
-----  
| `npm run export:pdf` | png → pdf | `public/export/full`  
| `npm run export` | всё сразу | txt + png + pdf |
```

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
```
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` — DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` — оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается **на каждом коммите** и пересобирает TXT → PNG → PDF, кладёт PNG-страницы и документ PDF в папку `public/export/full\_code\_all\_pages` и коммитит обновлённые `public/export/full\_code\_all\_pages` (с меткой `[skip ci]`, чтобы не зациклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code\_all\_pages`

Аудит контента

```
```bash  
node scripts/audit-coverage.mjs # какие темы без контента
node scripts/audit-coverage.mjs --md # markdown-таблицы
```
```

Актуальный отчёт: [`TOPICS\_AUDIT.md`](./TOPICS_AUDIT.md)

Как устроен интерфейс

- * **Содержание** — на десктопе боковая панель с поиском, на телефоне — липкая строка «Содержание · N из M» и вкладки
- * **Переходы между темами** — кнопки «Назад / Вперёд» в шапке, «Предыдущая / Следующая тема» под текстом; на клавиатуре —

Универсальное пособие • полный исходный код

* В самом исходном коде страницы (и внутри тегов) формулы (math) .

* Таким образом, при извлечении текстового содержимого технически остаются в валидном Markdown-формате латеха (math).

Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторные изображения.

* Они помещены внутрь оберток с классом `<div class="img">`

* Все отрисовки находятся внутри векторного тега `<svg>`

* Подпись к картинке, если она есть, идет сразу после SVG-тега.

Списки

* Стандартные списки оформлены через `<ul class="list-n">`

Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML.

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`

2. Элементы `.analogy-badge` получают выделение маркера `~`

3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ... ]` (или `[ Иллюстрация ... ]`).

4. Элементы списка `<li>` получают маркер `-`.

5. Результат извлекается в виде текста и чистится от лишнего.

ФАЙЛ 7/79: TOPICS_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

Аудит покрытия тем: где нет «объяснения на пальцах» и т.д.

> Сгенерировано скриптом `node scripts/audit-coverage.mjs`

> Критерии:

> * **Аналогия** — в HTML главы есть блок «Аналогия ...»

> * **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

Универсальное пособие • полный исходный код

```
-----
| Планарность: K5, K3,3 и формула Эйлера | `planarity-euler` |
| т «на пальцах». |
| 16. Флойд (Флойд–Уоршелл) | `floyd` | Есть матричный
| 17. Алгоритм Джонсона | `johnson-algo` | Перевзвешива
| Обзор: Кратчайшие пути и Графы | `intro` | Вводная стр
| Эйлер: Путь ≠ Цикл | `euler-path-vs-cycle` | Ключевая
```

```
---
## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации (
```

```
> Замкнуто: «5. Двумерная матрица префиксных сумм» полн
```

```
Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализа
8. Графы. Компоненты связности	`graph-components`	
24. Классы сложности, сведение задач	`complexity-cl	
```

```
---
## Сводная таблица по всем главам
```

| # | Глава | Аналогий | 2D-виз. | SVG | Картинка |
|----|---|----------|---------|-----|----------|
| 2 | 2. Двумерная разреженная матрица (Sparse Table) | | | | |
| 3 | 3. Декартово дерево (Treap) | 2 | – | – | – |
| 4 | 4. Splay-дерево | 2 | | – | – |
| 5 | 5. Двумерная матрица префиксных сумм | 1 | | | – |
| – | Планарность: K5, K3,3 и формула Эйлера | – | | | – |
| 6 | 6. Графы. Представление, DFS, BFS | 4 | | – | – |
| 8 | 8. Графы. Компоненты связности | 1 | – | – | – |
| 9 | 9. Графы. Топологическая сортировка | 1 | | | – |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | | | – |
| 12 | 12. Графы. Точки сочленения | 1 | | – | – |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | | | – |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | | | – |

```
  "types": ["node"],

  /* Bundler mode */
  "moduleResolution": "bundler",
  "allowImportingTsExtensions": true,
  "resolveJsonModule": true,
  "isolatedModules": true,
  "noEmit": true,
  "jsx": "react-jsx",

  /* Path mapping */
  "baseUrl": ".",
  "paths": {
    "@/*": ["src/*"]
  },

  /* Linting */
  "strict": true,
  "noUnusedLocals": true,
  "noUnusedParameters": true,
  "noFallthroughCasesInSwitch": true
},
"include": ["src", "vite.config.ts"]
}
```

ФАЙЛ 9/79: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 33 | РАЗМЕР: 812

```
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"
```



```

-----
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";

export default function App() {
  const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");
  const [activeChapterId, setActiveChapterId] = useState<string | null>(null);
  const [modalVizId, setModalVizId] = useState<string | null>(null);

  const index = Math.max(
    0,
    chapters.findIndex((c) => c.id === activeChapterId)
  );
  const activeChapter = chapters[index] ?? chapters[0];
  const prev = index > 0 ? chapters[index - 1] : undefined;
  const next = index < chapters.length - 1 ? chapters[index + 1] : undefined;

  const goTo = useCallback((id: string) => {
    setActiveChapterId(id);
    window.scrollTo({ top: 0, behavior: "smooth" });
  }, []);

  // Стрелки ← → листают темы (как на обучающих сайтах)
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
      const target = e.target as HTMLElement | null;
      if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target.tagName)) return;
      if (e.metaKey || e.ctrlKey || e.altKey) return;
      if (e.key === "ArrowRight" && next) goTo(next.id);
      if (e.key === "ArrowLeft" && prev) goTo(prev.id);
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown", onKey);
  }, [goTo, next, prev]);
}

```

```

        <div
          className="h-full bg-gradient-to-r from-blue-500 to-purple-500"
          style={{ width: `${progress}%` }}
        />
      </div>

      <ChapterView
        key={activeChapter.id}
        chapter={activeChapter}
        prev={prev}
        next={next}
        index={index}
        total={chapters.length}
        onGo={goTo}
        onOpenSimulator={setModalVizId}
      />
    </main>
  </div>
</>
) : (
  <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
    <SimulatorHub
      onOpen={setModalVizId}
      onGoChapter={({id}) => {
        setActiveTab("guide");
        goTo(id);
      }}
    />
  </main>
)}

<SimulatorModal vizId={modalVizId} onClose={() => {
  setActiveTab("guide");
  setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
  © 2026 Universal Educational Guide. Интерактивное руководство
</footer>
</div>
);

```

```
* повесить всплывающую карточку (как превью ссылок в Википедии)
*/
export function annotateTerms(root: HTMLElement): void {
  if (!cachedRegex) cachedRegex = buildRegex();
  const re = cachedRegex;

  const perTerm = new Map<string, number>();
  const perSurface = new Map<string, number>();
  const walker = document.createTreeWalker(root, NodeFilter.FILTER_ACCEPT, {
    acceptNode(node) {
      const parent = (node as Text).parentElement;
      if (!parent) return NodeFilter.FILTER_REJECT;
      if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
      if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
      return NodeFilter.FILTER_ACCEPT;
    },
  });

  const targets: Text[] = [];
  let current = walker.nextNode();
  while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
  }

  for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
      const surface = m[1].toLowerCase();
      const id = labelToId.get(surface);
    }
  }
}
```

```
        annotateTerms(el);
    } catch {
        /* если браузер не умеет lookbehind – просто оста
    }
};

// при смене главы
// eslint-disable-next-line react-hooks/exhaustive-dep
useEffect(run, deps);

// страховка: контент перерисовали, а подсказок в нём
useEffect(() => {
    const el = ref.current;
    if (!el) return;
    if (!el.querySelector(".term-hint")) run();
});
}
```

ФАЙЛ 12/79: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

```
@import "tailwindcss";

@layer utilities {
    .neon-glow-blue {
        box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
    }
    .neon-glow-emerald {
        box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
    }
    .neon-glow-rose {
        box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
    }
    .neon-glow-yellow {
        box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
    }
}
```

```

        opacity: 0;
        transform: translateY(4px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.term-popover {
    animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в В
.term-hint {
    cursor: help;
    border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
    color: #c7d2fe;
    border-radius: 3px;
    padding: 0 1px;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
    background: rgba(99, 102, 241, 0.22);
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Контент главы приходит готовым HTML — чиним его под

```

```

        font-size: 1.25rem !important;
        line-height: 1.3 !important;
    }
    .chapter-body :where(h3) {
        font-size: 1.05rem !important;
        line-height: 1.35 !important;
    }
    .chapter-body :where(.p-6, .p-8) {
        padding: 0.9rem !important;
    }
    .chapter-body :where(.text-3xl) {
        font-size: 1.4rem !important;
    }
    .chapter-body :where(.text-2xl) {
        font-size: 1.2rem !important;
    }
    .chapter-body :where(.text-xl) {
        font-size: 1.05rem !important;
    }
}

```

```

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
    width: 8px;
    height: 8px;
}
::-webkit-scrollbar-track {
    background: #0f172a;
}
::-webkit-scrollbar-thumb {
    background: #334155;
    border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
    background: #475569;
}

```

```

/* Индикатор длительности кадра в пошаговых визуализаци

```

```
    category?: string;
}
```

ФАЙЛ 15/79: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
  const src: string;
  export default src;
}
declare module '*.jpg?url' {
  const src: string;
  export default src;
}
declare module '*.jpeg' {
  const src: string;
  export default src;
}
declare module '*.png' {
  const src: string;
  export default src;
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 16/79: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 646 | РАЗМЕР: 10 240 В

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
            нужно покрыть отрезок, мы берем две самые бо
        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg border-1
        <summary class="p-4 font-bold text-slate-300
            -b border-slate-700/50">
                Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded">
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "treap",
        title: "3. Декартово дерево (Treap)",
        type: "html",
        description: "Дерево поиска по ключу и Куча по приоритету",
        category: "Продвинутые структуры",
        content: `
<section id="treap" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1 rounded">Тема
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">Декартово дерево
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
            <h3 class="text-xl font-bold text-blue-400">Описание

```



```

    id: "prefix-sums-2d",
    title: "5. Двумерная матрица префиксных сумм",
    type: "html",
    description: "Сжатие суммы подматрицы за  $O(1)$ ",
    category: "Алгоритмы матрицы",
    content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-emerald-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-emerald-300 italic">
              <p class="text-xl font-mono text-white">
                , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
          </div>
          <div class="grid grid-cols-1 xl:grid-cols-2">
            <div class="bg-slate-800 p-6 rounded-lg">
              <div class="absolute -top-3 left-4 l
                rder border-emerald-500 shadow-md">
                  × Аналогия 1: Вырезание прямоу
                </div>
              <p class="text-slate-300 text-sm mb">
                з него маленький целевой прямоугольник в цел
                ый верхний угол отрезается дважды! Поэтому
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",

```

```

<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Учеба в вузе
          </div>
          <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
>
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Ограничения: Циклы
          </div>
          <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "scc-kosaraju",

```

```

        category: "Графы. Продвинутые",
        content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
          </div>
          <p class="text-slate-300 text-sm mb">
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
      },
      {
        id: "graph-articulation",
        title: "12. Графы. Точки сочленения",
        type: "html",
        description: "Вершины, удаление которых убивает свя.",
        category: "Графы. Продвинутые",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
  <h3 class="text-xl font-bold text-indigo-400">Аналогия 1: Рисование не отрывая
  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-indigo-300 italic">Если не сходится нечетное число линий, значит, из
    <p class="text-xl font-mono text-white">а везде должно быть четное число (зашел-вышел)
  </div>
  <div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
        Аналогия 1: Рисование не отрывая
      </div>
      <p class="text-slate-300 text-sm mb-4">е сходится нечетное число линий, значит, из
      а везде должно быть четное число (зашел-вышел)
    </div>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "pathfinding-accel",
    title: "17. Графы. Ускорения поиска",
    type: "html",
    description: "",
    category: "Графы. Пути",
    content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">17. Графы. Пути
    <h2 class="text-2xl sm:text-3xl font-bold text-white">17. Графы. Пути
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
      <h3 class="text-xl font-bold text-blue-400">Аналогия 1: Рисование не отрывая
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Если не сходится нечетное число линий, значит, из
        <p class="text-xl font-mono text-white">а везде должно быть четное число (зашел-вышел)
      </div>
    </div>
  </div>
</section>`
  }
]

```

```

<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 order border-emerald-500 shadow-md">
      Аналогия: Прокладка интернет
    </div>
    <p class="text-slate-300 text-sm mb">
      с самых дешевых дорог в стране". Он строит
      единую сеть.</p>
    </div>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "mst-prima",
    title: "19. Графы. Остовное дерево. Прима",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          которое торчит из посещенных в непосещенных
        </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4

```

```

        <p class="text-slate-300 text-sm mb-4">
            изкому племени для объединения. К вечеру племена
            ства шлют гонцов к ближайшему чужому царству.
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "string-kmp",
        title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
                <p class="text-xl font-mono text-white">
            i]), который одновременно является её суффиксом.
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
                <!-- Аналогия 1 -->
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border
                    rder border-emerald-500 shadow-md">
                        Аналогия 1: Бумеранг (Префикс и суффикс)
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
                        идишь "АБРА". Если ты ошибешься при поиске

```

```

<h3 class="text-xl font-bold text-blue-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-blue-300 italic">
    <p class="text-xl font-mono text-white">
щегося с i.</p>
</div>
<div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Копипаста
        </div>
        <p class="text-slate-300 text-sm mb-4">
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной копи
        </div>
    </div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "complexity-classes",

```



```
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
];
```

ФАЙЛ 17/79: src/data/content_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМ

```
export interface Chapter {
  id: string;
  title: string;
  type: "html";
  content: string;
}

export const chapters: Chapter[] = [
  {
    id: "euler-path-vs-cycle",
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
    type: "html",
    content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1
  <h2 class="text-3xl font-bold text-white">He 6y,
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-blue-400
    </div>
    <div class="bg-slate-900 p-4 rounded-lg mb-
      <p class="text-lg text-blue-300 italic`
```

```

    </div>
    <p class="text-slate-300 text-sm mb-4">
      <strong>Гамильтон – это болезнь.</strong>
    </p>
    <p>ровно раз и вернуться домой.
      Ты обходишь все бары (вершины) ровно
      Главное – зайти и выйти, не заходя
      Никто не знает, как решать быстро.
      <br><span class="text-xs text-rose-400">
        лноту).</span>
      </p>
    </div>
  </div>

  <details class="bg-slate-900/40 rounded-lg border-1 border-slate-700">
    <summary class="p-4 font-bold text-slate-400">
      Душный режим: Формальное доказательство
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-4">
      <p class="text-blue-400">Теорема Эйлера
      <p>
        Связный граф содержит эйлеров цикл ⇔
        <br>Эйлеров путь (не цикл) ⇔ ровно 2 вершины с нечётной степенью
      </p>
      <p class="text-rose-400">Почему Гамильтон?
      <p>
        Задача коммивояжёра (TSP) – это взломать замок.
        Доказано, что любой полиномиальный алгоритм будет взломать
        В общем, не парься. Просто запомни!
      </p>
    </div>
  </details>
</div>
</section>`,
  },
  {
    id: "bridges-code",
    title: "Мосты в графах: код + визуализация",
  }
]

```

Из пещеры и нет других выходов — то
Если верёвка (ребро $v-u$) оборвётся —
Если бы из пещеры и был чёрный ход

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">

Аналогия: Кровеносный сосуд

</div>

<p class="text-slate-300 text-sm mb-4">

Граф — это кровеносная система. Ребра (артерии и вены, т.е. каналы, по которым течёт кровь) — это не мост, живём.

Если же перерезать единственную артерию, то все вены перестанут работать.

Алгоритм DFS ищет такие «критические» ребра.

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
 <summary class="p-4 font-bold text-slate-400">
 order-slate-700/50">

Полный код на C++ (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-y-4">

<p>Классическая реализация — <code class="text-slate-400">DFS</code> по графу. Если же перерезать единственную артерию, то все вены перестанут работать. Алгоритм DFS ищет такие «критические» ребра.</p>

<div class="bg-slate-950 p-4 rounded-lg border-rose-500 shadow-md">

<pre class="text-xs font-mono text-slate-400">

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежных ребер
```

```

                <p class="text-xs text-slate-400">Запис
            </div>
        </details>
    </div>
</section>`,
},
{
    id: "planarity-euler-formula",
    title: "Планарность: K5, K3,3 и формула Эйлера",
    type: "html",
    content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
        <h3 class="text-xl font-bold text-blue-400 r

        <div class="bg-slate-900 p-4 rounded-lg mb-4
            <p class="text-lg text-blue-300 italic r
            <p class="text-3xl font-mono text-white
            <div class="text-sm text-slate-400 spac
                <p><span class="text-blue-400 font-l
                <p><span class="text-emerald-400 fo
                <p><span class="text-rose-400 font-l
            </div>
        </div>

        <p class="text-slate-300 text-sm">
            <strong>Следствие:</strong> Для планарн
            ode>.
            Если ребер больше – граф гарантированно
        </p>
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap
```

```

        <p>
            Пусть K5 планарен.  $V = 5$ ,  $E = 10$ .
            <br> $F = E - V + 2 = 10 - 5 + 2 = 7$ 
            <br>Каждая грань ограничена минимум
            <br>Сумма длин граней  $= 2E = 20$ .
            <br>Отсюда:  $2E \geq 3F \Rightarrow 20 \geq 21$  — <span>
            <br>Значит, K5 не может быть планарн
        </p>
    </div>
</details>
</div>
</section>`,
    },
    {
        id: "graph-articulation",
        title: "12. Графы. Точки сочленения",
        type: "html",
        content: `<section id="graph-articulation" class="mb-6">
<div class="flex items-center mb-6">
    <span class="bg-rose-600 text-white px-4 py-1 rounded">
    <h2 class="text-3xl font-bold text-white">Точки
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-rose-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-6">
        <p class="text-lg text-rose-300 italic">
        <div class="text-left font-mono text-sm">
            <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>
            <div class="p-3 bg-slate-950 rounded">
                <span class="text-rose-400 font">
                <p class="text-xl font-bold tex">
                <p class="text-xs text-slate-400">
            </div>
        </div>
    </div>
`
    }
}

```

```

        </summary>
        <div class="p-5 text-sm text-slate-300 space-y-4">
            <p class="text-blue-400 font-bold">Особый случай (когда у него нет детей, то выше неё прыгать некуда). Поэтому для него у него <strong>больше 1 независимого ребенка.</strong>
            </p>
            <div class="bg-slate-950 p-4 rounded-lg">
                <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}</pre>
                </div>
            </div>
        </details>

        <div class="bg-indigo-950/60 p-6 rounded-xl border border-indigo-400">
            <h4 class="text-lg font-bold text-indigo-400">Мосты
            <p class="text-slate-300 text-sm mb-4">
                Это глубокий дуальный мост между ориентированным графом и неориентированным графом.
            </p>
        </div>
    </div>

```

```

    ...tickets21to24
  ].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph
const toRemove = ["graph-planar-colors", "graph-bridges

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
  if (!toRemove.includes(c.id)) {
    // We prefer the newer versions (like tickets14_20
    dedup.set(c.id, c);
  }
});

// Add the new ones
if (newChapters) {
  newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
  const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
  return numA - numB;
});

export const chapters: Chapter[] = finalChapters;

export const activeVizIds = [
  "euler-path-vs-cycle", "bridges-code", "planarity-euler
  "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
  "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi
  "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
  "string-z-func", "segment-trees", "sparse-table", "dynam
];

```

```

        "Очередь (FIFO). Кладём старт, достаём вершину —
        у рёбер.",
        complexity: "O(V + E)",
        demo: "bfs",
        simulator: "queue-bfs",
    },
    {
        id: "dfs",
        labels: ["DFS", "обход в глубину", "поиск в глубину"],
        title: "DFS — обход в глубину",
        short:
            "Прёшь вперёд до упора, упёрся — откатываешься на
        formal:
            "Стек (или рекурсия). Из DFS растут: времена вход
        complexity: "O(V + E)",
        demo: "dfs",
        simulator: "stack-dfs",
    },
    {
        id: "heap",
        labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
        title: "Куча (priority queue)",
        short:
            "Не отсортированный список, а «мешок, из которого
        formal:
            "Полное бинарное дерево в массиве: дети узла i ле
        complexity: "вставка/извлечение O(log n), «посмотре
        demo: "heap",
        simulator: "heap-beam-search",
    },
    {
        id: "stack",
        labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
        title: "Стек (LIFO)",
        short: "Стопка тарелок: кладёшь сверху и берёшь све
        formal: "push / pop / top за O(1). На стеке живёт р
        demo: "stack",
        simulator: "stack-dfs",
    },

```



```

        "обход в ширину по бору",
        "какой-то обход",
        "обходом дерева",
        "конечный автомат",
        "конечного автомата",
        "автомат",
        "автомата",
    ],
    title: "Обход бора в ширину (тот самый «какой-то обход»)",
    short:
        "В Ахо–Корасике бор обходят именно BFS: суффиксную",
        "а есть строго по уровням.",
    formal:
        "Кладём в очередь детей корня (их ссылка = корень",
        "кладём с в очередь.",
    complexity: "O(суммарной длины словаря × алфавит)",
    demo: "trie-bfs",
    simulator: "aho-corasick",
},
{
    id: "suffix-link",
    labels: ["суффиксная ссылка", "суффиксные ссылки",
    title: "Суффиксная ссылка",
    short:
        "«Куда откатиться, если следующая буква не подошла",
        "строки.",
    formal: "link(v) = go(link(parent(v)), ch). Считает",
    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топ",
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с",
        "циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л
```

```

    complexity: "препроцессинг  $O(n \log n)$ , запрос  $O(1)$ ",
    demo: "sparse-table",
    simulator: "sparse-table",
},
{
    id: "segment-tree",
    labels: ["дерево отрезков", "дереве отрезков", "дер"],
    title: "Дерево отрезков",
    short:
        "Матрёшка из отрезков: корень отвечает за весь ма
        товых кусков.",
    formal: "Строится за  $O(n)$ , запрос и обновление —  $O(1)$ ",
    demo: "segment-tree",
    simulator: "segment-trees",
},
{
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостов"],
    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается",
    formal: "Ребро  $(u, v)$  — мост  $\iff \text{low}[v] > \text{tin}[u]$ , где  $\text{tin}[u]$  — время, когда зашло ребро в родителя — нет).",
    complexity: " $O(V + E)$  одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "точка сочленения"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
    formal: "Для не-корня:  $v$  — точка сочленения  $\iff \exists \text{ребро } (u, v) \text{ такое, что } \text{low}[v] \geq \text{tin}[u]$ ",
    demo: "articulation",
    simulator: "graph-articulation",
},
{

```

```

    },
    {
      id: "potential",
      labels: ["потенциал", "потенциалы", "потенциалов",
        title: "Потенциалы (перевзвешивание Джонсона)",
        short:
          "Трюк «поднять все дороги на одинаковую высоту»:
        formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  – рас
          к путей сохраняется.",
        demo: "relax",
        simulator: "johnson-algo",
    },
    {
      id: "scc",
      labels: ["компонента сильной связности", "компоненты
        title: "Компоненты сильной связности",
        short:
          "Группы вершин, где из каждой можно дойти до каждо
        formal: "Косарайю: DFS с сохранением порядка выхода
        complexity: " $O(V + E)$ ",
        demo: "scc",
        simulator: "scc-kosaraju",
    },
    {
      id: "prefix-function",
      labels: [
        "префикс-функция", "префикс-функции", "префикс-фу
        "префикс", "префикса", "суффиксом", "суффикс", "с
        "подстроки", "подстроку", "подстрока",
      ],
      title: "Префикс-функция  $\pi$  (КМП)",
      short:
        "Для каждой позиции запоминаем: какой кусок начал
          от кусок.",
      formal:
        " $\pi[i]$  – длина наибольшего собственного префикса  $s$ 
          возвращая указатель по тексту.",
      complexity: " $O(n + m)$ ",

```

```

        "Дейкстра — неотрицательные веса,  $O(E \log V)$ . Форд-Беллман,  $O(V^3)$ .",
        demo: "relax",
        simulator: "dijkstra",
    },
    {
        id: "negative-cycle",
        labels: ["отрицательный цикл", "отрицательного цикла"],
        title: "Отрицательный цикл",
        short:
            "Круг, пройдя по которому вы становитесь «богаче»",
        formal:
            "Форд–Беллман: если на  $V$ -й итерации ещё удаётся с...",
        demo: "relax",
        simulator: "bellman-ford",
    },
    {
        id: "greedy",
        labels: [
            "жадный алгоритм", "жадный", "жадно", "жадная",
            "самое дешёвое ребро", "самое дешёвое ребро", "самое дешёвое ребро",
        ],
        title: "Жадный алгоритм",
        short:
            "На каждом шаге хватаем то, что лучше прямо сейчас",
        formal:
            "Корректность обычно доказывается «леммой о разрезе»",
        demo: "mst",
        simulator: "mst-kruskal",
    },
    {
        id: "euler",
        labels: ["эйлеров цикл", "эйлеров путь", "эйлерова"],
        title: "Эйлеров путь и цикл",
        short:
            "Пройти по каждому ребру ровно один раз, не отрываясь",
        formal:
            "Связный граф: эйлеров цикл    все степени чётные;

```

```

    id: "dijkstra",
    labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс",
    title: "Алгоритм Дейкстры",
    short:
        "Жадный «навигатор»: всегда раскрываем ближайший
        льные.",
    formal:
        "Куча хранит пары (расстояние, вершина). Достали
    complexity: "O(E log V) с бинарной кучей",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "bellman-ford",
    labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман",
    title: "Алгоритм Форда-Беллмана",
    short:
        "Тупой, но честный: V-1 раз проходим по ВСЕМ рёбрам",
    formal:
        "После k-й итерации гарантированно верны все кратчайшие
        ось — есть отрицательный цикл.",
    complexity: "O(V · E)",
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
        "промежуточная вершина", "промежуточную вершину",
    ],
    title: "Флойд-Уоршелл: разрешаем вершину k",
    short:
        "Внешний цикл — это не «шаг», а «какие пересадки можно сделать, чтобы совершить пересадку в k?",
    formal:
        "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок
```

```

    complexity: "O(E log E)",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "components",
    labels: [
        "компонента связности", "компоненты связности", "
        "компоненте связности", "связности", "связный", "
    ],
    title: "Компоненты связности",
    short:
        "Граф-архипелаг: острова, между которыми нет мост
        бой.",
    formal:
        "Считаются одним проходом: пока есть непосещённая
    complexity: "O(V + E)",
    demo: "components",
    simulator: "graph-dfs-bfs",
},
{
    id: "graph",
    labels: [
        "граф", "графа", "графе", "графы", "графов", "гра
        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Горо
    formal:
        "G = (V, E). Ребро бывает ориентированным (улица
    demo: "graph",
    simulator: "graph-dfs-bfs",
},
{
    id: "coord-compress",
    labels: ["сжатие координат", "сжатия координат", "с
    title: "Сжатие координат",

```

```

    ],
    title: "Включения-исключения на префиксных суммах",
    short:
        "Чтобы получить сумму прямоугольника, берём больш
        ый отрезали дважды.",
    formal:
        "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
    complexity: "запрос O(1) после O(nm) предподсчёта",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "reduction",
    labels: ["сведение", "сведения", "сведении", "сведён
    title: "Сведение задач",
    short:
        "«Я не умею решать A, зато умею B». Если A можно
    formal:
        "A ≤p B: есть полиномиальная функция, переводящая
        NP-полную задачу к новой.",
    demo: "np",
},
{
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки"
    title: "Запрос на отрезке",
    short:
        "Спрашиваем не про один элемент, а про кусок массива
        предподсчёты.",
    formal:
        "Суммы – префиксные суммы O(1). Минимум без изменен
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "с
    title: "Строка как массив символов",

```

```

    "Дерево вытянулось в «бамбук»:  $g \rightarrow p \rightarrow x$  идут в о
    м  $(p, x)$ . Именно это вдвое сокращает глубину вс
    formal:
    "Если крутить наивно (два раза поднимать  $x$ ), бамб
    demo: "zig-zig",
    simulator: "splay-rotations",
  },
  {
    id: "zig-zag",
    labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
    title: "Zig-Zag –  $x$  и родитель с разных сторон",
    short:
      "Узлы идут «змейкой»:  $p$  – левый сын  $g$ , а  $x$  – прав
       $p$  и  $g$  становятся двумя детьми  $x$ .",
    formal: "Эквивалентно двойному повороту AVL-дерева
    demo: "zig-zag",
    simulator: "splay-rotations",
  },
];

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e)

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  e.labels.map((label) => ({ label, id: e.id }))
).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 20/79: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМ

```
import { Chapter } from "../types";
```

```
export const graphChapters: Chapter[] = [
  {
```



```
<p class="text-xl font-mono text-white">
</div>

<div class="grid grid-cols-1 xl:grid-cols-2"
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
      rder border-emerald-500 shadow-md">
        Аналогия 1: Позитивное плани
      </div>
      <p class="text-slate-300 text-sm mb
        (нельзя поехать и заработать). Он всегда в
        <div id="slot-chapter-image-dijkstr
        </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
          border-rose-500 shadow-md">
            Ограничения
          </div>
          <p class="text-slate-300 text-sm mb
            охождение улицы), Дейкстра сломается, так к
          </div>
        </div>

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
    -b border-slate-700/50">
      Строгое доказательство (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300"
      <p>Алгоритм использует приоритетную
    g V). Гарантирует корректность только для н
    </div>
  </details>

<div id="slot-dijkstra-viz" class="mt-8"></div>
```

```

        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Отрицательный цикл
        </div>
        <p class="text-slate-300 text-sm mb-1">
            г станет ЕЩЕ короче – значит мы попали во в
        </div>
    </div>

    <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
    id: "floyd",
    title: "Билет 16. Флойд-Уоршелл",
    type: "html",
    category: "Графы",
    content: `<section id="floyd-content" class="mb-12">
        <div class="flex items-center mb-6">
            <span class="bg-blue-600 text-white px-4 py-1 rounded">
                <h2 class="text-3xl font-bold text-white">Флойд
            </div>

            <div class="space-y-8">
                <div class="bg-slate-700/50 p-6 rounded-xl border">
                    <h3 class="text-xl font-bold text-indigo-400">
                        Алгоритм Флойда-Уоршелла
                    </h3>

                    <div class="bg-slate-900 p-4 rounded-lg mb-4">
                        <p class="text-lg text-indigo-300 italic">
                            Алгоритм Флойда-Уоршелла
                        <p class="text-xl font-mono text-white">
                            Алгоритм Флойда-Уоршелла
                        </div>

                        <div class="grid grid-cols-1 xl:grid-cols-2">
                            <!-- Аналогия 1 -->
                            <div class="bg-slate-800 p-6 rounded-lg">
                                <div class="absolute -top-3 left-4

```

```

    <div class="bg-slate-900 p-4 rounded-lg mb-6 text-white">
      <p class="text-sm text-blue-300 italic mb-2">0cc
      <p class="text-lg font-mono text-white">Бор (Терминальные)
    </div>
    <p class="text-slate-300 text-sm">Позволяет найти
      -mono text-amber-300 bg-black/30 px-1 rounded">
      у.</p>
  </div>

  <!-- Аналогии -->
  <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
      <div class="absolute -top-3 left-4 bg-slate-700 text-emerald-500 shadow-md">
        Аналогия 1: Паутина (Бор)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представим, что мы двигаемся по веткам. Если нужно продолжить движение ("нити страховки") в самый длинный из известных путей.
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2 border-slate-800">
      <div class="absolute -top-3 left-4 bg-rose-900 text-rose-500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представим, что мы нашли и "he", потому что оно спрятано внутри терминальные ссылки</b> (term_link) – прямые ссылки на
    </div>
  </div>

  <details class="bg-slate-900/40 rounded-lg border border-slate-800">
    <summary class="p-4 font-bold text-slate-400 outline-slate-500">
      Внутренняя структура автомата и код BFS (Скрыть)
    </summary>
  </details>

```

```

        q.push(child);
    }
}
    </div>
  </div>
</details>
</div>
</section>`
  },
  {
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-1">
<div class="flex items-center mb-6">
  <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-purple-400">
    <p class="text-slate-300 text-sm mb-4">Когда
    ут быть большими дробными числами (долгота
    ву от 0 до N, и уже по ним строим графы.</p>
  </div>
</div>
</section>`
  }
];

```

```

    ],
    correctIndex: 2,
    explanation: "Если нечётных вершин больше 2, эйлерово
        ровно 2 нечётные или 0."
}
],
"bridges-code": [
{
    question: "После DFS у нас low[u] = 4, а tin[v] = 2",
    options: [
        "Да, так как low[u] (4) > tin[v] (2)",
        "Нет, low[u] должно быть меньше или равно",
        "Только если граф связный и неориентированный"
    ],
    correctIndex: 0,
    explanation: "Условие моста: low[u] > tin[v]. В нашем
        случае low[u] = 4, tin[v] = 2, поэтому условие
        выполняется."
},
{
    question: "Для чего нужен массив low[v]?",
    options: [
        "Для того чтобы было больше переменных в коде",
        "Чтобы знать минимальный tin предка, до которого
            нужно идти",
        "Чтобы хранить глубину рекурсии"
    ],
    correctIndex: 1,
    explanation: "low[v] хранит минимальное время входа
        в вершину v во время поиска мостов и точек сочленения."
},
{
    question: "Какова временная сложность алгоритма поиска
        мостов?",
    options: [
        "O(V^2) — ищем мосты в квадрате",
        "O(V + E) — каждый элемент один раз",
        "O(2^V) — полный перебор"
    ],
    correctIndex: 1,
    explanation: "DFS обходит каждую вершину и каждое
        ребро ровно один раз."
}
]
}

```

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 22/79: src/data/tickets/ticket1_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ

```
import { Chapter } from '../..types';

export const tickets1to5: Chapter[] = [
  {
    id: "segment-trees",
    title: "1. Дерево отрезков с операциями на отрезках",
    type: "html",
    description: "Дерево отрезков – структура данных для...",
    category: "Оптимизации и Продвинутое",
    content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>

  <div class="space-y-8">
    <div id="intro" class="bg-slate-700/50 p-6 rounded">
      <h3 class="text-xl font-bold text-blue-400">

      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
omise (lazy).</p>
      </div>

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
    `
  }
];
```

```

    </div>
</section>
\
},
{
  id: "sparse-table",
  title: "2. Двумерная разреженная матрица (Sparse Table)",
  type: "html",
  description: "Разреженная таблица для идемпотентных операций",
  category: "Продвинутые структуры",
  content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные за
            ь отрезок длины 7 (перекрывая друг друга по
            ничего складывать, просто пересекаем куски.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">

```

```

        <p class="text-xl font-mono text-white">
        метода: Split и Merge.</p>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2"
        <div class="bg-slate-800 p-6 rounded-lg"
            <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
                Аналогия 1: Удар катаной (Sp
            </div>
            <p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
            </div>
            <div class="bg-slate-900 p-6 rounded-lg"
                <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
                    Аналогия 2: Слияние капель (M
                </div>
                <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
            </div>
        </div>
    <details class="bg-slate-900/40 rounded-lg l
        <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300"
            <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
    }
}

```



```

        </div>
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000000; border-radius: 10px; padding: 10px; width: 100%; position: relative; height: 100px; background-color: #1e293b; color: white;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md" style="border: 2px solid #f4739c; border-radius: 10px; padding: 5px; text-align: center;">
                Аналогия 2: Балансировка черепахи
            </div>
            <p class="text-slate-300 text-sm mb-0" style="color: #a6b8c8; font-size: 0.9em; margin-bottom: 0;">
                и становиться кривым как бамбук. Но как толстая черепаха
            </p>
        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg" style="background-color: #1e293b; border: 1px solid #000000; border-radius: 10px; padding: 10px; width: 100%;">
        <summary class="p-4 font-bold text-slate-300" style="padding: 5px 10px; color: #a6b8c8; font-weight: bold;">
            -b border-slate-700/50" style="border: 1px solid #334d5d; border-radius: 10px; padding: 5px; text-align: center;">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300" style="padding: 5px 10px; color: #a6b8c8; font-size: 0.9em;">
                <pre class="bg-slate-950 p-4 rounded" style="background-color: #0f172a; border: 1px solid #000000; border-radius: 10px; padding: 10px; width: 100%;">
// Zig, Zag
function rotate(x) {
    let p = x.parent;
    if (p.left === x) { // Right rotate
        p.left = x.right;
        x.right = p;
    } else { // Left rotate
        p.right = x.left;
        x.left = p;
    }
}
} </pre>
            </div>
        </details>

        <div class="bg-indigo-950/60 p-6 rounded-xl" style="background-color: #1e293b; border: 1px solid #000000; border-radius: 10px; padding: 10px; width: 100%;">
            <h4 class="text-lg font-bold text-indigo-300" style="color: #a6b8c8; font-weight: bold; margin-bottom: 10px;">
                Заключение
            </h4>
            <div class="space-y-4 text-sm text-slate-300" style="color: #a6b8c8; font-size: 0.9em;">
                <div>
                    <strong class="text-white block" style="color: white; font-weight: bold; margin-bottom: 10px;">
                        Заключение
                    </strong>
                </div>
            </div>
        </div>
    </div>

```

```

</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
          <p class="text-xl font-mono text-white">
            , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
          </div>
        <div class="grid grid-cols-1 xl:grid-cols-2"
          <div class="bg-slate-800 p-6 rounded-lg"
            <div class="absolute -top-3 left-4 l
              rder border-emerald-500 shadow-md">
                × Аналогия 1: Вырезание прямоу
              </div>
            <p class="text-slate-300 text-sm mb-4">
              з него маленький целевой прямоугольник в цел
              ый верхний угол отрезается дважды! Поэтому
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`
  }
};

```

ФАЙЛ 23/79: src/data/tickets/ticket14_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

```
import { Chapter } from '../types';
```

```
export const tickets14to20: Chapter[] = [
```

```
{
```

```
  id: "dijkstra",
```

```
  title: "14. Графы. Поиск кратчайшего пути. Дейкстра
```

```

</section>`
  },
  {
    id: "bellman-ford",
    title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="bellman-ford-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded-md">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">15. Графы. Поиск кратчайшего пути. Форд-Беллман
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-rose-400">15.1. Алгоритм Форда-Беллмана
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана — это алгоритм поиска кратчайшего пути в графе с весами ребер. Он работает за время  $O(V^2)$ , где  $V$  — количество вершин в графе. Алгоритм работает на графах с отрицательными весами ребер, но не на графах с отрицательными циклами. Алгоритм работает на графах с отрицательными весами ребер, но не на графах с отрицательными циклами. Алгоритм работает на графах с отрицательными весами ребер, но не на графах с отрицательными циклами.
        <p class="text-xl font-mono text-white">
          1. Инициализация: для каждой вершины  $v$  установить кратчайший путь  $d[v] = \infty$ , кроме вершины источника  $s$ , для которой  $d[s] = 0$ .
          2. Для каждой вершины  $u$  в графе:
              <div class="grid grid-cols-1 xl:grid-cols-2">
                <div class="bg-slate-800 p-6 rounded-lg">
                  <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                    ♂ Аналогия 1: Параноик
                  </div>
                  <p class="text-slate-300 text-sm mb-2">Параноик проверяет ВСЕ возможные дороги V-1 раз подряд.
                </div>
                <div class="bg-slate-900 p-6 rounded-lg">
                  <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                    Отрицательный цикл
                  </div>
                  <p class="text-slate-300 text-sm mb-2">Параноик попал во временную петлю.
                </div>
              </div>
            </div>
          3. Если для какой-либо вершины  $u$  удалось улучшить кратчайший путь, то повторить шаг 2.
          4. Если для всех вершин  $u$  кратчайший путь не улучшился, то алгоритм завершается.
        </p>
      </div>
    </div>
  </div>
`
  }
}

```

```

        <li><b>Шаг k=2:</b> Разрешаем пересадки ч
        внутри этих путей уже могут быть пересадки ч
        <li>...</li>
        <li><b>Шаг k=N:</b> Ты проверил все
    /li>
  </ul>
</div>
</div>
</div>
</section>`
  },
  {
    id: "johnson-algo",
    title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-amber-400
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-xl font-mono text-white"
      </div>
      <div class="grid grid-cols-1 gap-6 mb-6">
        <div class="bg-slate-800 p-6 rounded-lg
          <div class="absolute -top-3 left-4 l
          er border-amber-500 shadow-md">
            ☞ Суть перевзвешивания
          </div>
          <p class="text-slate-300 text-sm mb
            Дейкстра (Билет 14) быстрая, но
            ает очень медленно. Мы хотим сделать <b>Джон
            е пути остались теми же.

```

```

<div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
    Аналогия: Прокладка интернет
</div>
<p class="text-slate-300 text-sm mb
с самых дешевых дорог в стране". Он строит
единую сеть.</p>
</div>
</div>
</div>
</section>`
},
{
  id: "mst-prima",
  title: "19. Графы. Остовное дерево. Прима",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-v
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-emerald-4
      <div class="bg-slate-900 p-4 rounded-lg mb-1
        <p class="text-lg text-emerald-300 ital
        <p class="text-xl font-mono text-white"
        которое торчит из посещенных в непосещенных
      </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
        rder border-emerald-500 shadow-md">
          ✦ Аналогия: Заражение вирусом
        </div>

```

```
        </div>
      </div>
    </div>
  </section>`
  }
};
```

ФАЙЛ 24/79: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМ

```
import { Chapter } from '../..types';

export const tickets21to24: Chapter[] = [
  {
    id: "string-kmp",
    title: "21. Префикс-функция (π) — Взгляд назад (КМП)",
    type: "html",
    category: "Строки",
    content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        сом, оканчивающимся в позиции i.</p>
      </div>

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
```

```

>
        <li><b>abc</b>: Опять мимо. <code>
        <li><b>abca</b>: 0! Начало ("a"
</code></li>
        <li><b>abcab</b>: Начало ("ab")
/li>
        <li><b>abcabc</b>: Начало ("abc
e></li>
        <li><b>abcabcd</b>: "d" всё исп
"abca" vs "abcd"). Конец "abcd" не совпадает
</ul>
</div>
</details>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код C++ (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`
},
{
    id: "string-z-func",
    title: "22. Z-функция – Сдвиг и наложение",
    type: "html",
    category: "Строки",

```

[i]</code> (где <i>i</i> указывает назад на сама себя.

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg l
<summary class="p-4 font-bold text-slate
-b border-slate-700/50">

Пример вычисления (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

<p>Тот же пример: abcabcd</p>

<ul class="list-disc pl-5 space-y-2

<code>Z[0]</code> – не счита

<code>Z[1]</code> (bcabcd

[1] = 0</code>

<code>Z[2]</code> (cabcd

<code>Z[3]</code> (abcd<

? Нет, в начале 'a', тут 'd'). Совпало 3 бу

<code>Z[4], Z[5], Z[6]</code>

</div>

</details>

<details class="bg-slate-900/40 rounded-lg l
<summary class="p-4 font-bold text-slate
-b border-slate-700/50">

Код C++ (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

<pre class="bg-slate-950 p-4 rounded

int l = 0, r = 0;

for (int i = 1; i < n; i++) {

if (i <= r) z[i] = min(r - i + 1, z[i - l]);

while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z

if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }

}</pre>

ет — мы падаем по суффиксной ссылке ("

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2"
  <div class="absolute -top-3 left-4 bg-rose-900
    -500 shadow-md">
```

Аналогия 2: Матёшка (Терминальные)

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Представ
  нашли и "he", потому что оно спрятано внутри
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border b
  <summary class="p-4 font-bold text-slate-400 ou
    r-slate-700/50">
```

Код (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-
```

```
<pre class="bg-slate-950 p-4 rounded overfl
```

```
int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch)
  }
  return t[v].link;
}
```

```
int go(int v, char c) {
  if (t[v].go[c] == -1) {
    if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
    else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
  }
  return t[v].go[c];
}</pre>
```

```
</div>
```

```
</details>
```

```
</div>
```

```
        <p class="text-slate-300 text-sm mb-6">
          ь отличный переводчик на английский (Сведения о
          аче В, то В – это <b>NP-Полная</b> (сосредоточена на
        </div>
      </div>
    </div>
  </div>
</section>`
  }
};
```

ФАЙЛ 25/79: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
  {
    id: "graph-dfs-bfs",
    title: "6. Графы. Представление, DFS, BFS",
    type: "html",
    description: "Представление графов и базовые обходы",
    category: "Графы. База",
    content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">6. Графы. Представление, DFS, BFS
  </div>
  <div class="space-y-8">
    {/* База представлений */}
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-slate-300">Представление графов
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-slate-300 italic">Графы – это структуры данных, состоящие из
    </div>
  </div>
</section>`
  }
];
```

```

        </p>
        <ul class="text-xs text-indigo-300">
          <li><b>Где используется:</b></li>
          <li>Топологическая сортировка (DFS)
          <li>Поиск мостов и точек сочленения
          <li>Сильно связанные компоненты Косараса
          <li>Поиск циклов и просто проверка
        </ul>
      </div>

      <!-- Аналогия BFS -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
          BFS (Поиск в ширину) = Волна
        </div>
        <p class="text-slate-300 text-sm mb-2">
          <b>Что делает:</b> Концентрические волны
        </p>
        <p>Работает через Очередь (FIFO). Продвигается в стороны</p>
        <ul class="text-xs text-emerald-300">
          <li><b>Где используется:</b></li>
          <li>Кратчайший путь в Невзвешенном графе
          <li>Алгоритм Ахо-Корасик (сборка слов)
          <li>Алгоритм Форда-Фалкерсона / Поиск в ширину
          <li>Двунаправленный поиск для ускорения
        </ul>
      </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg">
      <summary class="p-4 font-bold text-slate-300">
        -b border-slate-700/50">
        Код обходов C++ (Основа)
      </summary>
      <div class="grid grid-cols-1 md:grid-cols-2">
        <pre class="bg-slate-950 p-3 rounded">
// DFS (Рекурсия)

```



```

<h3 class="text-xl font-bold text-blue-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-blue-300 italic">
    <p class="text-xl font-mono text-white">
перед.</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Учеба в вузе
        </div>
        <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
>
    </div>
    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
            Ограничения: Циклы
        </div>
        <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg">
    <summary class="p-4 font-bold text-slate-300">
-b border-slate-700/50">
        Душный мод: Код на C++ (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <p class="text-indigo-400 font-bold">
        <div class="bg-slate-950 p-4 rounded">
            <pre class="text-xs font-mono">

```

```

vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
  <h3 class="text-xl font-bold text-emerald-400">Связь с Билетом 12

```

```

  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-emerald-300 italic">
      <div class="text-left font-mono text-sm">
        <p><strong class="text-white">Компонент ориентированного графа, в котором
      </p>
      </div>
    </p>
  </div>
```

```

  <p class="text-slate-300 text-sm mb-4">
    <strong>«Это просто цикл?» – Нет!</strong>
  </p>
  <ul class="list-disc list-inside text-sm text-slate-300">
    <li>Если у тебя есть цикл <code>А → Б → А</code>
    <li>Если ты добавишь вершину <strong>Г</strong> к трем вершинам – одна большая SCC</strong>.
    <li>Правило: <b>Если два цикла имеют хотя бы одну вершину, то они образуют один цикл</b>.
  </ul>
```

```

  <div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
        Аналогия: Интриги в офисе
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Сильная связность – это "клуб сидящих" – все сидит
        ни в одной SCC. Добавить курьера Гошу, который может
        йти наружу (к директору), но обратно в клуб не может.
      </p>
    </div>
```

```

    <div class="bg-slate-900 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
```

```

<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
          </div>
          <p class="text-slate-300 text-sm mb-4">
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    description: "Вершины, удаление которых убивает свя",
    category: "Графы. Продвинутое",
    content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

        <div class="bg-slate-900 p-6 rounded-lg border-
            <div class="absolute -top-3 left-4 bg-emerald-500 shadow-md">
                Телеком: Центральный свитч
            </div>
            <p class="text-slate-300 text-sm mb-4">
                У тебя несколько компьютеров в одной
                абелем (мостом). Сами свитчи — это точки со
            </p>
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border-
        <summary class="p-4 font-bold text-slate-400
            order-slate-700/50">
                Душный мод: Код и корень DFS (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300 space-y-2">
                <p class="text-blue-400 font-bold">Особый случай:
                <p>
                    Для стартовой вершины обхода правил
                    то выше неё прыгать некуда). Поэтому для неё
                    у него <strong>больше 1 независимого ребенка.
                </p>
                <div class="bg-slate-950 p-4 rounded-lg border-
                    <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);

```



```

        category: "Графы",
        content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрыв
          </div>
          <p class="text-slate-300 text-sm mb-4">
            е сходится нечетное число линий, значит, из
            а везде должно быть четное число (зашел-вышел)
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
];

```

=====

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

=====

```
// Adjacency list
const adj: number[][] = Array.from(
  { length: Object.keys(nodes).length },
  () => [],
);
edges.forEach((e) => {
  adj[e.u].push(e.v);
  adj[e.v].push(e.u);
});

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const newSteps: StepInfo[] = [];
    let timer = 0;
    const visited = new Set<number>();
    const ap = new Set<number>();
    const currentTin = Array(nodes.length).fill(-1);
    const currentLow = Array(nodes.length).fill(-1);

    const recordStep = (desc: string, u: number | null) => {
      newSteps.push({
        desc,
        visited: new Set(visited),
        ap: new Set(ap),
      });
    };
  });
}
```

```

        v,
      );
    }
  }
}

if (p === -1 && children > 1) {
  ap.add(v);
  recordStep(
    `Корень дерева DFS ${v} имеет >1 детей. Это
    v,
  );
} else if (p === -1) {
  recordStep(
    `Корень дерева DFS ${v} имеет <=1 ребенка, он
    v,
  );
}
};

for (let i = 0; i < nodes.length; i++) {
  if (!visited.has(i)) {
    dfs(i);
  }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
  desc: "",
  visited: new Set(),
  ap: new Set(),
  currentNode: null,
  tin: [],
  low: [],
};

```

```

        {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
      onClick={reset}
      className="flex items-center justify-center
        ors"
    >
      <RotateCcw className="w-4 h-4" />
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3 g
  <div className="lg:col-span-2 relative h-96 lg:l
    <svg className="w-full h-full" viewBox="0 0 8
      { /* Edges */ }
      {edges.map((edge, i) => {
        const u = nodes.find((n) => n.id === edge
        const v = nodes.find((n) => n.id === edge

        const isBridge =
          (edge.u === 3 && edge.v === 4) ||
          (edge.u === 4 && edge.v === 3);

        return (
          <line
            key={"edge-" + i}
            x1={u.x}
            y1={u.y}
            x2={v.x}
            y2={v.y}
            stroke={isBridge ? "#f43f5e" : "#475568"}
            strokeWidth={isBridge ? "4" : "2"}
            strokeDasharray={isBridge ? "8,4" : "1"}
          />
        );
      })}
    </div>
  </div>
</div>

```

```

        textAnchor="middle"
        fill="white"
        fontSize="14px"
        fontWeight="bold"
        className="select-none pointer-events-none"
    >
        {node.label}
    </text>

    {isVisited && (
        <
            <text
                x={node.x}
                y={node.y - r - 15}
                textAnchor="middle"
                fill="#94a3b8"
                fontSize="10px"
            >
                tin:{" "}
                {currentStep.tin[node.id] >= 0
                    ? currentStep.tin[node.id]
                    : "-"}
            </text>
            <text
                x={node.x}
                y={node.y + r + 20}
                textAnchor="middle"
                fill="#94a3b8"
                fontSize="10px"
            >
                low:{" "}
                {currentStep.low[node.id] >= 0
                    ? currentStep.low[node.id]
                    : "-"}
            </text>
        </>
    )}
</g>

```

```

<h5 className="text-xs text-slate-500 font-medium">
  ЛОГ ДЕЙСТВИЙ:
</h5>
<div className="space-y-2">
  {steps
    .slice(0, currentStepIndex + 1)
    .reverse()
    .map((step, idx) => {
      <div
        key={idx}
        className={`text-xs p-2 rounded $
xt-slate-500`} `>
        >
          {step.desc}
        </div>
      </div>
    })}
  </div>
</div>
<div className="mt-4 shrink-0">
  <div className="w-full bg-slate-800 h-2 rounded">
    <div
      className="bg-rose-500 h-full transition"
      style={{
        width: `${(currentStepIndex / Math.max(
          steps.length, 1
        )) * 100}%`
      }}
    />
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>
);
};

```

```

        { u: 'A', v: 'C', w: 1 },
        { u: 'C', v: 'D', w: 3 },
        { u: 'C', v: 'E', w: 4 },
        { u: 'D', v: 'F', w: 2 },
        { u: 'E', v: 'F', w: 1 },
    ];
    if (cycle) {
        return [...defaultEdges, { u: 'D', v: 'A', w: -6 }
    ] else {
        return [...defaultEdges, { u: 'D', v: 'A', w: -2 }
    ]
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
    const list: { [key: string]: { v: string; w: number } } = {};
    nodes.forEach(n => (list[n.id] = []));
    ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
    return list;
};
const adjList = getAdjList(edges);

const codeLines = [
    { line: 1, text: 'function bellman_ford(graph, start)' },
    { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
    { line: 3, text: '    for i from 1 to V - 1: // |V| - 1 iterations' },
    { line: 4, text: '        for (u, v, weight) in E:' },
    { line: 5, text: '            if dist[u] + weight < dist[v]' },
    { line: 6, text: '                dist[v] = dist[u] + weight' },
    { line: 7, text: '    for (u, v, weight) in E: // Проверка на отрицательный цикл' },
    { line: 8, text: '        if dist[u] + weight < dist[v]' },
    { line: 9, text: '            return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
    { line: 10, text: '    return dist' },
];

useEffect(() => {
    const simulationSteps: Step[] = [];

```

```

        log: ` Ранний выход после итерации ${iter}.
        activeCodeLine: 10
    });
    break;
}
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
    let cycleFound = false;
    for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
            cycleFound = true;
            simulationSteps.push({
                iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
                distances: { ...dist }, previous: { ...prev },
                log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}
                activeCodeLine: 9,
            });
            break;
        }
    }

    if (!cycleFound) {
        simulationSteps.push({
            iteration: vCount, checkingEdge: null,
            distances: { ...dist }, previous: { ...prev },
            log: ` Проверка завершена. Ни одно ребро не попало в цикл.
            activeCodeLine: 10,
        });
    }
}

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {

```



```

        s">Шаг    </button>
    </div>
</div>

<div className="flex flex-col lg:flex-row gap-6 md:gap-10" >
  {/* Граф */}
  <div className="w-full lg:w-2/3 bg-blue-950/20 rounded-2xl
    y-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-blue-950/20
      ld shadow-sm">
      Итерация {currentStep.iteration} (Шаг {currentStep.step})
    </div>

    {currentStep.hasNegativeCycle && (
      <div className="absolute top-3 right-3 bg-rose-500/50
        animate-pulse shadow-lg shadow-rose-600/50" >
        ⚠ Отрицательный цикл!
      </div>
    )}

  </div>
  <svg className="w-full h-auto max-h-[500px]" >
    <defs>
      <marker id="arrow-bf-normal" markerWidth="10" markerHeight="10"
        <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
      <marker id="arrow-bf-active" markerWidth="10" markerHeight="10"
        <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
      <marker id="arrow-bf-cycle" markerWidth="10" markerHeight="10"
        <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
    </defs>

    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id === edge.u);
      const vNode = nodes.find((n) => n.id === edge.v);
      const isChecking = currentStep.checkingEdges === idx;
      const isNeg = edge.w < 0;
      const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));
    })}
  </svg>
</div>

```



```
import { useState, useEffect, useCallback } from 'react'
import { Play, Pause, RotateCcw, StepForward } from 'lu
```

```
const NODES = [
  { id: 0, label: '0', x: 300, y: 50 },
  { id: 1, label: '1', x: 150, y: 150 },
  { id: 2, label: '2', x: 450, y: 150 },
  { id: 3, label: '3', x: 75, y: 250 },
  { id: 4, label: '4', x: 225, y: 250 },
  { id: 5, label: '5', x: 375, y: 250 },
  { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
  { from: 0, to: 1 }, { from: 0, to: 2 },
  { from: 1, to: 3 }, { from: 1, to: 4 },
  { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
  0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};
```

```
interface Step {
  activeLine: number;
  queue: number[];
  visited: number[];
  currentNode: number | null;
  logs: string;
}
```

```
const generateBfsSteps = (): Step[] => {
  const steps: Step[] = [];
  const queue: number[] = [];
  const visited = new Set<number>();

  // Init
```

```

        for (const n of neighbors) {
            if (!visited.has(n)) {
                steps.push({
                    activeLine: 7,
                    queue: [...queue],
                    visited: Array.from(visited),
                    currentNode: curr,
                    logs: `Рассматриваем соседа: ${n}. Он еще не
                });

                visited.add(n);
                queue.push(n);

                steps.push({
                    activeLine: 9,
                    queue: [...queue],
                    visited: Array.from(visited),
                    currentNode: curr,
                    logs: `Помечаем ${n} как посещенную и добав.
                });
            }
        }
    } else {
        steps.push({
            activeLine: 6,
            queue: [...queue],
            visited: Array.from(visited),
            currentNode: curr,
            logs: `Соседей нет или все уже посещены.`
        });
    }
}

steps.push({
    activeLine: 12,
    queue: [],
    visited: Array.from(visited),
    currentNode: null,

```

```

        <span className="text-blue-400"> </span> Ал
    </h3>
    <p className="text-slate-400 text-sm mt-1">Ви
</div>
<div className="flex bg-slate-800 p-1.5 rounded
    <button
        onClick={() => { setIsPlaying(false); setSt
        className="p-2 text-slate-400 hover:text-wh
        title="Сброс"
    >
        <RotateCcw className="w-5 h-5" />
    </button>
    <button
        onClick={() => setIsPlaying(p => !p)}
        className="p-2 text-blue-400 hover:text-bl
        title={isPlaying ? 'Пауза' : 'Воспроизведен
    >
        {isPlaying ? <Pause className="w-5 h-5" />
    </button>
    <button
        onClick={handleNext}
        disabled={stepIdx >= STEPS.length - 1}
        className="p-2 text-emerald-400 hover:text-
        ed:hover:bg-transparent"
        title="Шаг вперед"
    >
        <StepForward className="w-5 h-5" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-2 g
    {/* Визуал графа */}
    <div className="bg-slate-800 border border-slate
        50px]">
        <svg className="w-full h-full min-h-[350px]" *
            {/* Рёбра */}
            {EDGES.map((e, idx) => {

```

```

<g key={n.id} className="transition-tran
  {isCurrent && {
    <circle cx={n.x} cy={n.y} r="30" fill
  }}
  <circle
    cx={n.x} cy={n.y}
    r="20"
    fill={fill}
    stroke={stroke}
    strokeWidth="3"
    className="transition-colors duration
  />
  <text
    x={n.x} y={n.y}
    textAnchor="middle"
    dy=".3em"
    className="fill-white font-bold text
  >
    {n.label}
  </text>
</g>
  );
  }}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800
    er">
    <div className="bg-slate-800 text-slate-400 font-mono
      se tracking-wider">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">Mar {step}
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },

```

```
        <div className="mt-2 text-sm text-amber-300
ms-center justify-center">
            "{step.logs}"
        </div>
    </div>
</div>
</div>
</div>
);
}
```

ФАЙЛ 29/79: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string }> = {
  dijkstra: {
    src: dijkstraImg,
    alt: 'Добрый Дейкстра',
    caption: 'Добрый — только позитив!',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Фура по болоту',
    caption: 'Тяжёлая, но ей пофиг на ямы!',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
```

```

}

/**
 * Переходы «предыдущая / следующая тема» — как на Code
 * Компактный вариант живёт в шапке главы, крупный — по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next }) => {
  if (compact) {
    return (
      <div className="flex items-center gap-2 text-xs">
        <button
          disabled={!prev}
          onClick={() => prev && onGo(prev.id)}
          title={prev ? prev.title : "Это первая тема"}
          className="flex items-center gap-1 px-2.5 py-1
            :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
        >
          <ChevronLeft className="w-4 h-4" />
          <span className="hidden sm:inline">Назад</span>
        </button>
        <span className="text-slate-500 font-mono tabular-nums" />
        {index + 1} / {total}
      </span>
      <button
        disabled={!next}
        onClick={() => next && onGo(next.id)}
        title={next ? next.title : "Это последняя тема"}
        className="flex items-center gap-1 px-2.5 py-1
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
      >
        <span className="hidden sm:inline">Вперёд</span>
        <ChevronRight className="w-4 h-4" />
      </button>
    </div>
  )
  }
  return (

```



```
import React, { useCallback, useEffect, useRef, useState } from 'react';
import { Maximize2, MousePointerClick, Sparkles } from 'react-icons/sf';
import type { Chapter } from '../types';
import { useTermHints } from '../hooks/useTermHints';
import { TermPopover, type HintAnchor } from './hints/TermPopover';
import { getViz } from './vizRegistry';
import { ChapterNav } from './ChapterNav';
```

```
interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  onOpenSimulator: (vizId: string) => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```
export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator }) => {
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
```

```
  useTermHints(contentRef, [chapter.id]);
```

```
  const cancelHide = useCallback(() => {
    if (hideTimer.current) {
      window.clearTimeout(hideTimer.current);
      hideTimer.current = null;
    }
  }, []);
```

```
  const scheduleHide = useCallback(() => {
    cancelHide();
  }, [cancelHide]);
```

```

    const el = (e.target as HTMLElement).closest<HTMLElement>
    if (el) open(el);
  };

const viz = getViz(chapter.id);

return (
  <>
    <article className="chapter-body">
      <div
        ref={contentRef}
        className="prose prose-invert max-w-none"
        onMouseOver={handlePointerOver}
        onMouseOut={handlePointerOut}
        onClick={handleClick}
        onFocus={handleFocus}
        dangerouslySetInnerHTML={{ __html: chapter.content }}
      />

      <p className="mt-6 flex items-start gap-2 text-
        ed-lg px-3 py-2">
        <MousePointerClick className="w-4 h-4 shrink-0" />
        <span>
          Подчёркнутые термины – интерактивные: наведи
          объяснение на пальцах, мини-анимацию и кнопки
        </span>
      </p>

      {viz && (
        <section id="chapter-viz" className="mt-8 scroll-mt-8">
          <div className="flex items-center justify-between">
            <h3 className="flex items-center gap-2 text-
              ed-lg">
              <Sparkles className="w-4 h-4 text-indigo-500" />
              Демонстрация: {viz.title}
            </h3>
            <button
              onClick={() => onOpenSimulator(chapter.id)}
              className="flex items-center gap-1.5 text-sm font-medium">

```

```

    highlight: string;
}

const CODE_LINES: CodeLine[] = [
    { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
    { line: "    visited[v] = true;", highlight: "normal" },
    { line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
    { line: "    ", highlight: "normal" },
    { line: "    for (int to : adj[v]) {", highlight: "normal" },
    { line: "        if (to == p) continue;", highlight: "normal" },
    { line: "        ", highlight: "normal" },
    { line: "        if (visited[to]) {", highlight: "normal" },
    { line: "            low[v] = min(low[v], tin[to]);", highlight: "normal" },
    { line: "        } else {", highlight: "normal" },
    { line: "            dfs(to, v);", highlight: "normal" },
    { line: "            low[v] = min(low[v], low[to]);", highlight: "normal" },
    { line: "        ", highlight: "normal" },
    { line: "        if (low[to] > tin[v]) {", highlight: "normal" },
    { line: "            // ЭТО МОСТ!", highlight: "normal" },
    { line: "        }", highlight: "normal" },
    { line: "    }", highlight: "normal" },
    { line: "}", highlight: "normal" },
];

interface DfsStep {
    currentNode: number | null;
    visitedIndices: number[];
    tin: Record<number, number>;
    low: Record<number, number>;
    stack: number[];
    bridges: string[];
    log: string;
    activeEdge: [number, number] | null;
    backEdge: [number, number] | null;
    activeCodeLine: number[];
}

```

```

tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1
activeEdge: [1,6], backEdge: null,
log: "Из В идём в G. tin[G]=3, low[G]=3.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 1, visitedIndices: [0,1,6],
tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1
activeEdge: null, backEdge: null,
log: "У G нет соседей (кроме В). Возврат. low[G]=3 :
activeCodeLine: [13,14,15]
},
{
currentNode: 2, visitedIndices: [0,1,6,2],
tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, sta
activeEdge: [1,2], backEdge: null,
log: "Из В идём в С. tin[C]=4, low[C]=4.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 2, visitedIndices: [0,1,6,2],
tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
activeEdge: null, backEdge: [2,0],
log: "Видим соседа А (уже посещён)! Обратное ребро
activeCodeLine: [7,8,9]
},
{
currentNode: 3, visitedIndices: [0,1,6,2,3],
tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
activeEdge: [2,3], backEdge: null,
log: "Идём С→D. tin[D]=5, low[D]=5.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 4, visitedIndices: [0,1,6,2,3,4],
tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
activeEdge: [3,4], backEdge: null,
log: "D→E. tin[E]=6, low[E]=6.",

```

```

        currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
        activeEdge: null, backEdge: null,
        log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,G) - мост",
        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
        activeEdge: null, backEdge: null,
        log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D)",
        activeCodeLine: [11,13,16,17,18]
    },
    {
        currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
        activeEdge: null, backEdge: null,
        log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм завершён.",
        activeCodeLine: []
    }
];

```

```

export function DfsBridgesSimulator() {
    const [dfsIdx, setDfsIdx] = useState(0);
    const [dfsAuto, setDfsAuto] = useState(false);
    const dfsTimer = useRef<NodeJS.Timeout | null>(null);

    useEffect(() => {
        if (dfsAuto) {
            dfsTimer.current = setInterval(() => {
                setDfsIdx(prev => {
                    if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
                    return prev + 1;
                });
            }, 2500);
        }
        return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
    }, [dfsAuto]);
}

```

```

const u = GRAPH_NODES.find(n => n.id === e.u);
const v = GRAPH_NODES.find(n => n.id === e.v);
const bridge = step.bridges.includes(e.key);
const activeTree = step.activeEdge && (
  (step.activeEdge[0]===e.u && step.activeEdge[1]===e.v) ||
  (step.activeEdge[0]===e.v && step.activeEdge[1]===e.u)
);
const activeBack = step.backEdge && (
  (step.backEdge[0]===e.u && step.backEdge[1]===e.v) ||
  (step.backEdge[0]===e.v && step.backEdge[1]===e.u)
);
let sc = "#475569", sw = 2, dash = "none";
if (bridge) { sc = "#eab308"; sw = 4; }
else if (activeTree) { sc = "#10b981"; sw = 2; }
else if (activeBack) { sc = "#f43f5e"; sw = 2; }
else if (step.visitedIndices.includes(e.u) || step.visitedIndices.includes(e.v)) { sc = "#444"; sw = 2; }
return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
  stroke={sc} strokeWidth={sw} strokeDash={dash}
  className="transition-all duration-500"/>
}}}
{step.currentNode !== null && step.currentNode.id === e.u ? e.v : e.u}
const n = GRAPH_NODES.find(x => x.id === e.u);
if (!n) return null;
return <circle cx={n.x} cy={n.y} r={18} fill={cur} stroke={st}/>
>;
}}())}
{GRAPH_NODES.map(n => {
  const cur = step.currentNode === n.id;
  const vis = step.visitedIndices.includes(n.id);
  const st = step.stack.includes(n.id);
  return <g key={n.id}>
    <circle cx={n.x} cy={n.y} r={12}
      fill={cur ? "#10b981" : st ? "#064e3b" : "#444"}
      stroke={cur ? "#fff" : st ? "#34d399" : "#444"}
      strokeWidth={2.5}
      className="transition-all duration-300"/>
    <text x={n.x} y={n.y+3} textAnchor="middle">{n.label}</text>
  </g>
)}

```

```

        </pre>
      </div>
    </div>

    { /* Stack */
    <div className="bg-slate-900/50 rounded-xl bo
      <div className="text-[10px] font-bold text-
        <span>Стек DFS</span>
        <span className="text-indigo-400">{step.s
      </div>
      <div className="flex flex-col-reverse gap-1
        {step.stack.length === 0
          ? <div className="text-[10px] text-slate
            : step.stack.map((id, i) => {
              <div key={id} className={`px-2 py-0
                i === step.stack.length-1
                  ? "bg-emerald-600/20 text-emera
                  : "bg-slate-900 text-slate-400 l
              }`}>Вершина {GRAPH_NODES.find(n=>n.
                <span className="text-slate-500 m
              </div>
            })
          }
        </div>
      </div>
    </div>

    { /* Log */
    <div className="bg-indigo-950/10 border borde
      <p className="text-[11px] text-slate-300 le
      <div className="mt-2 pt-2 border-t border-s
        <span className="text-slate-400">Мосты:</
        <span className="font-mono font-bold text
          {step.bridges.length > 0 ? step.bridges
        </span>
      </div>
    </div>
  </div>
</div>

```

```

    { u: 'C', v: 'B', w: 1 },
    { u: 'C', v: 'G', w: 3 },
    { u: 'C', v: 'E', w: 8 },
    { u: 'B', v: 'G', w: 4 },
    { u: 'B', v: 'D', w: 5 },
    { u: 'G', v: 'D', w: 1 },
    { u: 'G', v: 'E', w: 2 },
    { u: 'D', v: 'F', w: 3 },
    { u: 'E', v: 'F', w: 1 },
  ];

```

```

const adjList: { [key: string]: { v: string; w: number } } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
  adjList[e.u].push({ v: e.v, w: e.w });
});

```

```

const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push(start, 0)' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbors(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push(v, dist[v])' },
  { line: 11, text: '    return dist // Все кратчайшие расстояния найдены' },
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };

```



```

        log: `  Смотрим соседа ${v}. Текущее расстояние
              ${dist[u] + edge.w}.`,
        activeCodeLine: 8,
    ));

    if (dist[u] + edge.w < dist[v]) {
        dist[v] = dist[u] + edge.w;
        prev[v] = u;
        simulationSteps.push({
            currentNode: u, checkingEdge: { u, v },
            distances: { ...dist }, visited: { ...visited },
            log: `  Улучшение (Релаксация)! Новое кратчайшее
                  расстояние до ${v} равно ${dist[v]}.`,
            activeCodeLine: 9,
        });
    }
}

simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: `  Дейкстра всё доставил! Все возможные кратчайшие
          расстояния найдены!`,
    activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => {
            setCurrentStepIndex((prev) => prev + 1);
        }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [currentStepIndex, isPlaying]);

```

```

        disabled={currentStepIndex >= steps.length}
        className="flex items-center gap-1 bg-indigo-950/20
        px-3 py-2 rounded-lg text-sm font-medium tr
      >
        Шаг
      </button>
    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6 m
    {/* Граф */}
    <div className="w-full lg:w-2/3 bg-indigo-950/20
      justify-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-indigo-950/20
        font-bold shadow-sm">
        Шаг {currentStepIndex + 1} из {steps.length}
      </div>

      <svg className="w-full h-auto max-h-[500px]" >
        <defs>
          <marker id="arrow-dh-normal" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
          </marker>
          <marker id="arrow-dh-active" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
          </marker>
          <marker id="arrow-dh-opt" markerWidth="6"
            <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
          </marker>
        </defs>
        {/* Пёпса */}
        {edges.map((edge, idx) => {
          const uNode = nodes.find((n) => n.id ===
          const vNode = nodes.find((n) => n.id ===
          const isChecking =
            currentStep.checkingEdge &&
            (currentStep.checkingEdge.u === edge.u &

```

```

        return (
          <g key={node.id} className="transition-
            <circle
              cx={node.x} cy={node.y}
              r={isCurrent ? 22 : 18}
              fill={isCurrent ? '#4f46e5' : isVis
              stroke={isCurrent ? '#a5b4fc' : isV
              strokeWidth={isCurrent ? 4 : 2}
              className="transition-all duration-
            />
            <text x={node.x} y={node.y + 5} fill=
              {node.id}
            </text>
            <text x={node.x} y={node.y - 28} fill
"middle" className="bg-indigo-950 px-1 py-0
              {dist === 999 ? '∞' : `d=${dist}`}
            </text>
            <text x={node.x} y={node.y + 35} fill
              {node.name.split(' ')[0]}
            </text>
          </g>
        );
      }}}
    </svg>
    <div className="w-full bg-slate-950/80 p-4 ro
      <p className="font-semibold text-amber-400
      <p className="min-h-[40px] flex items-cente
    </div>
  </div>

  { /* Список смежности */ }
  <div className="w-full lg:w-1/3 bg-emerald-950/
    <h4 className="text-lg font-bold text-emerald
    <div className="space-y-2.5 flex-1 overflow-y
      {Object.keys(adjList).map((u) => {
        const isCurrentNode = currentStep.current
        return (

```

```

<div className="overflow-x-auto">
  <table className="w-full text-left border">
    <thead>
      <tr className="border-b border-rose-900">
        <th className="py-2 px-3">Вершина</th>
        <th className="py-2 px-3">Расстояние</th>
        <th className="py-2 px-3">Из (пред. шаг)</th>
      </tr>
    </thead>
    <tbody className="text-sm">
      {nodes.map((n) => {
        const dist = currentStep.distances[n.id]
        const prev = currentStep.previous[n.id]
        const vis = currentStep.visited[n.id]
        const isCurr = currentStep.currentNode === n.id

        return (
          <tr key={n.id} className={`border-b border-rose-900 ${
            isCurr ? 'bg-indigo-950/60 font-medium' : ''
          }`}>
            <td className="py-2.5 px-3 flex items-center">
              <span className={`w-2 h-2 rounded-full ${vis ? 'bg-emerald-500' : 'bg-gray-300'}`} />
              {n.name}
            </td>
            <td className="py-2.5 px-3 font-medium">
              {dist === 999 ? '∞' : dist}
            </td>
            <td className="py-2.5 px-3 text-gray-500">
              {prev || '-'}
            </td>
          </tr>
        )
      })}
    </tbody>
  </table>
</div>
</div>
</div>

```

```

    n: number;
    action: 'call' | 'memo_hit' | 'base_case' | 'calc' |
    desc: string;
    codeLine: number;
    memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "    if (n in memo) return memo[n];",
  /* 3 */ "    if (n <= 2) return 1;",
  /* 4 */ "    memo[n] = fibMemo(n - 1, memo) + fibMemo",
  /* 5 */ "    return memo[n];",
  /* 6 */ "}";
];

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |

  useEffect(() => {
    const generatedSteps: DPStep[] = [];
    let stepId = 0;
    const memoMap: { [key: number]: number } = {};

    function simulateFib(n: number): number {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вызов fibMemo(${n}). Проверяем наличие в
        codeLine: 2,
        memoState: { ...memoMap }
      });
    }
  });
}

```

```

        generatedSteps.push({
          id: stepId++,
          n,
          action: 'calc',
          desc: `Сохраняем в кэш: мемо[${n}] = ${res1} + ${res2}`,
          codeLine: 5,
          memoState: { ...memoMap }
        });

        return memoMap[n];
      }

const finalRes = simulateFib(targetN);

generatedSteps.push({
  id: stepId++,
  n: targetN,
  action: 'done',
  desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
  codeLine: 6,
  memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex(prev => prev + 1);
    }, speed);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [steps, speed]);

```

```

        <button
          onClick={() => { setIsPlaying(false); set
            className="p-2 text-slate-400 hover:text-
            title="Сбросить"
        >
          <RotateCcw className="w-4 h-4" />
        </button>
        <button
          onClick={() => { setIsPlaying(false); set
            disabled={currentStepIndex === 0}
            className="p-2 text-slate-400 hover:text-
            title="Шаг назад"
        >
          <ChevronLeft className="w-5 h-5" />
        </button>
        <button
          onClick={() => setIsPlaying(!isPlaying)}
          className={`flex items-center gap-1.5 px-
            isPlaying
              ? 'bg-amber-500 text-slate-950 hover:
              : 'bg-emerald-600 text-white hover:
          }`}
        >
          {isPlaying ? <Pause className="w-4 h-4" />
            {isPlaying ? 'Пауза' : 'Пуск'}}
        </button>
        <button
          onClick={() => { setIsPlaying(false); set
            disabled={currentStepIndex === steps.leng
            className="p-2 text-slate-400 hover:text-
            title="Шаг вперед"
        >
          <ChevronRight className="w-5 h-5" />
        </button>
      </div>
    </div>
  </div>

```

```

        </div>
      </div>
    })
  </div>

  { /* Tabs */ }
  <div className="flex items-center gap-2 border-b
    <button
      onClick={() => setActiveTab('table')}
      className={`flex items-center gap-2 px-6 py-3
        activeTab === 'table'
          ? 'border-emerald-500 text-emerald-400 bg
            : 'border-transparent text-slate-400 hove
        }` }
    >
      <Eye className="w-4 h-4" />
      Стен-бай-стен состояние
    </button>
    <button
      onClick={() => setActiveTab('code')}
      className={`flex items-center gap-2 px-6 py-3
        activeTab === 'code'
          ? 'border-emerald-500 text-emerald-400 bg
            : 'border-transparent text-slate-400 hove
        }` }
    >
      <Code className="w-4 h-4" />
      Интерактивный код
    </button>
  </div>

  { /* Tab Content */ }
  <div className="min-h-[320px] flex flex-col justi
    { activeTab === 'table' && {
      <div className="bg-slate-950 p-6 rounded-2xl
        <h5 className="font-bold text-white text-sm
          <span className="text-emerald-400"> </span>
        </h5>

```



```
        <span className="p-1.5 bg-emerald-500/20">
          <span>{currentStep?.desc}</span>
        </div>
      </div>
    )}
  </div>

  {/* Progress Bar */}
  <div className="mt-8 pt-6 border-t border-slate-800">
    <div className="flex items-center justify-between">
      <span>Прогресс выполнения</span>
      <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
    </div>
    <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
      <div
        className="h-full bg-gradient-to-r from-emerald-500 to-emerald-400"
        style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
      ></div>
    </div>
  </div>
</div>
);
}
```

ФАЙЛ 35/79: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },
  { id: 1, label: "B", x: 280, y: 140 },
  { id: 2, label: "C", x: 100, y: 140 },
  { id: 3, label: "D", x: 280, y: 250 },
```

```

const nextEdges = [...c1VisitedEdges, key];
const nextPath = [...c1Path, vId];
setC1VisitedEdges(nextEdges);
setC1Path(nextPath);

if (nextEdges.length === C1_EDGES.length) {
  const startDegree = C1_EDGES.filter(e => e.u === vId).length;
  const endDegree = C1_EDGES.filter(e => e.v === vId).length;
  const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
  if (isCycle && nextPath[0] === vId) {
    setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
  } else {
    setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С`);
  }
  setC1Done(true);
} else {
  setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $`);
}
};

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
        <button onClick={resetC1} className="flex items-center justify-center gap-2 border border-slate-700 transition-all">
          <Undo className="h-3.5 w-3.5" /> Сброс
        </button>
      </div>

      <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
        <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate_1px)] border border-slate-800">
          <svg className="w-full h-[260px] z-10 relative">
            {C1_EDGES.map((e, i) => {
              const u = C1_NODES.find(n => n.id === e.u)!;
              const v = C1_NODES.find(n => n.id === e.v)!;
              const key = `${Math.min(e.u, e.v)}-${Math.max(e.u, e.v)}`;
            })}
          </svg>
        </div>
      </div>
    </div>
  )
);

```

ФАЙЛ 36/79: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';
```

```
interface Question {
  id: number;
  question: string;
  options: string[];
  correctAnswer: number;
  explanation: string;
}
```

```
const quizQuestions: Question[] = [
  {
    id: 1,
    question: 'Что мы делаем в алгоритме Краскала, если ребро, красный) цвет?',
    options: [
      'Мы радуемся и добавляем его в минимальное остовное дерево',
      'Мы перекрашиваем их в синий цвет и делим граф по компонентам связности',
      'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
      'Мы вызываем алгоритм Дейкстры для поиска нового минимального ребра'
    ],
    correctAnswer: 2,
    explanation: 'Верно! Если вершины уже в одном клане, добавление этого ребра нарушит структуру дерева.'
  },
  {
    id: 2,
    question: 'Почему алгоритм Дейкстры впадает в ступор на графе с отрицательными весами?',
    options: [
      'Потому что он написан только для положительных весов',
      'Потому что он не учитывает отрицательные веса',
      'Потому что он не использует приоритетную очередь',
      'Потому что он не проверяет, достиг ли он минимального пути'
    ],
    correctAnswer: 1,
    explanation: 'Алгоритм Дейкстры не учитывает отрицательные веса, поэтому может застрять в бесконечном цикле.'
  }
];
```

```
      'СНМ (Система непересекающихся множеств / Disjoin  
      'Стек (LIFO).',  
      'Красно-черное дерево.',  
      'Хэш-таблица со списками коллизий.'  
    ],  
    correctAnswer: 0,  
    explanation: 'Блестяще! СНМ (DSU) позволяет за прак  
      .'  
  }  
];
```

```
export const ExpressQuiz: React.FC = () => {  
  const [currentQIndex, setCurrentQIndex] = useState<num  
  const [selectedOption, setSelectedOption] = useState<  
  const [isAnswered, setIsAnswered] = useState<boolean>  
  const [score, setScore] = useState<number>{0};  
  const [showResults, setShowResults] = useState<boolean>  
  
  const currentQ = quizQuestions[currentQIndex];  
  
  const handleSelectOption = (index: number) => {  
    if (isAnswered) return;  
    setSelectedOption(index);  
    setIsAnswered(true);  
    if (index === currentQ.correctAnswer) {  
      setScore(prev => prev + 1);  
    }  
  };  
  
  const handleNext = () => {  
    if (currentQIndex + 1 < quizQuestions.length) {  
      setCurrentQIndex(prev => prev + 1);  
      setSelectedOption(null);  
      setIsAnswered(false);  
    } else {  
      setShowResults(true);  
    }  
  };  
};
```

```

        <span>Пройти тест заново</span>
      </button>
    </div>
  );
}

return (
  <div className="bg-slate-900/90 border border-slate-
    >
    <div className="flex items-center justify-between"
      <div className="flex items-center gap-3">
        <span className="bg-purple-600 text-white px-3">
          Экспресс-тест
        </span>
        <h3 className="text-xl font-bold text-white">
        </div>
        <span className="text-sm font-mono text-slate-400">
          Вопрос {currentQIndex + 1} / {quizQuestions.length}
        </span>
      </div>

      <div className="mb-8">
        <h4 className="text-xl md:text-2xl font-bold text-white">
          {currentQ.question}
        </h4>

        <div className="space-y-4">
          {currentQ.options.map((option, idx) => {
            const isSelected = selectedOption === idx;
            const isCorrect = idx === currentQ.correctAnswer;

            let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
            if (isAnswered) {
              if (isCorrect) {
                optionStyle = "bg-emerald-950/80 border border-emerald-500";
              } else if (isSelected) {
                optionStyle = "bg-rose-950/80 border-rose-500";
              } else {

```

```

        {currentQ.explanation}
      </p>
    </div>
  )}

  <div className="flex justify-end pt-4 border-t border-gray-200">
    <button
      onClick={handleNext}
      disabled={!isAnswered}
      className={
        "px-8 py-3.5 font-bold rounded-xl transition-colors duration-200"
        (!isAnswered
          ? "bg-slate-800 text-slate-500 border border-slate-900"
          : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700"
        )
      }
    >
      {currentQIndex + 1 < quizQuestions.length ? 'Следующий' : 'Завершить'}
    </button>
  </div>
</div>
);
};

```

ФАЙЛ 37/79: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';
```

```

interface Step {
  k: number; i: number; j: number;
  matrix: number[][]; updated: boolean;
  log: string; activeCodeLine: number;
}

```

```
export default function FloydViz() {
```

```

    { line: 3, text: '      for k from 0 to V - 1: // Пос
    { line: 4, text: '          for i from 0 to V - 1:' }
    { line: 5, text: '              for j from 0 to V - 1
    { line: 6, text: '                  if dist[i][k] + d
    { line: 7, text: '                      dist[i][j] =
    { line: 8, text: '      return dist' },
  ];

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist = initialMatrix.map(row => [...row]);

  simulationSteps.push({
    k: -1, i: -1, j: -1, matrix: dist.map(r => [...r])
    log: '  Инициализация матрицы прямых путей,  $\infty$  (999)
    activeCodeLine: 2,
  });

  const N = 7;
  for (let k = 0; k < N; k++) {
    simulationSteps.push({
      k, i: -1, j: -1, matrix: dist.map(r => [...r]),
      log: `+ Внешний цикл. Разрешаем пути через пром
      activeCodeLine: 3,
    });

    for (let i = 0; i < N; i++) {
      for (let j = 0; j < N; j++) {
        if (i === j || i === k || j === k) continue;

        const oldVal = dist[i][j];
        const viaVal = dist[i][k] + dist[k][j];
        if (dist[i][k] !== 999 && dist[k][j] !== 999 &&
          dist[i][j] > viaVal) {
          dist[i][j] = viaVal;
          simulationSteps.push({
            k, i, j, matrix: dist.map(r => [...r]), u
            log: `  Улучшение пути [${i}]→[${j}] чере
             $\infty$ '.`,

```

```

    <div>
      <h3 className="text-2xl font-bold text-white">
        <p className="text-sm text-emerald-300">Полн
      </div>
    </div>

    <div className="flex items-center gap-2">
      <button onClick={() => { setIsPlaying(false);
        text-sm"> Сброс</button>
      <button onClick={() => setIsPlaying(!isPlaying
        -amber-600' : 'bg-emerald-600'}`}>{isPlaying
      <button onClick={() => { setIsPlaying(false);
        ; }} className="px-3 py-2 bg-emerald-600 te
    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6 ml
    {/* Граф */}
    <div className="w-full lg:w-2/3 bg-indigo-950/20
      stify-center min-h-[400px]">
        <div className="absolute top-3 left-3 bg-indi
          ont-bold shadow-sm">
            {currentStep.k >= 0 && currentStep.k < 7 ?
          </div>

        <svg className="w-full h-auto max-h-[500px]"
          <defs>
            <marker id="arrow-fw-normal" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
            <marker id="arrow-fw-active" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
            <marker id="arrow-fw-via" markerWidth="6"
            th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
          </defs>
            {Object.keys(adjList).flatMap((uStr) => {
              const u = parseInt(uStr);
              return adjList[u].map((edge) => {
                const uNode = nodesSvg.find((n) => n.id

```



```

    <div className="w-full bg-slate-950/80 p-4 rounded-2xl">
      <p className="font-semibold text-amber-400">
        <p className="min-h-[40px] flex items-center">
      </div>
    </div>

    { /* Список смежности */ }
    <div className="w-full lg:w-1/3 bg-emerald-950/50 p-4 rounded-2xl">
      <h4 className="text-lg font-bold text-emerald-400">
        <div className="space-y-2 text-sm font-mono text-white">
          {Object.keys(adjList).map((uStr) => {
            const u = parseInt(uStr);
            const isI = currentStep.i === u;
            const isK = currentStep.k === u;
            return (
              <div key={u} className={`flex flex-wrap justify-between items-center p-2 ${
                isI ? 'bg-amber-900/40 border-amber-500' :
                isK ? 'bg-indigo-900/40 border-indigo-500' :
                'bg-slate-900/40 border-slate-500'
              }`} >
                <span className="font-bold text-white text-sm">
                  {u}
                </span>
                {adjList[u].map((edge, idx) => {
                  const isUpd = currentStep.i === u;
                  const isVia = currentStep.k !== u;
                  const isTarget = currentStep.j === edge.v);
                  return (
                    <span key={idx} className={`px-2 py-1 text-sm font-mono ${
                      isUpd ? 'bg-amber-500 text-white border-amber-500' :
                      isVia ? 'bg-indigo-500 text-white border-indigo-500' :
                      'bg-slate-500 text-white border-slate-500'
                    }`} >
                      {edge.v} ({edge.w})
                    </span>
                  )
                })}
              </div>
            )
          })}
        </div>
      </div>
    </div>
  </div>
</div>

```

```

    { /* Код алгоритма */
    <div className="w-full lg:w-1/2 bg-slate-900/60" >
      <div>
        <h4 className="text-lg font-bold text-slate" >
          <div className="bg-slate-950/80 rounded-lg p-4" >
            {codeLines.map(item => (
              <div key={item.line} className={`py-1.5 px-4 ${
                e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'
              }`} >
              <span className="text-slate-600 w-5 shrink-0" >{item.line}</span>
              <span className="whitespace-pre flex-grow-1" >{item.code}</span>
            </div>
            )
          )}
          </div>
        </div>
      </div>
    </div>
  </div>
);
}

```

ФАЙЛ 38/79: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';

```

```

type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };

```

```

const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
];

```

```

        itedNodes: Array.from(vis) }));

    for (const u of adj[v]) {
        if (!vis.has(u)) {
            steps.push({ type: 'process', node: v, desc: `Процесс узла ${v}`,
                .stack], visitedNodes: Array.from(vis) });
            dfs(u);
        }
    }

    stack.pop();
    steps.push({ type: 'backtrack', node: v, desc: `Откат от узла ${v}`,
        ueOrStack: [...stack], visitedNodes: Array.from(vis) });
}

dfs(start);
return steps;
}

```

```

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начало поиска с узла ${start}`,
        ray.from(vis)] });

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Процесс узла ${v}`,
            q], visitedNodes: [...Array.from(vis)] });
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
            }
        }
    }
}

```

```

    }
  }, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
  <div className="space-y-4">
    <div className="flex bg-slate-900 border border-slate-800 p-4">
      <button onClick={() => setMode("dfs")}
        className={`px-4 py-2 text-xs font-medium rounded-md
          text-slate-400 hover:text-slate-300`} `>
        DFS (В глубину)
      </button>
      <button onClick={() => setMode("bfs")}
        className={`px-4 py-2 text-xs font-medium rounded-md
          text-slate-400 hover:text-slate-300`} `>
        BFS (В ширину / Волна)
      </button>
    </div>

    <div className="flex flex-col md:flex-row gap-4">
      <div /* SVG Graph View */
        className="bg-slate-900 border border-slate-800 p-4
          flex flex-1 min-h-[300px]">
        <svg className="w-full h-full max-w-[600px] max-h-[300px]">
          <!-- Edges -->
          {edges.map((e, i) => {
            const u = nodes.find(n => n.id === e.u);
            const v = nodes.find(n => n.id === e.v);
            // Check if edge is part of current step
            const edgeTraversed = isVisited(e.v) || isCurrent(e.v);

            return (
              <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
                className={`stroke-slate-400 ${edgeTraversed ? 'stroke-slate-300'}
                  stroke-width={2} `/>
            );
          })}
        </svg>
      </div>
    </div>
  </div>
);

```

```

                                className="stroke-rose-500"
                                ))
                                </g>
                            )
                        }}}}
                    </svg>
                </div>

                {/* Structure View (Stack / Queue) */}
                <div className="bg-slate-900 border border-rose-500
r">
                    <div className={`absolute -top-3 left-3
                                ${mode === 'dfs' ? 'bg-rose-500' : 'bg-slate-900'}
                                width-300px height-300px border border-emerald-500`} `>
                        {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
                    </div>

                    <div className="flex flex-col gap-2 p-3" `>
                        {step.queueOrStack && step.queueOrStack.length > 0 ? (
                            <div key={idx} className={`w-full
                                ${mode === 'dfs' ? 'bg-rose-500' : 'bg-slate-900'}
                                text-emerald-200`} `>
                                {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
                                ?_10px_rgba(245,158,11,0.2)]]' : ''}
                                `>
                                    Узел: {item}
                                    {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
                                    ?_10px_rgba(245,158,11,0.2)]]' : ''}
                                    <span className="ml-2" `>
                                        {item}
                                    </span>
                                </div>
                            </div>
                        ) : (
                            <div className="text-slate-500" `>
                                Структура пустая
                            </div>
                        )}
                    </div>
                </div>
            </div>

```

ФАЙЛ 39/79: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
```

```
interface Patient {
  id: string;
  name: string;
  emoji: string;
  symptom: string;
  priority: number; // 1 to 99 (Severity)
  animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  let idx = a.length - 1;
  while (idx > 0) {
    const parent = Math.floor((idx - 1) / 2);
    if (a[parent].priority < a[idx].priority) {
      [a[parent], a[idx]] = [a[idx], a[parent]];
      idx = parent;
    } else break;
  }
  return a;
}
```

```
function siftDown(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  const n = a.length;
  let idx = 0;
  while (true) {
    let largest = idx;
    const l = 2 * idx + 1;
```

```

-----
    { name: 'Иван (Инфаркт)', symptom: '  Болит сердце'
    { name: 'Маша (Перелом)', symptom: '  Открытый пере
    { name: 'Семён (Температура)', symptom: '  Жар 39.
    { name: 'Кот Барсик (Укус)', symptom: '  Укусил шме
    { name: 'Оля (Порез)', symptom: '  Глубокий порез',
    { name: 'Пётр (Кашель)', symptom: '  Простуда', emo
  ];
  start.forEach((p, i) => {
    arr = heapInsert(arr, { id: `init${i}`, ...p, animS
  });
  return arr;
})());

```

```

let uid = 300;

```

```

export const HeapViz: React.FC = () => {
  const [heap, setHeap]           = useState<Patient[]>([]);
  const [customName, setCustomName] = useState('');
  const [customPriority, setCustomPriority] = useState(0);
  const [log, setLog]             = useState<string>('');
  const [shakeEmpty, setShake]    = useState(false);
  const [busy, setBusy]           = useState(false);
  const [healingPatient, setHealing] = useState<Patient>({});

```

```

  const addLog = (msg: string) => setLog(prev => [msg,

```

```

  // --- INSERT: Новый пациент поступает в приемный пок

```

```

  const insertPatient = useCallback((name: string, prio
    if (busy || heap.length >= 15) {
      addLog('⚠ Все палаты переполнены! Дождитесь выпис
      setShake(true);
      setTimeout(() => setShake(false), 400);
      return;
    }
    setBusy(true);

```

```

  const preset = PATIENT_PRESETS[Math.floor(Math.rand

```

```

  const finalName = name || preset.name.split(' ')[0]

```

```

    });

    // --- EXTRACT MAX: Направить самого тяжелого в реанимацию
    const extractMax = useCallback(() => {
      if (busy || heap.length === 0) {
        setShake(true);
        addLog('⚠ Нет пациентов для экстренной госпитализации');
        setTimeout(() => setShake(false), 400);
        return;
      }
      setBusy(true);
      const topPatient = heap[0];
      setHealing(topPatient);

      addLog(`  СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

      // Step 1: Root lights up as winner
      setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'win' } : x));

      setTimeout(() => {
        // Step 2: Root exits the tree, tree reorganizes
        setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'exit' } : x));

        setTimeout(() => {
          // Run binary heap extraction
          setHeap(prev => {
            if (prev.length === 0) return [];
            const a = prev.map(x => ({ ...x }));
            a[0] = { ...a[a.length - 1] };
            a.pop();
            return siftDown(a).map(x => ({ ...x, animState: 'exit' }));
          });

          // Patient gets healed in the bed
          setTimeout(() => {
            setHealing(prev => prev ? { ...prev, animState: 'healed' } : null);
            addLog(`  Успех! Пациент ${topPatient.name} перенес операцию!`);
            setTimeout(() => {

```



```

        stroke="#475569" strokeWidth="2"
      />
    );
  }
  return res;
});

return (
  <div className="flex flex-col gap-6">
    { /* МНЕМОНИКА / ПРАВИЛО */ }
    <div className="bg-emerald-950/40 border border-emerald-500">
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-emerald-400 text-center">
          Priority Queue</h3>
        <p className="text-xs text-slate-300 mt-1 leading-relaxed">
          В реанимации не важно, кто пришел раньше, а кто в
          настоящее время</b> (максимальный приоритет).
          Куча (Heap) автоматически перестраивает папку,
          оказавшаяся на самом вершине дерева (в корне).
        </p>
      </div>
    </div>

    { /* УПРАВЛЕНИЕ */ }
    <div className="flex flex-col lg:flex-row items-stretch">
      { /* Добавить случайного */ }
      <button
        onClick={handleRandomInsert}
        disabled={busy || heap.length >= 15}
        className="flex-1 min-w-[150px] bg-gradient-to-r from-emerald-500 to-emerald-600
          opacity-40 disabled:cursor-not-allowed text-white font-medium
          py-2 px-4 transition-all cursor-pointer focus:outline-none"
      >
        <span> Привезти по Скорой (INSERT)</span>
      </button>
    </div>
  </div>
);

```

```

        transition-all cursor-pointer flex items-center
      >
        <span> В реанимацию (EXTRACT MAX)</span>
      </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800
        cursor-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

  { /* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */ }
  <div className="grid grid-cols-1 lg:grid-cols-12" >

    { /* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */ }
    <div className={`lg:col-span-8 bg-slate-900 rounded-2xl
      -[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`>
      <div className="absolute top-4 left-4 text-[14px] font-medium">
        Сортировка по тяжести (Двоичная куча / Max-Heap)
      </div>

      <div className="relative w-full h-full min-h-[400px]">
        <svg className="absolute inset-0 w-full h-full" >
          {edges}
        </svg>

        {heap.map((patient, idx) => {
          const pos = nodePos(idx);
          const isRoot = idx === 0;
          return (
            <div
              key={patient.id}
              className={`
                absolute flex flex-col items-center
                -translate-x-1/2 -translate-y-1/2 t

```

```

<div className="flex-1 flex flex-col justify-
  {healingPatient ? (
    <div className="flex flex-col items-cente
      <div className="relative">
        <span className="text-6xl block anim-l
        <span className="absolute -top-4 -right
        <span className="absolute -bottom-2 -
      </div>
      <div>
        <h4 className="text-white font-black
          <span>Пациент: {healingPatient.name
        </h4>
        <p className="text-emerald-400 text-x
          Состояние: Стабилизация ({healingPa
        </p>
        <div className="flex justify-center g
          <span className="w-2 h-2 rounded-fu
          <Heart className="w-4 h-4 text-rose
        </div>
      </div>
    </div>
  ) : (
    <div className="text-center text-slate-60
      <span className="text-5xl mb-2 block">
      <p className="text-xs font-bold font-mo
      <p className="text-[10px] mt-1">Ждём са
    </div>
  )}
</div>

<div className="w-full h-3 bg-gradient-to-r f
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-

```

```
-----  
/** Наведение на карточку ставит анимацию на паузу – мож  
export const DemoPauseContext = createContext(false);
```

```
export function useLoop(steps: number, ms = 900) {  
  const [step, setStep] = useState(0);  
  const paused = useContext(DemoPauseContext);  
  const period = Math.round(ms * SPEED);  
  
  useEffect(() => {  
    if (steps <= 1 || paused) return;  
    const t = setInterval(() => setStep((s) => (s + 1) *  
    return () => clearInterval(t);  
  }, [steps, period, paused]);  
  
  return step;  
}
```

```
export const C = {  
  idle: "#334155",  
  idleStroke: "#475569",  
  text: "#e2e8f0",  
  dim: "#94a3b8",  
  active: "#6366f1",  
  done: "#10b981",  
  warn: "#f59e0b",  
  bad: "#f43f5e",  
  edge: "#475569",  
};
```

```
export const Svg: React.FC<{ children: React.ReactNode;  
  const paused = useContext(DemoPauseContext);  
  return (  
    <div className="w-full">  
      <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">  
        {children}  
      </svg>  
      {caption && (  
        <div className="text-[10px] text-slate-400 text-right">
```

```

x2={b[0]}
y2={b[1]}
stroke={color}
strokeWidth={width}
strokeDasharray={dashed ? "4 3" : undefined}
style={{ transition: MOVE }}
/>
);

```

ФАЙЛ 41/79: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
  const step = useLoop(3, 1200);
  // i -> j напрямую 9; через k=1 получаем 3+4=7
  const pos: Record<string, [number, number]> = { i: [4, 4], j: [9, 9] };
  const viaOpen = step >= 1;
  const relaxed = step >= 2;
  return (
    <Svg
      caption={
        step === 0
          ? "k ещё запрещена: путь i → j = 9"
          : step === 1
            ? "Разрешаем пересадку через k: 3 + 4 = 7"
            : "7 < 9 → D[i][j] = 7"
        }
    >
    <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.done} />
    <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.bad} />
    <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.bad} />

```

```

        <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}
      >>
    >>
    {Object.entries(pos).map(([k, [x, y]]) => {
      const ci = compOf(k);
      return <Node key={k} x={x} y={y} r={12} label={ci} />
    })}
  </Svg>
);
};

```

/** Что такое граф: вершины, рёбра, направление и вес.

```

export const GraphBasicsDemo: React.FC = () => {
  const step = useLoop(3, 1300);
  const pos: Record<string, [number, number]> = { A: [50, 100], B: [150, 100], C: [250, 100] };
  const edges: [string, string, number][] = [
    ["A", "B", 1],
    ["B", "C", 1],
    ["C", "A", 1],
  ];
  return (
    <Svg>
      <caption>{
        step === 0
          ? "Вершины – объекты (города, слова, состояния)"
          : step === 1
            ? "Рёбра – связи между ними"
            : "Вес ребра – цена перехода"
      }
    >
    {edges.map(([u, v, w], i) => {
      const a = pos[u];
      const b = pos[v];
      return (
        <g key={i}>
          <Edge a={a} b={b} color={step >= 1 ? C.active : C.inactive} />
          {step >= 2 && (
            <text>
              x={(a[0] + b[0]) / 2}
              y={(a[1] + b[1]) / 2 - 4}
              textAnchor="middle"
              fontSize={10}
            </text>
          )}
        </g>
      )
    })}
  )
}

```

```

        {v}
      </text>
    </g>
  )))

  {step >= 1 &&
    sorted.map(({v, i}) => (
      <g key={v}>
        <rect x={30 + i * 52} y={62} width={46} height={16}>
        <text x={53 + i * 52} y={77} textAnchor="middle">
          {v}
        </text>
      </g>
    )))

  {step >= 2 &&
    raw.map(({v, i}) => (
      <g key={`c${i}`}>
        <rect x={startX + i * (boxW + 4)} y={98} width={boxW} height={16}>
        <text x={startX + i * (boxW + 4) + boxW / 2} y={112} textAnchor="center">
          {sorted.indexOf(v)}
        </text>
      </g>
    )))
  </Svg>
);
};

/* ----- Splay: три случая поворота ----- */

type Pt = [number, number];

const SplayFrames: React.FC<{
  frames: { pos: Record<string, Pt>; edges: [string, string][];
  captions: string[];
  ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);

```

```

    />
  );

  /** Zig-Zig – x и p с одной стороны, крутим сначала вер
  export const ZigZigDemo: React.FC = () => {
    <SplayFrames
      captions={["Линия g → p → x («бамбук»)", "Поворот В
      frames=[
        { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
        { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
        { pos: { x: [102, 24], p: [152, 70], g: [202, 112]
      ]}
    />
  );

  /** Zig-Zag – x и p с разных сторон, крутим сначала ниж
  export const ZigZagDemo: React.FC = () => {
    <SplayFrames
      captions={["Зигзаг: g влево, x вправо", "Поворот НИ
      frames=[
        { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
        { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
        { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
      ]}
    />
  );

```

ФАЙЛ 42/79: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],

```



```

    <Node key={k} x={x} y={y} label={k} fill={k ===
    )})
  </Svg>
);
};

export const ToposortDemo: React.FC = () => {
  const nodes = [
    { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
    { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
  ];
  const edges: [string, string][] = [
    ["труссы", "штаны"],
    ["носки", "штаны"],
    ["штаны", "боты"]
  ];
  const order = ["труссы", "носки", "штаны", "боты"];
  const step = useLoop(order.length + 1, 800);
  const placed = order.slice(0, step);
  const pos = Object.fromEntries(nodes.map((n) => [n.id, n.x]));
  return (
    <Svg caption={`Порядок: ${placed.join(" → ") || "..."}
    {edges.map(([u, v], i) => {
      <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
    })}
    {nodes.map((n) => {
      const idx = placed.indexOf(n.id);
      return (
        <g key={n.id}>
          <rect x={n.x - 34} y={n.y - 12} width={68} height={24} />
          <text x={n.x} y={n.y} textAnchor="middle" dx={
            {idx >= 0 && <circle cx={n.x + 30} cy={n.y - 12} r={10} />
            {idx >= 0 && {
              <text x={n.x + 30} y={n.y - 12} textAnchor="middle" />
            }
          }
        </g>
      );
    })}
    <text x={224} y={62} textAnchor="middle" fontSize=16 />
    <text x={224} y={74} textAnchor="middle" fontSize=16 />
  );
};

```

```

        <Node key={k} x={x} y={y} label={k} fill={cut ?
        }}}
    </Svg>
  );
};

```

```

export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
  };
  const edges: [string, string][] = [["A", "B"], ["A",
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали вершину C → две отдельные
      {edges.map(([u, v], i) => {cut && (u === "C" || v
      {Object.entries(pos).map(([k, [x, y]]) =>
        cut && k === "C" ? (
          <circle key={k} cx={x} cy={y} r={12} fill="no
        ) : (
          <Node key={k} x={x} y={y} label={k} fill={k =
            } />
        )
      )}
    </Svg>
  );
};

```

```

export const MstDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10
  };
  const edges = [
    { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
    { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
  ];
  const chosen = ["A-D", "A-B", "B-C", "D-E"];
  const step = useLoop(chosen.length + 1, 800);

```



```

const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f44336"];
const root = (i: number): number => (p[i] === i ? i : p[p[i]]);
return (
  <Svg caption="find = «кто твой староста», union = 0"
    {p.map((par, i) =>
      par !== i ? {
        <path key={i} d={`M ${pos[i][0]} ${pos[i][1]}
          fill="none" stroke={rootColor[root(i)]} stroke-width={2}
        } : null
      )}
    {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
  </Svg>
);
};

```

```

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "•", parent: null as number },
  { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
  { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
  { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
  { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
  { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

```

```

export const TrieDemo: React.FC = () => {
  const step = useLoop(TRIE_NODES.length + 1, 700);
  return (
    <Svg caption="Путь от корня по буквам = слово; общий предок"
      {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
      })}
      {TRIE_NODES.map((n) => <Node key={n.id} x={n.x} y={n.y} />)}
    </Svg>
  );
};

```

```

export const TrieBfsDemo: React.FC = () => {

```

```

    )))
  </Svg>
);
};

export const SegmentTreeDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const nodes = [
    { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
    { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
    { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
    { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
    { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
    { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
    { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
  ];
  return (
    <Svg caption="Запрос собирается из  $O(\log n)$  готовых"
      {nodes.map((n, i) => (
        <g key={i}>
          <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w}
            fill={n.lvl === step ? C.active : C.idle} stroke={C.active} />
          <text x={n.x} y={n.y} textAnchor="middle" dom
            fill={n.lvl === step ? C.active : C.idle} />
        </g>
      )))}
    </Svg>
  );
};
};
```

```

export const PrefixSumDemo: React.FC = () => {
  const a = [3, 1, 4, 1, 5, 9];
  const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
  const step = useLoop(4, 1200);
  const l = 1, r = 3;
  const show = step >= 1;
  return (
    <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] - P[${l}]` :
      "о префиксные суммы P"}>

```

```

    )))
  </Svg>
);
};

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
  const avg = step ? (total / step).toFixed(2) : "-";
  return (
    <Svg caption={`Сделано ${step} операций, суммарно $
      {costs.map((c, i) => (
        <rect key={i} x={14 + i * 30} y={104 - c * 10} width={30} height={10}
        fill={i < step ? (c > 2 ? C.bad : C.done) : C.done}
      ))}
      <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} strokeDash=[4, 4]}
      <text x={130} y={20} textAnchor="middle" fontSize={14}
    </Svg>
  );
};

export const NpDemo: React.FC = () => {
  const step = useLoop(2, 1400);
  return (
    <Svg caption={step === 0 ? "Найти решение – долго (N^2)" : "Решение найдено"}
      <ellipse cx={130} cy={64} rx={112} ry={48} fill={C.done} stroke={C.done} strokeDash=[4, 4]}
      <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333" stroke={C.done} strokeDash=[4, 4]}
      <text x={92} y={64} textAnchor="middle" dominantBaseline="middle"
      <text x={206} y={34} textAnchor="middle" fontSize={14}
      <circle cx={196} cy={84} r={13} fill={step === 1 ? C.done : C.wait} stroke={C.wait} strokeDash=[4, 4]}
      <text x={196} y={84} textAnchor="middle" dominantBaseline="middle"
      ?"></text>
      <text x={130} y={124} textAnchor="middle" fontSize={14}
    </Svg>
  );
};
};
```

```

        <g key={idx}>
          <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
            fill={idx === step - 1 ? C.active : fill}
            {filled && {
              <text x={57 + c * 38} y={36 + r * 32} text={C[idx]}
                fontSize={10} fontWeight="700" fill="black" />
            }}
        </g>
      );
    ))
  })
  <text x={130} y={122} textAnchor="middle" fontSize={14}
    каждая клетка считается один раз и переиспользуется />
</text>
</Svg>
);
};

```

ФАЙЛ 44/79: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
  import {
    AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
    SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
  } from "../demosStruct";
import {
  ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
  ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"

```

```

const [paused, setPaused] = useState(false);
const Cmp = REGISTRY[kind];
if (!Cmp) return null;
return (
  <DemoPauseContext.Provider value={paused}>
    <div
      className="bg-slate-950/70 rounded-lg border bo
      onMouseEnter={() => setPaused(true)}
      onMouseLeave={() => setPaused(false)}
    >
      <Cmp />
    </div>
  </DemoPauseContext.Provider>
);
};

```

ФАЙЛ 45/79: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";

```

```

interface Props {
  vizId: string | null;
  onClose: () => void;
}

```

```

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
  useEffect(() => {
    if (!vizId) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();

```



```
};
```

ФАЙЛ 46/79: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}
```

```
interface Props {
  anchor: HintAnchor | null;
  onClose: () => void;
  onOpenSimulator: (vizId: string) => void;
  onCardEnter: () => void;
  onCardLeave: () => void;
}
```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor,
  const cardRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ top: number; left: n

  useLayoutEffect(() => {
    if (!anchor) {
      setPos(null);
```

```

    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
      visibility: pos ? "visible" : "hidden",
    }}
    role="dialog"
  >
    <div className="bg-slate-900 border border-indigo-500" >
      <div className="flex items-start gap-2 px-3 pt-3" >
        <h4 className="text-sm font-bold text-indigo-400" >Термин
        <button
          onClick={onClose}
          className="text-slate-500 hover:text-white"
          aria-label="Закрыть подсказку"
        >
          <X className="w-4 h-4" />
        </button>
      </div>

      <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
        <p className="text-[13px] leading-relaxed text-slate-300">
          {entry.demo && <MiniDemo kind={entry.demo} />}
        </p>
        {entry.formal && (
          <p className="text-[11.5px] leading-relaxed text-slate-300">
            {entry.formal}
          </p>
        )}
        {entry.complexity && (
          <div className="text-[11px] font-mono text-slate-300">
            Сложность: {entry.complexity}
          </div>
        )}
      </div>
    </div>
  </div>

```

```

const edges = [
  { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
  { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
  { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
  { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
  { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
  { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];

const isNodeVisible = (id: string) => {
  if (id === 'S' && (step === 0 || step === 7)) return true;
  return false;
};

const getPotential = (id: string) => {
  if (step >= 2 && step < 7) {
    if (id === 'S') return 0;
    if (id === 'A') return 0;
    if (id === 'B') return -2;
    if (id === 'C') return -1;
  }
  return null;
};

const getEdgeColorClass = (e: any) => {
  if (e.u === 'S') {
    if (step === 0 || step === 7) return "opacity-0";
    if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
    if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
    return "stroke-slate-700 stroke-dasharray-4";
  }

  if (e.id === 'AB') {
    if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
    if (step > 4) return "stroke-emerald-500";
  }

  if (e.id === 'BC') {
    if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
  }
};

```

```

        if (e.id === 'BC') {
            if (step < 5) return e.weight;
            if (step === 5) return `${e.weight} + (${ -2`
            return e.newWeight;
        }
        if (e.id === 'CA') {
            if (step < 6) return e.weight;
            if (step === 6) return `${e.weight} + (${ -1`
            return e.newWeight;
        }
        return e.weight;
    };

const isEdgeTextHighlighted = (e: any) => {
    if (e.id === 'AB' && step === 4) return true;
    if (e.id === 'BC' && step === 5) return true;
    if (e.id === 'CA' && step === 6) return true;
    return false;
};

const isEdgeTextEmerald = (e: any) => {
    if (e.id === 'AB' && step > 4) return true;
    if (e.id === 'BC' && step > 5) return true;
    if (e.id === 'CA' && step > 6) return true;
    return false;
};

const getDesc = () => {
    switch (step) {
        case 0: return (
            <div>
                <b>Исходный граф.</b> Имеется ребро
                Если мы запустим быстрый алгоритм Д
            </div>
        );
        case 1: return (
            <div>
                <b>Шаг 1: Точка отсчета.</b><br/>

```

```

        Старый вес: 2. <br/>
        Вычисляем:  $2 + (-1) - 0 =$  <span cla
    </div>
    );
    case 7: return (
        <div className="text-emerald-300">
            <b>Готово!</b> Фиктивную вершину S
            Все новые веса ребер в графе стали
            При этом старые кратчайшие пути оста
        шин!
    </div>
    );
    default: return "";
    }
};

return (
    <div className="bg-slate-800 p-4 rounded-xl border
    <h4 className="text-sm md:text-base font-bo
    <span className="w-2.5 h-2.5 bg-amber-4
    Визуализация: Перевзвешивание алгоритмом
    </h4>

    <div className="flex flex-col md:flex-row g
        <div className="bg-[#0f172a] flex-1 min
        nter p-4">
            <svg width="400" height="300" class
                <defs>
                    <marker id="arrow-slate" vi
                    start-reverse">
                        <path d="M 0 0 L 10 5 L
                    </marker>
                    <marker id="arrow-slate-dim
                    uto-start-reverse">
                        <path d="M 0 0 L 10 5 L
                    </marker>
                    <marker id="arrow-amber" vi
                    start-reverse">

```



```
        {/* Progress Dots */}
        <div className="flex justify-center"
            {Array.from({length: MAX_STEP +
                <div key={i} className={`w-12 h-12 rounded-full
125' : i < step ? 'bg-indigo-900' : 'bg-slate-500'
                )}}
            </div>
        </div>
    </div>
</div>
);
}
```

ФАЙЛ 48/79: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect } from "react";
import {
    Play,
    Pause,
    RotateCcw,
    Layers,
} from "lucide-react";

interface GNode {
    id: number;
    x: number;
    y: number;
    label: string;
}

interface GEdge {
    u: number;
    v: number;
```

```

const generatedSteps: AlgoStep[] = [];
const visited = new Set<number>();
const order: number[] = [];
const sccMap: Record<number, number> = {};

const recordStep = (
  phase: AlgoStep["phase"],
  desc: string,
  currentNode: number | null,
  transposed: boolean,
  currentSccId: number,
) => {
  generatedSteps.push({
    phase,
    desc,
    visited: new Set(visited),
    currentNode,
    order: [...order],
    transposed,
    sccMap: { ...sccMap },
    currentSccId,
    activeEdges: transposed
      ? initialEdges.map((e) => ({ u: e.v, v: e.u })
        : [...initialEdges],
  ));
};

recordStep(
  "init",
  "Инициализация алгоритма Косарайю. Начинаем первый",
  null,
  false,
  -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

```



```

        `Первый проход завершен! Стек выхода в топологиче
        ебра.` ,
        null,
        true,
        -1,
    );

    // Transpose adj
    const adjT: number[][] = Array.from({ length: 5 },
    initialEdges.forEach((e) => adjT[e.v].push(e.u));

    // Reset visited for phase 2
    visited.clear();
    let sccId = 0;

    const dfs2 = (u: number, currentScc: number) => {
        visited.add(u);
        sccMap[u] = currentScc;
        recordStep(
            "dfs2",
            `DFS 2: заходим в ${u} на транспонированном гра
            u,
            true,
            currentScc,
        );

        for (const to of adjT[u]) {
            if (!visited.has(to)) {
                dfs2(to, currentScc);
            }
        }
    };

    // Run DFS2 using the order (reversed from back to
    for (let i = order.length - 1; i >= 0; i--) {
        const u = order[i];
        if (!visited.has(u)) {
            sccId++;

```

```

    sccMap: {},
    currentSccId: -1,
    activeEdges: initialEdges,
  });

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1) {
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1500);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getSccColor = (id: number) => {
  if (id === 1) return "fill-emerald-600 stroke-emera
  if (id === 2) return "fill-indigo-600 stroke-indigo
  return "fill-slate-800 stroke-slate-500";
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Layers className="w-5 h-5 text-emerald-400"
          Визуализатор Алгоритма Косарайю (Поиск SCC)
        </h3>
      <div className="flex items-center gap-2 bg-slate-

```

```

        <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
    </marker>
    <marker
      id="arrow-active"
      viewBox="0 0 10 10"
      refX="22"
      refY="5"
      markerWidth="6"
      markerHeight="6"
      orient="auto-start-reverse"
    >
      <path d="M 0 1 L 10 5 L 0 9 z" fill="#1
    </marker>
    <marker
      id="arrow-transposed"
      viewBox="0 0 10 10"
      refX="22"
      refY="5"
      markerWidth="6"
      markerHeight="6"
      orient="auto-start-reverse"
    >
      <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
    </marker>
  </defs>

```

```

{ /* Edges */ }
{initialEdges.map((edge, idx) => {
  const u = initialNodes.find((n) => n.id ==
  const v = initialNodes.find((n) => n.id ==

  // If transposed, draw reversed coordinates
  const x1 = step.transposed ? v.x : u.x;
  const y1 = step.transposed ? v.y : u.y;
  const x2 = step.transposed ? u.x : v.x;
  const y2 = step.transposed ? u.y : v.y;

  const isSccEdge =

```

```

    }}}

    { /* Nodes */
    {initialNodes.map((node) => {
      const isCurrent = step.currentNode === node
      const isVisited =
        step.visited.has(node.id) ||
        (step.phase === "dfs1" && step.currentNode === node)
      const sccId = step.sccMap[node.id];

      let classes = "fill-slate-800 stroke-slate-800"
      if (sccId) {
        classes = getSccColor(sccId);
      } else if (isCurrent) {
        classes = "fill-amber-600 stroke-amber-600"
      } else if (isVisited && step.phase === "dfs1") {
        classes = "fill-emerald-700 stroke-emerald-700"
      }

      return (
        <g key={node.id}>
          <circle
            cx={node.x}
            cy={node.y}
            r="20"
            className={`transition-all duration-200ms`}
            strokeWidth="3"
          />
          <text
            x={node.x}
            y={node.y}
            dy=".3em"
            textAnchor="middle"
            fill="white"
            fontSize="13px"
            fontWeight="bold"
            className="select-none"
          />
        </g>
      )
    })}
  )}

```

```

    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-auto" >
    {/* Top-Sort output / Order stack representation */}
    <div>
      <span className="text-[10px] text-slate-500" >
        СТЕК ВЫХОДА (DFS 1 ORDER):
      </span>
      <div className="flex flex-wrap gap-1 bg-slate-900" >
        {step.order.length === 0 ? (
          <span className="text-slate-600 text-[10px]" >
            Пустой стек
          </span>
        ) : (
          [...step.order].reverse().map(({val, idx}) => {
            <div
              key={idx}
              className="bg-indigo-950 border border-slate-800 px-2 py-1"
            >
              {val}
            </div>
          })
        )}
      </div>
    </div>

    {/* List of actions log */}
    <div>
      <span className="text-[10px] text-slate-500" >
        ЛОГ ОПЕРАЦИЙ:
      </span>
      <div className="space-y-1.5 max-h-[150px]" >
        {steps
          .slice(0, currentStepIdx + 1)
          .reverse()
          .slice(0, 5)
          .map(({s, idx}) => {
            <div

```

ФАЙЛ 49/79: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check
  id: 'react';
```

```
interface Vertex {
  id: number;
  label: string;
  x: number;
  y: number;
  color: string; // 'none', 'red', 'blue', 'purple', 'm
}
```

```
interface Edge {
  id: string;
  u: number;
  v: number;
  weight: number;
  status: 'pending' | 'inspecting' | 'included' | 'reje
  color: string;
}
```

```
interface StepLog {
  title: string;
  description: string;
  type: 'info' | 'success' | 'warning' | 'error';
}
```

```
// 9 Vertices layout
```

```
const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
  { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
```

```

    ));

// --- KRUSKAL STEPS ---
const kruskalSteps = [
  // Step 1: Inspect A-B (2)
  () => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Шаг 1: Берем самое дешевое ребро A-B (в
      description: 'Оно соединяет две бесцветные верши
      type: 'info'
    }, ...prev]));
  },
  // Step 2: Include A-B
  () => {
    setVertices(prev => prev.map(v => v.id === 0 || v
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро A-B в осто́ве',
      description: 'Вершины A and B окрашены в красны
      type: 'success'
    }, ...prev]));
  },
  // Step 3: Inspect D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
      description: 'Оно соединяет две другие бесцветн
      type: 'info'
    }, ...prev]));
  },
  // Step 4: Include D-E
  () => {
    setVertices(prev => prev.map(v => v.id === 3 || v
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро D-E в осто́ве',

```

```

        title: 'Слияние кланов: Ребро A-D добавлено',
        description: 'Мы объединили красный и синий клан',
        type: 'success'
    }, ...prev]));
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Берем ребро B-E (вес 6)',
        description: 'НО: это ребро соединяет вершины B
        type: 'warning'
    }, ...prev]));
},
// Step 10: Reject B-E
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Мы выбрасываем ребро B-E. Иначе п
        type: 'error'
    }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Шаг 6: Берем ребро E-F (вес 7)',
        description: 'Оно соединяет фиолетовую вершину
        type: 'info'
    }, ...prev]));
},
// Step 12: Include E-F
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро E-F в осто́ве',

```



```

        type: 'error'
      }, ...prev]);
    },
    // Step 17: Inspect G-H (10)
    () => {
      setEdges(prev => prev.map(e => e.id === '6-7' ? {
        setLogs(prev => [{
          title: 'Шаг 9: Берем ребро G-H (вес 10)',
          description: 'Соединяет фиолетовую G и бесцветную H',
          type: 'info'
        }, ...prev]));
    },
    // Step 18: Include G-H
    () => {
      setVertices(prev => prev.map(v => v.id === 7 ? {
        setEdges(prev => prev.map(e => e.id === '6-7' ? {
        setLogs(prev => [{
          title: 'Успех: Ребро G-H в осто́ве',
          description: 'Вершина H перекрашена в фиолетовую',
          type: 'success'
        }, ...prev]));
      },
    // Step 19: Inspect E-H (11) - Cycle!
    () => {
      setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
          title: 'Шаг 10: Берем ребро E-H (вес 11)',
          description: 'Оба конца E и H уже фиолетовые. Соединяем их!',
          type: 'warning'
        }, ...prev]));
    },
    // Step 20: Reject E-H
    () => {
      setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
          title: 'Отказ: Цикл обнаружен!',
          description: 'Выбрасываем ребро E-H.',
          type: 'error'
        }, ...prev]));
    },

```

```

// Step 2: Pop D-E (3)
() => {
  setEdges(prev => prev.map(e => e.id === '3-4' ? {
  setLogs(prev => [{
    title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
    description: 'Плесень тянется по ребру D-E к вершине E',
    type: 'info'
  }, ...prev]));
},
// Step 3: Include D-E, add D's edges to heap
() => {
  setVertices(prev => prev.map(v => v.id === 3 ? {
  setEdges(prev => prev.map(e => e.id === '3-4' ? {
    ..e, status: 'in_heap' } : e));
  setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-F (8)']);
  setLogs(prev => [{
    title: 'Успех: Плесень захватила вершину D',
    description: 'В кучу добавлены новые пути из D: A-D (5), B-E (6), E-F (7), D-F (8)',
    type: 'success'
  }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
  setLogs(prev => [{
    title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
    description: 'Плесень расширяется в сторону вершины A',
    type: 'info'
  }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
  setVertices(prev => prev.map(v => v.id === 0 ? {
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
    ..e, status: 'in_heap' } : e));
  setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D-F (8)']);
  setLogs(prev => [{
    title: 'Успех: Плесень поглотила вершину A',
    description: 'В кучу добавлены новые пути из A: A-B (2), B-E (6), E-F (7), D-F (8)',
    type: 'success'
  }, ...prev]));
},

```

```

        ear' } : e));
    setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
    setLogs(prev => [{
        title: 'Успех: Вершина C захвачена',
        description: 'Ребро C-F(9) добавлено в кучу.',
        type: 'success'
    }, ...prev]));
},
// Step 10: Pop B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
        description: 'ВНИМАНИЕ! Плесень проверяет вершину B',
        type: 'warning'
    }, ...prev]));
},
// Step 11: Reject B-E
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
    setLogs(prev => [{
        title: 'Отказ: Игнорируем внутреннее ребро',
        description: 'Ребро B-E отброшено. Плесень не р',
        type: 'error'
    }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
        description: 'Плесень из центра Токио (E) тянет',
        type: 'info'
    }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {

```

```

    }, ...prev]));
},
// Step 17: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
    setLogs(prev => [{
        title: 'Отказ: Игнорируем ребро C-F',
        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
        description: 'Плесень ползет к вершине H.',
        type: 'info'
    }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
        eap' } : e));
    setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
    setLogs(prev => [{
        title: 'Успех: Вершина H захвачена',
        description: 'В кучу добавлено H-I(12).',
        type: 'success'
    }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Шаг 11: Куча выдает E-H (вес 11)',

```

```

const boruvkaSteps = [
  // Step 1: Five-Year Plan 1 (Inspecting outgoing)
  () => {
    // Highlight the cheapest outgoing edges for EACH
    // Node 0(A) -> A-B(2)
    // Node 1(B) -> A-B(2)
    // Node 2(C) -> B-C(4)
    // Node 3(D) -> D-E(3)
    // Node 4(E) -> D-E(3)
    // Node 5(F) -> E-F(7)
    // Node 6(G) -> D-G(8)
    // Node 7(H) -> G-H(10)
    // Node 8(I) -> H-I(12)
    setEdges(prev => prev.map(e =>
      ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
    ]));
    setLogs(prev => [{
      title: 'Первая пятилетка: Каждая деревня выбирает',
      description: 'Каждая из 9 деревень одновременно выбирает',
      text: 'А выбирает A-B(2), D выбирает D-E(3), а G выбирает D-G(8)',
      type: 'info'
    }, ...prev]);
  },
  // Step 2: Concurrently merge into Kolkhozes (Components)
  () => {
    // Concurrently build chosen roads. Merge into:
    // Component 1: {A, B, C} (colored red)
    // Component 2: {D, E, F, G, H, I} (colored gold/blue)
    setVertices(prev => prev.map(v =>
      [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
    ));
    setEdges(prev => prev.map(e => {
      if (['0-1', '1-2'].includes(e.id)) {
        return { ...e, status: 'included', color: 'red' };
      }
      if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
        return { ...e, status: 'included', color: 'blue' };
      }
      return e;
    }));
  },
];

```



```

    if (color === 'red') return '#ef4444';
    if (color === 'blue') return '#3b82f6';
    if (color === 'purple') return '#a855f7';
    if (color === 'mold') return '#10b981';
  }
  return '#475569';
};

return (
  <div className="space-y-12">

    {/* 3-in-1 MST Simulator Box */}
    <div className="bg-slate-900/90 border border-slate-800" >
      <div className="flex flex-col lg:flex-row justify-between" >
        <div>
          <div className="flex items-center gap-2 mb-2" >
            <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-bold uppercase tracking-wider">
              Сунер-Интерактив 3 в 1
            </span>
            <h3 className="text-2xl font-bold text-white">
              MST
            </h3>
          </div>
          <p className="text-slate-400 text-sm">
            Наблюдай за работой Краскала (Раскраска),
          </p>
        </div>

        {/* Algorithm Tabs */}
        <div className="flex flex-wrap items-center gap-2" >
          <div className="flex items-center bg-slate-800/50" >
            <button
              onClick={() => handleReset('kruskal')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
                +
                (activeAlgo === 'kruskal'
                  ? 'bg-indigo-600 text-white shadow-md'
                  : 'text-slate-400 hover:text-slate-300')
              }
            >
              Краскала
            </button>
          </div>
        </div>
      </div>
    </div>
  )
)

```

```

        <RotateCcw className="w-3.5 h-3.5" />
        <span>Сбросить</span>
    </button>

    <button
      onClick={handleNextStep}
      disabled={stepIndex >= currentSteps.length}
      className={
        "flex items-center justify-center gap-2
initial cursor-pointer " +
        (stepIndex >= currentSteps.length
          ? "bg-slate-800 text-slate-500 border
: activeAlgo === 'kruskal'
          ? "bg-indigo-600 hover:bg-indigo-500
: activeAlgo === 'prim'
          ? "bg-emerald-600 hover:bg-emerald-500
          : "bg-rose-600 hover:bg-rose-500 text
        )
      >
      <SkipForward className="w-4 h-4" />
      <span>{stepIndex === 0 ? 'Начать' : stepIndex}
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3"
  {/* Graph Canvas */}
  <div className="lg:col-span-2 bg-slate-950 rounded
    active min-h-[480px] overflow-hidden">

    {/* Legend */}
    <div className="absolute top-4 left-4 bg-slate-900
      -center gap-3 text-xs">
      <div className="flex items-center gap-1.5">
        <div className="w-3 h-3 rounded-full bg-
        </div>
        {activeAlgo === 'kruskal' ? (
          <>

```



```

    { /* SVG Edges */ }
    <svg className="absolute inset-0 w-full h-full"
    spectRatio="none">
      {edges.map(edge => {
        const u = vertices[edge.u];
        const v = vertices[edge.v];
        const strokeColor = getEdgeStroke(edge);
        const isInspecting = edge.status === 'inspect';
        const isRejected = edge.status === 'rejected';
        const isInHeap = edge.status === 'in_heap';

        return (
          <g key={edge.id}>
            <line
              x1={u.x + "%"}
              y1={u.y + "%"}
              x2={v.x + "%"}
              y2={v.y + "%"}
              stroke={strokeColor}
              strokeWidth={isInspecting ? 4.5 : 1}
              strokeDasharray={isRejected ? '4,4' : ''}
              className={"transition-all duration-200"}
            />
            <text
              x={({u.x + v.x} / 2 + "%")}
              y={({u.y + v.y} / 2 + "%")}
              fill={isInspecting ? '#f59e0b' : '#94a3b8'}
              : '#94a3b8'}
              fontSize="4.5"
              fontFamily="monospace"
              fontWeight="bold"
              textAnchor="middle"
              dominantBaseline="middle"
              className="select-none filter drop-shadow"
              dy="-1.5"
            >
              {edge.weight}
            </text>
          </g>
        );
      })}
    </svg>
  );
}

```

```

    { /* Logs / Heap */ }
    <div className="bg-slate-950 rounded-2xl border-2 border-b
      <div className="p-4 bg-slate-900 border-b border-2 border-
        <h4 className="font-bold text-slate-200 text-sm">
          Ход выполнения
        </h4>
      </div>

    {activeAlgo === 'prim' && {
      <div className="p-3 bg-slate-900/60 border-2 border-
        <p className="text-[11px] font-bold text-slate-200">
          ✂ Приоритетная очередь (Min-Heap):
        </p>
        <div className="flex flex-wrap gap-1.5">
          {heapQueue.length === 0 ? {
            <span className="text-xs text-slate-200">
              Нет элементов
            </span>
          } : {
            heapQueue.map((item, idx) => {
              <span key={idx} className={"px-2 py-1 text-xs font-mono" +
                " shadow-md animate-pulse" : "bg-slate-800 text-slate-200"}>
                {item}
              </span>
            })
          })
        </div>
      </div>
    })

    <div className="flex-1 p-4 overflow-y-auto">
      {logs.map((log, idx) => {
        <div
          key={idx}
          className={
            "p-4 rounded-xl border transition-all duration-200" +
            (log.type === 'success'
              ? 'bg-emerald-950/40 border-emerald-500/40'
              : log.type === 'warning'
              ? 'bg-amber-950/40 border-amber-500/40'
              : 'bg-slate-950/40 border-slate-500/40')
          >

```

```

        className={"px-3 py-1.5 rounded text-xs font-medium"}
        == 'prim' ? "bg-emerald-600 text-white" : "bg-white text-slate-700"
      >
        Прима (Билет 19)
      </button>
      <button
        onClick={() => setActiveCheatTab('boruvka')}
        className={"px-3 py-1.5 rounded text-xs font-medium"}
        == 'boruvka' ? "bg-rose-600 text-white" : "bg-white text-slate-700"
      >
        Борувка (Билет 20)
      </button>
    </div>
  </div>

```

```

{activeCheatTab === 'kruskal' && {
  <div className="grid grid-cols-1 lg:grid-cols-2" >
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-inherit" />
        </p>
        <p className="text-2xl font-black text-white">Крускал</p>
        <p className="text-slate-400 text-xs font-medium">Минимальное количество ребер, соединяющих все вершины графа в один цикл, называется минимальным остовным деревом.
        </p>
      </div>
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Award className="w-3.5 h-3.5 text-inherit" />
        </p>
        <p className="text-2xl font-black text-white">Награда</p>
        <p className="text-slate-400 text-xs font-medium">Награда за решение задачи.
        </p>
      </div>
      <div className="bg-slate-950 p-4 rounded-2xl">
        <p className="text-slate-400 text-xs font-medium">
          <Award className="w-3.5 h-3.5 text-inherit" />
        </p>
        <p className="text-2xl font-black text-white">Награда</p>
        <p className="text-slate-400 text-xs font-medium">Награда за решение задачи.
        </p>
      </div>
    </div>
  </div>

```

```

        <Clock className="w-3.5 h-3.5 text-emerald-400" />
      </p>
      <p className="text-2xl font-black text-emerald-400" />
      <p className="text-slate-400 text-xs mt-1" />
    </div>
    <div className="bg-slate-950 p-4 rounded-3xl" />
      <p className="text-slate-400 text-xs font-medium" />
      <Award className="w-3.5 h-3.5 text-emerald-400" />
      <p className="text-2xl font-black text-emerald-400" />
      <p className="text-slate-400 text-xs mt-1" />
    </div>
    <div className="bg-slate-950 p-4 rounded-3xl" />
      <p className="text-slate-400 text-xs font-medium" />
      ложении:</p>
      <p className="text-xs text-slate-300 leading-relaxed" />
      ие чистые вершины через самое дешевое доступное ребро.</p>
    </div>
    <div className="lg:col-span-2" />
      <div className="bg-slate-950 p-4 rounded-3xl" />
        <p className="text-slate-400 text-xs font-medium" />
        <Code className="w-3.5 h-3.5 text-emerald-400" />
        <p>
          <pre className="text-xs text-emerald-400 font-mono" />
            80 flex-1">
{`import heapq

def prim(graph, start):
    visited = [False] * len(graph)
    heap = [(0, start, -1)] # (weight, node, parent)
    mst = []

    while heap:
        w, u, prev = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        if prev != -1: mst.append((prev, u, w))

```



```
      complexity: 'O(V³)',
      border: 'border-emerald-500/30 hover:border-emerald-500/40',
      titleColor: 'text-emerald-400',
      badge: 'text-emerald-400/70 bg-emerald-950/40',
    },
  ];

export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {cards.map((c, i) => (
        <div key={i} className={`bg-slate-800 rounded-2xl p-4`}>
          <div className="h-48 overflow-hidden">
            <img src={c.img} alt={c.alt} className="w-full h-full object-cover grid" />
          </div>
          <div className="p-4">
            <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`}>{c.title}</h3>
            <p className="text-slate-400 text-sm">{c.desc} <span className={`font-bold ${c.titleColor}`}>{c.complexity}</span>
          </p>
          <div className={`mt-3 text-xs font-mono px-2`} >{c.time}</div>
        </div>
      ))}
    </div>
  );
}
```

ФАЙЛ 51/79: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";
```

```

    <span className="p-1.5 rounded-lg bg-indigo-600" >
      <List className="w-4 h-4" />
    </span>
    <span className="min-w-0 flex-1">
      <span className="block text-[10px] uppercase" >
        Содержание • {index + 1} из {total}
      </span>
      <span className="block text-[13px] font-bold" >
      </span>
    </button>
  </div>

  {open && (
    <div className="lg:hidden fixed inset-0 z-[90]" >
      <div className="absolute inset-0 bg-slate-950" >
        <div className="absolute inset-y-0 left-0 w-[100px] h-[100px] bg-white" >
          <div className="flex items-center justify-center" >
            <span className="font-bold text-white text-[12px]" >
              <button
                onClick={() => setOpen(false)}
                className="p-2 -mr-2 text-slate-400 hover:bg-slate-800"
                aria-label="Заккрыть содержание"
              >
                <X className="w-5 h-5" />
              </button>
            </div>
          </div>
          <div className="flex-1 min-h-0">
            <Sidebar
              selectedChapterId={selectedChapterId}
              setSelectedChapterId={setSelectedChapterId}
              onNavigate={() => setOpen(false)}
            />
          </div>
        </div>
      </div>
    </div>
  )}
</>

```

```

>
<button
  id="tab-guide-btn"
  onClick={() => setActiveTab('guide')}
  className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
    activeTab === 'guide'
      ? 'bg-indigo-600 text-white shadow-lg s
      : 'text-slate-400 hover:text-white'
  }`}
>
  <BookOpen className="w-4 h-4" /> Учебное по
</button>
<button
  id="tab-simulator-btn"
  onClick={() => setActiveTab('simulator')}
  className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
    activeTab === 'simulator'
      ? 'bg-indigo-600 text-white shadow-lg s
      : 'text-slate-400 hover:text-white'
  }`}
>
  <Cpu className="w-4 h-4" /> Тренажёры
</button>
<a
  id="download-pdf-btn"
  href="/export/full_code_all_pages.pdf"
  download="full_code_all_pages.pdf"
  target="_blank"
  rel="noreferrer"
  className="flex items-center justify-center
over:bg-emerald-600/20 border border-emeral
title="Скачать полный PDF сборник всех стра
>
  <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a

```



```

        {id:1,x:270,y:100},
        {id:2,x:230,y:230},
        {id:3,x:90,y:230},
        {id:4,x:50,y:100}
    ];
    const edges = [
        [0,1],[0,2],[0,3],[0,4],
        [1,2],[1,3],[1,4],
        [2,3],[2,4],
        [3,4]
    ];
    function findPt(id:number) { return pts.f
    function intersect(a:number,b:number,c:nu
        if (a===c||a===d||b===c||b===d) return
        const p1=findPt(a),p2=findPt(b),p3=find
        function ccw(ax:number,ay:number,bx:num
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses: number[] = [];
    for(let i=0;i<edges.length;i++) {
        for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1]
                if (!crosses.includes(i)) crosses.p
                if (!crosses.includes(j)) crosses.p
            }
        }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(
        const xing = crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x
            stroke={xing ? "#f43f5e" : "#4f46e5"}
        }
    }
    const labels = ["A","B","C","D","E"];

```

```

        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
        && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses:number[]=[];
    for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++)
        if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
            if (!crosses.includes(i)) crosses.push(i);
            if (!crosses.includes(j)) crosses.push(j);
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
        const xing=crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
            stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
    }
    const labels = ["\u04141", "\u04142", "\u04143"];
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
            <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
        </g>);
    }
    return <>{lines}{circles}</>;
    })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20" style={{z-index: 20}}>
     $V=6, E=9 \Rightarrow 2V-E=8$  (двудольный)
</div>
</div>
</div>

```

```

<div className="bg-amber-950/40 border border-amber-950/40 padding-10">
    ▲ Запомни:  $K_5$  и  $K_{3,3}$  — два кита непланарности.
    гомеоморфный  $K_5$  или  $K_{3,3}$  — он непланарен. Теорема Кюри-Вейля.
</div>

```

```

export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'red',
      голода после матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён', emoji: '🍲', color: 'blue',
      ишу ИИ за порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'green',
      ез сметаны, плиз.', animState: 'idle' },
  ]));

  const [servedHistory, setServedHistory] = useState<string[]>([]);
  const [isServing, setIsServing] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>([]);

  const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

  // ENQUEUE: Встать в конец очереди (хвост / Tail)
  const enqueue = useCallback(() => {
    if (isServing) return;
    if (queue.length >= 6) {
      addLog('⚠ Хвост очереди уже на улице, повару тяжёло!');
      setShake(true);
      setTimeout(() => setShake(false), 400);
      return;
    }
  }, [isServing, queue.length]);

  const preset = PRESETS[counter % PRESETS.length];
  counter++;

  const newGuest: Customer = {
    id: Date.now().toString(),
    ...preset,
    animState: 'in',
  };

  setQueue(prev => [...prev, newGuest]);

```

```

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: '🍲', color: 'red', text: 'У меня нет голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: '🍲', color: 'blue', text: 'Я напишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'green', text: 'Я не ем без сметаны, плиз.', animState: 'in' },
  ]);
  setServedHistory([]);
  setIsServing(false);
  setLog([' Столовая сброшена. Снова голодная толпа!']);
  setTimeout(() => {
    setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
  });
};

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-indigo-950/40 border border-indigo-500 p-6">
      <div className="text-3xl font-black text-white mb-2">ПРАВИЛО</div>
      <div>
        <h3 className="font-black text-indigo-400 text-2xl mb-1">ПРАВИЛО</h3>
        <p className="text-xs text-slate-300 mt-1 leading-tight">
          В очереди за супом всё честно. Кто первый встал, тот и ест.
          Новые гости встают строго в конец очереди.
          Здесь нет обхода и блата – только строгий порядок.
        </p>
      </div>
    </div>

    {/* УПРАВЛЕНИЕ */}
    <div className="flex items-center gap-3 flex-wrap">
      <button
        className="text-white text-sm font-medium"
        onClick={enqueue}
      >Встать в очередь
      </button>
    </div>
  </div>
);

```

```
-----
<div className="flex items-end justify-between">
```

```
  { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
  <div className="flex flex-col items-center">
    <div className="relative">
      <div className="absolute -top-10 left-1/2">
        <div className="w-1.5 h-6 bg-slate-500">
          <div className="w-2 h-4 bg-slate-400">
          </div>
        </div>
        <span className="text-5xl block animate-pulse">
        </span>
      </div>
      <div className="text-center">
        <span className="text-3xl block"> <
        <span className="text-[10px] font-bold font-mono tracking-wider">
          ПОВАР
        </span>
      </div>
    </div>
  </div>
```

```
  { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }
  <div className="flex flex-col items-center">
    <span className="text-2xl animate-pulse">
    <span className="text-[8px] font-mono tracking-wider">
    </span>
  </div>
```

```
  { /* СПИСОК ОЧЕРЕДИ */ }
  <div className="flex-1 flex items-end gap-4">
    {queue.length === 0 ? (
      <div className="flex-1 flex flex-col justify-between">
        <span className="text-4xl mb-1"> </
        <p className="text-xs font-bold font-mono tracking-wider">
        <p className="text-[10px]">Все сыты
      </div>
    ) : (
      queue.map((customer, idx) => {
        const isHead = idx === 0;
        const isTail = idx === queue.length - 1;
```

```

        </span>
      })
    </div>
  </div>
);
})
})
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 round
  <div className="absolute top-4 left-4 text-[10px] font-mono">
    Сытые гости (Архив FIFO)
  </div>

  <div className="absolute top-4 right-4 flex items-center justify-center gap-1
    order-emerald-800/80 text-[9px] font-mono">
    <Sparkles className="w-3.5 h-3.5 animate-pulse">
    <span>СЫТЫ </span>
  </div>

  <div className="flex-1 flex flex-col justify-between p-4">
    {servedHistory.length === 0 ? (
      <div className="flex-1 flex flex-col items-center justify-center gap-2">
        <span className="text-5xl mb-2"> </span>
        <p className="text-xs font-bold font-mono">
          <p className="text-[10px] mt-1">Здесь 6
        </div>
      ) : (
        <div className="flex flex-col gap-1.5 w-full">
          {servedHistory.map((item, idx) => {

```

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';

interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
  depth: number;
  x?: number;
  y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
  const [words, setWords] = useState<string[]>(['CAR', 'FISH', 'SALMON']);
  const [newWord, setNewWord] = useState('');
  const [nodes, setNodes] = useState<{ [key: number]: TrieNode }>({});
  const [simulationStep, setSimulationStep] = useState<number>(0);
  const [bfsQueue, setBfsQueue] = useState<number[]>([]);
  const [currentNode, setCurrentNode] = useState<number>(0);
  const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и нажми на кнопку "Симуляция".');
  const [isTalking, setIsTalking] = useState<boolean>(false);
  const [isBlinking, setIsBlinking] = useState<boolean>(false);

  // Регулярное моргание рыбы
  useEffect(() => {
    const blinkInterval = setInterval(() => {
      setIsBlinking(true);
      setTimeout(() => setIsBlinking(false), 200);
    }, 1000);
    return () => clearInterval(blinkInterval);
  }, []);
```

```

    }
    if (!newNodes[curr].words.includes(word)) {
      newNodes[curr].words.push(word);
      newNodes[curr].is_terminal = true;
    }
  });

  let currentX = 0;
  const paddingX = 70;
  const paddingY = 80;

  const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
      newNodes[u].x = 40 + currentX * paddingX;
      currentX++;
    } else {
      let leftX = -1;
      let rightX = -1;
      childKeys.forEach(v => {
        layoutDFS(v, depth + 1);
        if (leftX === -1) leftX = newNodes[v].x!;
        rightX = newNodes[v].x!;
      });
      newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : (leftX + rightX) / 2;
      if (leftX === -1) currentX++;
    }
  };

  layoutDFS(0, 0);

  setNodes(newNodes);
  setSimulationStep(0);
  setBfsQueue([]);
  setCurrentNode(null);
  setSpeechText(

```



```

    rootChildren.forEach(({childId}) => {
      updatedNodes[childId].fail = 0;
    });

    setNodes(updatedNodes);
    setBfsQueue(queue);
    setSimulationStep(1);
    setCurrentNode(null);
    setSpeechText(
      'Начинаем обход в ширину (BFS)! Все вершины на графике  

      уже суффикса просто нет. Жми «Следующий шаг»!'
    );
  };

// Один шаг BFS
const stepBFS = () => {
  if (bfsQueue.length === 0) {
    setSimulationStep(2);
    setCurrentNode(null);
    setSpeechText(
      'Ура! Очередь пуста! Мы успешно построили полный  

      граф. Теперь можно полюбоваться таблицей переходов!'
    );
    return;
  }

  const r = bfsQueue[0];
  const newQueue = bfsQueue.slice(1);
  const updatedNodes = { ...nodes };
  const rNode = updatedNodes[r];
  const childrenIds = Object.values(rNode.children);

  let logs = `Рассматриваем вершину #${r} (${rNode.children.length} детей)`;

  for (const [char, u] of Object.entries(rNode.children)) {
    newQueue.push(u);
    let v = rNode.fail;
    while (v !== 0 && updatedNodes[v].children[char] != null) {
      v = updatedNodes[v].fail;
    }
    updatedNodes[rNode.fail] = { children: { ...rNode.children, [char]: u }, fail: v };
    logs += `Добавляем ребро ${r} -> ${u} с суффиксом ${char}`;
  }

  setBfsQueue(newQueue);
  setNodes(updatedNodes);
  setSimulationStep(3);
  setCurrentNode(rNode);
  setSpeechText(logs);
};

```

```

    </div>
  </div>
  <div className="flex items-center gap-2">
    <button
      onClick={() => buildInitialTrie(words)}
      className="flex items-center gap-1 bg-slate
        transition-colors"
      title="Сбросить автомат"
    >
      <RotateCcw className="w-4 h-4" /> Сбросить
    </button>
  </div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
  {/* Аватар Лосося (Сеня) с анимацией моргания и
  <div className="flex flex-col items-center just
    <div className="w-40 h-40 relative flex items
      full border border-blue-500/30 shadow-inner
      {/* SVG Лосося */}
      <svg
        viewBox="0 0 200 200"
        className={`w-36 h-36 transition-transform
          isTalking ? 'scale-105 drop-shadow-[0_0_
        ]`}
      >
        {/* Тело лосося */}
        <ellipse cx="100" cy="100" rx="70" ry="45"
        <path d="M 40 85 Q 100 55 160 85 Q 100 115
          40 85"
          fill="#be123c"
          className="origin-[35px_100px] animate-
        />

```

```

    </span>
  </div>

  {/* Пузырь с текстом (Баббл) */}
  <div className="md:col-span-2 bg-slate-800 border-2 border-t-transparent min-h-[140px]">
    {/* Треугольник баббла */}
    <div className="absolute -left-3 top-1/2 -transform rotate(-45deg) border-b-slate-800 border-b-[12px] border-b-transparent" />

    <div className="flex items-center space-x-2 text-white">
      <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
      <span>Прямое вещание из аквариума:</span>
    </div>

    <p className="text-slate-200 text-base leading-relaxed">
      {speechText}
    </p>

    <div className="mt-4 pt-3 border-t border-slate-700">
      <div className="text-xs text-slate-400 flex justify-between">
        <Info className="w-3.5 h-3.5 text-blue-400" />
        Статус: {simulationStep === 0 ? 'Бор готов к запуску!' : 'Бор работает!'}
      </div>

      <div className="flex gap-2">
        {simulationStep === 0 && (
          <button
            onClick={startBFS}
            className="bg-emerald-600 hover:bg-emerald-500 text-white px-2 py-1 shadow-lg shadow-emerald-900/40 transition" />
            <Play className="w-4 h-4" /> Запустить
          </button>
        )}
        {simulationStep === 1 && (
          <button
            onClick={stopBFS}
            className="bg-red-600 hover:bg-red-500 text-white px-2 py-1 shadow-lg shadow-red-900/40 transition" />
            <Stop className="w-4 h-4" /> Остановить
          </button>
        )}
      </div>
    </div>
  </div>

```

```

return (
  <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
    <defs>
      <marker id="arrowhead" markerWidth="6"
        <polygon points="0 0, 6 3, 0 6" fill=
      </marker>
      <marker id="fail-arrow" markerWidth="6"
        <polygon points="0 0, 6 3, 0 6" fill=
      </marker>
    </defs>

    {/* Draw Edges */}
    {Object.values(nodes).map(node => {
      if (node.id === 0) return null;
      const parent = nodes[node.parent];
      if (!parent.x || !parent.y || !node.x ||

      const isCurrent = currentNode === node.

      return (
        <g key={`edge-${node.id}`}>
          <line
            x1={parent.x}
            y1={parent.y + 16}
            x2={node.x}
            y2={node.y - 16}
            stroke={isCurrent ? '#60a5fa' : '#c0c0c0'}
            strokeWidth="2"
            markerEnd="url(#arrowhead)"
          />
          <text
            x={({parent.x + node.x} / 2 - 10)}
            y={({parent.y + node.y} / 2}
            fill="#94a3b8"
            fontSize="12"
            fontWeight="bold"
          >
            {node.char}

```

```

let fill = '#1e293b';
let stroke = '#475569';

if (isCurrent) {
  fill = '#2563eb';
  stroke = '#60a5fa';
} else if (isInQueue) {
  fill = '#b45309';
  stroke = '#fbbf24';
} else if (node.is_terminal) {
  fill = '#059669';
  stroke = '#34d399';
}

return (
  <g key={`node-${node.id}`}>
    <circle
      cx={node.x}
      cy={node.y}
      r="16"
      fill={fill}
      stroke={stroke}
      strokeWidth="2"
    />
    <text
      x={node.x}
      y={node.y + 4}
      fill="white"
      fontSize={isRoot ? "10" : "12"}
      fontWeight="bold"
      textAnchor="middle"
    >
      {isRoot ? 'R' : node.char}
    </text>
    {!isRoot && (
      <text
        x={node.x + 12}

```

```

        }}}
      </div>
    </div>

    <form onSubmit={handleAddWord} className="flex flex-col gap-2" >
      <input
        type="text"
        value={newWord}
        onChange={(e) => setNewWord(e.target.value)}
        placeholder="НОВОЕ СЛОВО..."
        maxLength={8}
        className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
      />
      <button
        type="submit"
        className="bg-slate-700 hover:bg-slate-600 transition-colors"
      >
        <Plus className="w-3.5 h-3.5" /> Добавить
      </button>
    </form>
  </div>

  {/* Таблица Вершин и суффиксных ссылок */}
  <div className="lg:col-span-2 bg-slate-900/50 p-4" >
    <h5 className="text-white font-bold mb-3 flex items-center" >
      <span className="flex items-center gap-1.5" >
        <span> </span> Таблица переходов и fail-ссылок
      </span>
      <span className="text-xs text-slate-400 font-normal" >
        Всего вершин: {Object.keys(nodes).length}
      </span>
    </h5>

    <div className="max-h-[220px] overflow-y-auto" >
      <table className="w-full text-left border-collapse" >
        <thead>

```

```

        <span>#{node.id}</span>
        {node.id === 0 && <span className=
</td>
<td className="py-2.5 px-3">
    <span className="bg-slate-800 b
        {node.char}
    </span>
</td>
<td className="py-2.5 px-3 text-s
    {node.id === 0 ? '-' : `#${node
</td>
<td className="py-2.5 px-3">
    {node.id === 0 ? {
        <span className="text-slate-5
    } : {
        <span className="bg-indigo-95
            #{node.fail}
        </span>
    }}
</td>
<td className="py-2.5 px-3">
    {node.id === 0 || node.term_lin
        <span className="text-slate-5
    } : {
        <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Ск
            #{node.term_link}
        </span>
    }}
</td>
<td className="py-2.5 px-3 text-s
    {Object.keys(node.children).len
    ? 'ø'
    : Object.entries(node.children
        .map(([c, id]) => `${c}→#
        .join(', '))}
</td>
<td className="py-2.5 px-3">

```

```
-----
    result?: number;                                // partial
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
function buildTreeFull(arr: number[]): Record<number, number> {
    const n = arr.length;
    const tree: Record<number, number> = {};
    function go(v: number, l: number, r: number) {
        if (l === r) { tree[v] = arr[l]; return; }
        const m = (l + r) >> 1;
        go(2 * v, l, m);
        go(2 * v + 1, m + 1, r);
        tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
    }
    go(1, 0, n - 1);
    return tree;
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {
    const n = arr.length;
    const steps: Step[] = [];
    let id = 0;
    const tree: Record<number, number> = {};

    const CODE = [
        "build(v, l, r) {", // 1
        "    if (l === r) {", // 2
        "        tree[v] = A[l]; return", // 3
        "    }", // 4
        "    mid = (l + r) >> 1", // 5
        "    build(2v, l, mid)", // 6
        "    build(2v+1, mid+1, r)", // 7
        "    tree[v] = tree[2v]+tree[2v+1]", // 8
        "}", // 9
    ];

    return steps;
}
```



```

        codeLine: 2, treeState: tree, highlightNodes: [v]
    }
    return 0;
}
if (ql <= l && r <= qr) {
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}] = ${tree[v]}`, codeLine: 4, treeState: tree, highlightNodes: [v]
    return tree[v];
}
const m = (l + r) >> 1;
steps.push({ id: id++, desc: `query(${v}, [${l}..${r}] = ${tree[v]}`, codeLine: 6, treeState: tree, highlightNodes: [v]
const left = go(2 * v, l, m, ql, qr);
const right = go(2 * v + 1, m + 1, r, ql, qr);
const res = left + right;
steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`, codeLine: 8, treeState: tree, highlightNodes: [v],
    rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [res]
return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}] = ${ans}`, codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans
return steps;
}

```

```

// 3. Bottom-Up Query steps (iterative on implicit tree)
function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
    const n = arr.length;
    // Build iterative tree: leaves at [n..2n-1]
    const tree: Record<number, number> = {};
    for (let i = 0; i < n; i++) tree[n + i] = arr[i];
    for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

    const steps: Step[] = [];
    let id = 0;
    let l = qL + n;
    let r = qR + n + 1; // half-open [l, r)
    let res = 0;

```

```

        v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
    "build(v, l, r) {",
    "    if (l == r) { tree[v] = A[l]; return; }",
    "    // -----",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    build(2*v, l, mid);",
    "    build(2*v+1, mid+1, r);",
    "    tree[v] = tree[2*v] + tree[2*v+1];",
    "}"
];

```

```

const QUERY_TD_CODE = [
    "query(v, l, r, ql, qr) {",
    "    if (ql > r || qr < l) return 0;",
    "    // -----",
    "    if (ql <= l && r <= qr) return tree[v];",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    // спускаемся в обоих детей",
    "    return query(left) + query(right);",
    "}"
];

```

```

const QUERY_BU_CODE = [
    "queryBU(ql, qr) {",
    "    res = 0;",
    "    l = ql + n; r = qr + n + 1;",
    "    // -----",
    "    while (l < r) {",
    "        if (l & 1) res += tree[l++];",
    "        // -----",
    "        if (r & 1) res += tree[--r];",
    "        l >>= 1; r >>= 1;",
    "    }",
    "    return res; }"
];

```

```

];

```



```
</pre>
```

```

{ /* Description + result */
<div className="px-3 py-2.5 bg-slate-900/60 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-900/60" style={{
  px: 3, py: 2.5, bg: 'slate-900/60', border: '1px solid slate-900/60' }}>
  <span className={`mt-0.5 inline-block w-2 h-2 rounded-sm`} style={{
    mt: 0.5, display: 'inline-block', width: 2, height: 2, border: '1px solid',
    borderStyle: 'solid', color: 'emerald' ? 'text-emerald-300' : 'text-slate-400' }}>
    <span className="leading-relaxed">{step.desc}</span>
    {step.result !== undefined && {
      <span className={`ml-auto shrink-0 font-mono`} style={{
        ml: 'auto', shrink: 0, font: 'mono' }}>
        {step.result}
      </span>
    }}
  </div>

```

```

{ /* Controls */
<div className="px-3 py-2.5 bg-slate-950 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-950" style={{
  px: 3, py: 2.5, bg: 'slate-950', border: '1px solid slate-950' }}>
  <div className="flex items-center bg-slate-900 rounded-sm" style={{
    bg: 'slate-900', padding: 10, gap: 10 }}>
    <button onClick={() => { setPlaying(false); setSpeed(1200); }} style={{
      padding: 5 10, border: '1px solid', borderStyle: 'solid', color: 'white',
      background: 'slate-800', borderRadius: 5, transition: 'all 0.1s' }}>
      <RotateCcw className="w-3 h-3" />
    </button>
    <button onClick={() => { setPlaying(false); setSpeed(700); }} style={{
      padding: 5 10, border: '1px solid', borderStyle: 'solid', color: 'white',
      background: 'slate-800', borderRadius: 5, transition: 'all 0.1s' }}>
      <RotateCw className="w-3 h-3" />
    </button>
    <button onClick={() => setPlaying(!playing)} style={{
      padding: 5 10, border: '1px solid', borderStyle: 'solid', color: 'white',
      background: playing ? 'rose-500' : 'slate-800', borderRadius: 5, transition: 'all 0.1s' }}>
      {playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />
    </button>
    <button onClick={() => { setPlaying(false); setSpeed(120); }} style={{
      padding: 5 10, border: '1px solid', borderStyle: 'solid', color: 'white',
      background: 'slate-800', borderRadius: 5, transition: 'all 0.1s' }}>
      <RotateCcw className="w-3 h-3" />
    </button>
  </div>
  <select value={speed} onChange={e => setSpeed(Number(e.target.value))} style={{
    text: [9px] font-bold rounded px-1.5 py-1 outline-1 outline-slate-900/60 }}>
    <option value={1200}>Медл.</option>
    <option value={700}>Норма</option>
    <option value={350}>Быстро</option>
    <option value={120}>Турбо</option>
  </select>
</div>

```

```

const buildSteps = useMemo(() => genBuildSteps(arr),
const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr),
const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr),

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
  <div className="bg-slate-900 rounded-2xl border border-indigo-500" >
    /* ---- TOP BAR: array editor + query range ---- */
    <div className="mb-6 space-y-4">
      <div className="flex flex-col lg:flex-row gap-4">
        /* Array input */
        <form onSubmit={handleApply} className="flex flex-col">
          <div className="flex items-center justify-between">
            <label className="text-[10px] font-bold uppercase">
              Array input
            </label>
            <button type="button" onClick={randomize} className="bg-indigo-500 text-white px-2 py-1">
              Refresh
            </button>
          </div>
          <div className="flex gap-2">
            <input
              type="text"
              value={customInput}
              onChange={e => setCustomInput(e.target.value)}
              className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
              placeholder="5, 8, 3, 12, 7, 2"
            />
            <button type="submit" className="bg-indigo-500 text-white px-2 py-1" >
              Применить
            </button>
          </div>
          <div className="flex flex-wrap gap-1.5 pt-1">
            {arr.map((v, i) => (
              <span key={i} className="bg-slate-900 border border-indigo-500 padding-2px">
                <span className="text-slate-500">A[{i}]</span>
                <span className="text-slate-500">{v}</span>
              </span>
            ))}
            <span className="text-xs text-slate-500 font-roboto">
              Total sum: {totalSum}
            </span>
          </div>
        </form>
      </div>
    </div>
  )
)

```

```

{ /* ---- TABS ---- */
<div className="flex flex-wrap items-center gap-1"
  <button onClick={() => setView('build')} className="border-indigo-500 border-2 p-2"
    -colors ${view === 'build' ? 'border-indigo-500' : 'border-slate-200'}`}>
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} className="border-emerald-500 border-2 p-2"
    on-colors ${view === 'queries' ? 'border-emerald-500' : 'border-slate-200'}`}>
    <Search className="w-3.5 h-3.5" /> Запрос: св
  </button>
  <button onClick={() => setView('theory')} className="border-blue-500 border-2 p-2"
    n-colors ${view === 'theory' ? 'border-blue-500' : 'border-slate-200'}`}>
    <Info className="w-3.5 h-3.5" /> Почему / Сравн
  </button>
</div>

```

```

{ /* ---- TAB CONTENT ---- */
{view === 'build' && (
  <SimPanel
    key={`build-${arr.join('-')}`}
    title="Построение дерева (рекурсия)"
    icon=" "
    steps={buildSteps}
    code={BUILD_CODE}
    color="indigo"
    arr={arr}
  />
)}

```

```

{view === 'queries' && (
  <div className="grid grid-cols-1 xl:grid-cols-2"
    <SimPanel
      key={`qtd-${arr.join('-')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Сверху

```



```
export const Sidebar: React.FC<SidebarProps> = ({ selectQuery }) => {
  const [query, setQuery] = useState("");
```

```
  const groups = useMemo(() => {
    const q = query.trim().toLowerCase();
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
    const map = new Map<string, typeof chapters>();
    for (const c of filtered) {
      const key = c.category?.trim() || "Прочие темы";
      if (!map.has(key)) map.set(key, []);
      (map.get(key) as typeof chapters).push(c);
    }
    return Array.from(map.entries());
  }, [query]);
```

```
  return (
    <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:p-4">
      <div className="p-4 pb-3 shrink-0">
        <h3 className="text-xs font-bold text-slate-400">
          <Compass className="w-4 h-4 text-indigo-400" />
        </h3>
        <div className="relative">
          <Search className="w-4 h-4 text-slate-500 absolute right-0 top-0">
            <input
              type="text"
              value={query}
              onChange={(e) => setQuery(e.target.value)}
              placeholder="Поиск по темам..."
              className="w-full bg-slate-800/70 border border-slate-500 focus:outline-none focus:border-indigo-500"
            />
            {query && (
              <button
                type="button"
                onClick={() => setQuery("")}
                className="absolute right-2 top-1/2 -translate-y-1/2 text-slate-500"
                aria-label="Очистить поиск"
              />
            )}
          </div>
        </div>
      </div>
    </aside>
  );
}
```



```

        }` }
      />
    </button>
  );
  }}}
</div>
</div>
  )})
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-l
  ] text-slate-300 space-y-1.5 hidden lg:block">
  <div className="font-bold text-indigo-400 flex .
    <Sparkles className="w-3.5 h-3.5" /> Как поль
  </div>
  <p className="leading-relaxed">
    Подчёркнутые термины в тексте раскрываются по
    симулятора.
  </p>
  <p className="text-slate-400 italic">Стрелки ←
  </div>
</aside>
  );
};

```

ФАЙЛ 58/79: `src/components/Simulator.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

```

```

interface Plate {
  id: string;
  name: string;
  icon: string;

```

```

    });
    const [newCustomText, setNewCustomText] = useState('');
    const [newCustomProb, setNewCustomProb] = useState(0);
    const [heapMessage, setHeapMessage] = useState<string>('');

    // Stack handlers
    const addPlate = () => {
        const presets = [
            { name: 'Сковорода от омлета', icon: '🍳' },
            { name: 'Кастрюля от супа', icon: '🍲' },
            { name: 'Чашка от кофе', icon: '☕' },
            { name: 'Миска с остатками салата', icon: '🥗' },
            { name: 'Тарелка от тако', icon: '🌮' },
        ];
        const randomPreset = presets[Math.floor(Math.random() * presets.length)];
        const newPlate = {
            id: Date.now().toString(),
            name: randomPreset.name,
            icon: randomPreset.icon
        };
        setStack(prev => [...prev, newPlate]);
        setPopMessage(`Положили поверх стопки: ${newPlate.name}`);
    };

    const popPlate = () => {
        if (stack.length === 0) {
            setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
            return;
        }
        const topPlate = stack[stack.length - 1];
        setStack(prev => prev.slice(0, prev.length - 1));
        setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name}`);
    };

    const resetStack = () => {
        setStack([
            { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
            { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
        ]);
    };

```

```

    });
    setDequeueMessage(" Очередь восстановлена.");
  };

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCustomText.trim()) return;
  const item: Hypothesis = {
    id: Date.now().toString(),
    text: `Кандидат: "${newCustomText}"`,
    probability: Number(newCustomProb)
  };
  setHeap(prev => [...prev, item]);
  setNewCustomText('');
  setHeapMessage(` Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
  if (heap.length === 0) {
    setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
    return;
  }
  // Find highest probability
  let maxIdx = 0;
  for (let i = 1; i < heap.length; i++) {
    if (heap[i].probability > heap[maxIdx].probability) {
      maxIdx = i;
    }
  }
  const best = heap[maxIdx];
  setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
  setHeapMessage(` Выбран самый "тяжелый" кандидат: `);
};

const resetHeap = () => {
  setHeap([
    { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"',

```

```

        <span className="text-xs bg-blue-500/20 text-left" style={activeTab === 'DFS' ? {} : {}}>
          DFS
        </span>
      </button>

      <button
        onClick={() => setActiveTab('queue')}
        className={`flex items-center justify-between border-1px rounded p-2
          ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-white' : 'bg-slate-800/60 border-slate-700/60 text-slate-200'}
        `}
      >
        <div className="flex items-center gap-3">
          <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-white' : 'text-slate-200'} rounded`} style={activeTab === 'queue' ? {} : {}}>
            <div className="text-left">
              <div className="font-bold text-sm text-white">Queue</div>
              <div className="text-xs opacity-80">FIFO</div>
            </div>
          </div>
          <span className="text-xs bg-indigo-500/20 text-left" style={activeTab === 'queue' ? {} : {}}>
            BFS
          </span>
        </button>

      <button
        onClick={() => setActiveTab('heap')}
        className={`flex items-center justify-between border-1px rounded p-2
          ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500 text-white' : 'bg-slate-800/60 border-slate-700/60 text-slate-200'}
        `}
      >
        <div className="flex items-center gap-3">
          <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-white' : 'text-slate-200'} rounded`} style={activeTab === 'heap' ? {} : {}}>
            <div className="text-left">
              <div className="font-bold text-sm text-white">Heap</div>
              <div className="text-xs opacity-80">Приоритет</div>
            </div>
          </div>
          <span className="text-xs bg-emerald-500/20 text-left" style={activeTab === 'heap' ? {} : {}}>
            BFS
          </span>
        </button>
      </div>
    </div>
  </div>
</div>

```

```

        onClick={resetStack}
        title="Сбросить"
        className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
      >
        <RotateCcw className="w-5 h-5" />
      </button>
    </div>
  </div>

  {popMessage && {
    <div className="bg-blue-950/60 border borde
imate-fade-in">
      <span className="inline-block w-2 h-2 roun
        {popMessage}
      </div>
    }}

  {/* Visualization */}
  <div className="bg-slate-950/60 p-6 rounded-2
    >
    {stack.length === 0 ? {
      <div className="text-center py-12 text-sl
        <div className="text-5xl mb-3">    </div>
        <p className="text-base font-bold">Стон
        <p className="text-xs mt-1">Добавьте гр
      </div>
    } : {
      <div className="w-full max-w-sm flex flex
        <div className="absolute top-0 text-xs
full flex items-center gap-1">
        <span>ВЕРШИНА СТЕКА (TOP)</span>
        <ArrowDown className="w-3.5 h-3.5 anim
      </div>

      {/* Reversed map so top of stack is vis
      {stack.slice().reverse().map((plate, ind
        const isTop = index === 0;

```

```

<div>
  <h3 className="text-lg font-bold text-white">
    <span> Симуляция очереди за супом</span>
    <span className="text-xs font-mono bg-indigo-950/60 border border-indigo-950/60 padding-2">
      /span>
    </h3>
    <p className="text-slate-400 text-xs mt-1">
      Кто первым пришел, тот первым получил суп
    </p>
  </div>
  <div className="flex flex-wrap items-center">
    <button
      onClick={addPerson}
      className="flex-1 md:flex-none flex item-center justify-center gap-2.5 rounded-xl text-sm font-bold shadow-lg"
    >
      <Plus className="w-4 h-4" /> Встать в очередь
    </button>
    <button
      onClick={servePerson}
      className="flex-1 md:flex-none flex item-center justify-center gap-2.5 rounded-xl border-indigo-500/40 px-4 py-2.5 rounded"
    >
      Налить суп первому
    </button>
    <button
      onClick={resetQueue}
      title="Сбросить"
      className="p-2.5 bg-slate-800 hover:bg-slate-700 text-white transition-colors cursor-pointer"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
  </div>
</div>

```

```

{dequeueMessage && {
  <div className="bg-indigo-950/60 border border-indigo-950/60 padding-2">

```

```

        >
        {isFirst && {
            <span className="absolute -top-
se tracking-wider shadow">
                Первый (Head)
            </span>
        }}
        <span className="text-4xl mb-2">{
        <span className={`font-bold text-
            {person.name}
        </span>
        <span className={`text-[10px] fon
            isFirst ? 'bg-indigo-500/30 tex
        }`}>
            {isFirst ? 'Получает сун' : `В
        </span>
        </div>
    });
    }}}

    <div className="flex flex-col items-cen
-600 min-w-[120px] h-[120px]">
        <Plus className="w-6 h-6 mb-1" />
        <span className="text-xs font-mono up
    </div>
    </div>
    })
    <div className="mt-6 text-xs text-slate-500
        ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
    </div>
    </div>
    </div>
    })

    {/* HEAP TAB */}
    {activeTab === 'heap' && {
        <div className="space-y-6">
            <div className="bg-slate-800/80 p-5 rounded-x

```

```

        onChange={e => setNewCustomText(e.target.value)}
        placeholder='например: "Привет, я лучший"'
        className="w-full bg-slate-900 border border-emerald-500 transition-colors"
      />
    </div>
    <div className="md:col-span-3">
      <label className="block text-xs font-bold">
        Вероятность: <span className="text-emerald-500">
      </label>
      <input
        type="range"
        min="0.01"
        max="0.99"
        step="0.01"
        value={newCustomProb}
        onChange={e => setNewCustomProb(Number(e.target.value))}
        className="w-full accent-emerald-500 cursor-pointer"
      />
    </div>
    <div className="md:col-span-3">
      <button
        type="submit"
        disabled={!newCustomText.trim()}
        className="w-full flex items-center justify-center gap-2 bg-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed h-10"
      >
        <Plus className="w-4 h-4" /> Добавить в список
      </button>
    </div>
  </form>

```

```

{heapMessage && (
  <div className="bg-emerald-950/60 border border-emerald-500/40 gap-3 animate-fade-in">
    <span className="inline-block w-2 h-2 rounded-full" style={heapMessage} />
  </div>
)}

```



```

    const q = query.trim().toLowerCase();
    if (!q) return all;
    return all.filter(
      (i) =>
        i.entry.title.toLowerCase().includes(q) ||
        (i.entry.hint ?? "").toLowerCase().includes(q)
        (i.chapterTitle ?? "").toLowerCase().includes(q)
    );
  }, [query]);

return (
  <div>
    <div className="mb-6">
      <h2 className="text-xl sm:text-2xl font-extrabold">
        <p className="text-sm text-slate-400 leading-relaxed">
          Все интерактивные демонстрации в одном месте.
          здесь их можно открыть напрямую.
        </p>
      </div>

      <div className="relative mb-6 max-w-md">
        <Search className="w-4 h-4 text-slate-500 absolute">
          <input
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder="Поиск тренажёра..."
            className="w-full bg-slate-900 border border-
              focus:outline-none focus:border-indigo-500"
          />
          {query && (
            <button
              onClick={() => setQuery("")}
              className="absolute right-2 top-1/2 -translate-y-1/2"
              aria-label="Очистить поиск"
            >
              <X className="w-4 h-4" />
            </button>
          )}
        </div>
      </div>
    </div>
  )
)

```

```
    });  
};
```

ФАЙЛ 60/79: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useCallback, useMemo, useState } from "react";  
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";
```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерево.
```

```
 * поднять отмеченный узел в корень. Клик по узлу = один шаг.
```

```
 * Порядок поворотов выбираешь сам; правильный ли он пока не знаем.
```

```
 * только после того, как узел окажется наверху (или пометить узел).
```

```
 */
```

```
export interface TNode {  
  key: number;  
  left: TNode | null;  
  right: TNode | null;  
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null });
```

```
const clone = (t: TNode | null): TNode | null =>  
  t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и просеивание
```

```
export const PRESETS: { name: string; hintCase: string; target: number; }[] =
```

```
{
```

```
  name: "Линия влево",
```

```
  hintCase: "Zig-Zig",
```

```
  target: 1,
```

```
export const findPath = (root: TNode | null, key: number): TNode[] => {
  const path: TNode[] = [];
  let cur = root;
  while (cur) {
    path.push(cur);
    if (key === cur.key) return path;
    cur = key < cur.key ? cur.left : cur.right;
  }
  return [];
};
```

```
/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
  const path = findPath(root, key);
  if (path.length < 2) return root;

  const x = path[path.length - 1];
  const p = path[path.length - 2];
  const gg = path.length >= 3 ? path[path.length - 3] : null;

  if (p.left === x) {
    p.left = x.right;
    x.right = p;
  } else {
    p.right = x.left;
    x.left = p;
  }

  if (!gg) return x;
  if (gg.left === p) gg.left = x;
  else gg.right = x;
  return root;
}
```

```
/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
  const path = findPath(root, key);
```

```

const rows = Math.max(1, Math.max(...nodes.map((n) =>
const colW = 46;
const rowH = 62;
return {
  nodes: nodes.map((n) => ({ ...n, x: n.x * colW + co
  edges,
  width: cols * colW,
  height: rows * rowH + 20,
});
}

```

/* ----- КОМПОНЕНТ -----

```

export const SplayRotationSandbox: React.FC = () => {
  const [presetIdx, setPresetIdx] = useState(0);
  const preset = PRESETS[presetIdx];

  const [tree, setTree] = useState<TNode>(() => preset.t
  const [history, setHistory] = useState<TNode[]>([]);
  const [moves, setMoves] = useState<{ node: number; co
  const [showAnswer, setShowAnswer] = useState(false);

  const target = preset.target;
  const solved = tree.key === target;

  const reset = useCallback(
    (idx = presetIdx) => {
      setPresetIdx(idx);
      setTree(PRESETS[idx].build());
      setHistory([]);
      setMoves([]);
      setShowAnswer(false);
    },
    [presetIdx]
  );

  const { nodes, edges, width, height } = useMemo(() =>
  const path = useMemo(() => new Set(findPath(tree, tar

```

```

    >
    <Dices className="w-3.5 h-3.5" /> Другое дере
  </button>
</div>

```

```

<div className="flex gap-2 mb-3 overflow-x-auto n
  {PRESETS.map((p, i) => {
    <button
      key={p.name}
      onClick={() => reset(i)}
      className={`px-3 py-1.5 rounded-lg text-[11
        i === presetIdx ? "bg-indigo-600 text-whi
      }`}
    >
      {p.name}
    </button>
  )})}
</div>

```

```

<div className="bg-slate-900 rounded-xl border bo
  <svg
    viewBox={`0 0 ${width} ${height}`}
    className="h-auto block mx-auto"
    style={{ minWidth: Math.min(width * 1.5, 420)
    role="img"
  >
    {edges.map(([a, b], i) => {
      const na = nodes.find((n) => n.key === a);
      const nb = nodes.find((n) => n.key === b);
      if (!na || !nb) return null;
      const onPath = path.has(a) && path.has(b);
      return (
        <line
          key={i}
          x1={na.x}
          y1={na.y}
          x2={nb.x}
          y2={nb.y}

```

```

<div className="flex flex-wrap items-center gap-2"
  <button
    onClick={undo}
    disabled={history.length === 0}
    className="flex items-center gap-1.5 px-3 py-1
      text-white disabled:opacity-40 text-xs font-bol
  >
    <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
  </button>
  <button
    onClick={() => reset()}
    className="flex items-center gap-1.5 px-3 py-1
      text-white text-xs font-bold transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" /> Заново
  </button>
  <button
    onClick={() => setShowAnswer((s) => !s)}
    className={`flex items-center gap-1.5 px-3 py-1
      border border-slate-200
      ${showAnswer
        ? "bg-amber-600/20 border-amber-500 text-amber-500"
        : "bg-slate-800 border-slate-700 text-slate-200"}
    `}
  >
    {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
  </button>

  <span className="text-[11px] text-slate-500 ml-2"
    поворотов: {moves.length} • минимум: {minimalMoves}
  </span>
</div>

```

```

{showAnswer && !solved && expected && (
  <div className="mt-3 text-[12px] bg-amber-950/40 p-3"
    Сейчас splay крутил бы узел{" "}
    <span className="font-mono font-bold">{expected}
  </div>
)}

```

ФАЙЛ 61/79: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";
```

```
/**
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
 *
 * Анимация намеренно медленная и разбита на два типа кадров:
 * * «прицел» – пара узлов, которую сейчас будут крутить.
 * * «результат» – узлы плавно переезжают на новые места.
 * Без такой паузы поворот выглядел как рывок, по которому
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
```

```
interface Frame {
  pos: Partial<Record<NodeId, [number, number]>>;
  edges: [NodeId, NodeId][];
  title: string;
  text: string;
  /** Пара, которую крутим на этом шаге: кадр-«прицел». */
  focus?: [NodeId, NodeId];
  /** Кто переехал на этом кадре – подсвечиваем кольцом. */
  moved?: NodeId[];
  ms: number;
}
```

```
interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
```



```

        moved: ["x", "p", "B"],
        ms: MOVE_MS,
    },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
    pos: { g: [230, 35], D: [310, 105], p: [140, 105], C:
    edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as const;

const ZZ_S1 = {
    pos: { p: [150, 35], x: [70, 105], C: [195, 105], g:
    edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as const;

const ZIGZIG: Case = {
    key: "zig-zig",
    label: "Zig-Zig",
    when: "x и p – дети с ОДНОЙ стороны (оба слева или об
    rotations: "2 поворота, первым – верхний",
    frames: [
        {
            ...(ZZ_S0 as unknown as Pick<Frame, "pos" | "edge
            title: "До: прямая линия g → p → x",
            text: "Три узла выстроились в «бамбук» – самая не
            ms: MOVE_MS,
        },
        {
            ...(ZZ_S0 as unknown as Pick<Frame, "pos" | "edge
            title: "Прицел 1: ВЕРХНЯЯ пара (g, p)",
            text: "Главная хитрость случая: первым крутим дед
            focus: ["g", "p"],
            ms: FOCUS_MS,
        },
        {
            ...(ZZ_S1 as unknown as Pick<Frame, "pos" | "edge

```

```

rotations: "2 поворота, первым – нижний",
frames: [
  {
    ...(ZG_S0 as unknown as Pick<Frame, "pos" | "edge"
    title: "До: g → p влево, p → x вправо",
    text: "Узлы идут «змейкой». Здесь порядок обратный",
    ms: MOVE_MS,
  },
  {
    ...(ZG_S0 as unknown as Pick<Frame, "pos" | "edge"
    title: "Прицел 1: НИЖНЯЯ пара (p, x)",
    text: "Дед g пока стоит на месте и не участвует.",
    focus: ["p", "x"],
    ms: FOCUS_MS,
  },
  {
    ...(ZG_S1 as unknown as Pick<Frame, "pos" | "edge"
    title: "Шаг 1 выполнен: змейка выпрямилась",
    text: "x занял место p и забрал его к себе левым",
    moved: ["x", "p", "C"],
    ms: MOVE_MS,
  },
  {
    ...(ZG_S1 as unknown as Pick<Frame, "pos" | "edge"
    title: "Прицел 2: пара (g, x)",
    text: "Остался одиночный поворот вокруг деда – и",
    focus: ["g", "x"],
    ms: FOCUS_MS,
  },
  {
    pos: { x: [170, 45], p: [80, 125], g: [265, 125],
    edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
    title: "Готово: p и g – два ребёнка x",
    text: "Дерево стало симметричным: все четыре подд
    moved: ["x", "p", "g", "C"],
    ms: MOVE_MS,
  },
],

```

```

    {(Object.keys(pos) as NodeId[]).map((id) => {
      const p = pos[id];
      if (!p) return null;
      const c = COLOR[isSubtree(id) ? "sub" : id];
      const r = isSubtree(id) ? 15 : 19;
      const hot = Boolean(focus && focus.includes(id));
      const justMoved = Boolean(moved?.includes(id));
      return (
        <g
          key={id}
          style={{
            transform: `translate(${p[0]}px, ${p[1]}px)`,
            transition: TRANSITION,
            opacity: dim(id) ? 0.3 : 1,
          }}
        >
          {(hot || justMoved) && (
            <circle
              r={r + 7}
              fill="none"
              stroke={hot ? "#fbbf24" : "#a5b4fc"}
              strokeWidth={2}
              strokeDasharray="4 4"
              opacity={0.9}
            />
          )}
          <circle r={r} fill={c.fill} stroke={c.stroke} />
          <text
            textAnchor="middle"
            dominantBaseline="central"
            fontSize={isSubtree(id) ? 12 : 14}
            fontWeight="700"
            fill={isSubtree(id) ? "#94a3b8" : "#fff"}
          >
            {id}
          </text>
        </g>
      )
    })
  }

```

```

        {c.label}
      </button>
    )))
  </div>

  <div className="mb-3 text-xs sm:text-[13px] text-
    <span className="font-bold text-indigo-300">Kor,
    <span className="text-slate-500">({active.rotat
  </div>

  <div className="bg-slate-900 rounded-xl border bo
    <Tree frame={current} />
  </div>

  {/* индикатор: сколько осталось до следующего кадра */}
  <div className="h-1 w-full bg-slate-800 rounded-f
    <div
      key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
      className="h-full bg-indigo-500"
      style={
        playing
          ? { animation: `demoProgress ${duration}m
            : { width: "100%", background: "#475569"
        }
      }
    />
  </div>

  <div className="mt-3 min-h-[76px]">
    <div className="flex items-center gap-2 mb-1">
      <span
        className={`text-[10px] font-bold uppercase
          current.focus ? "bg-amber-500/20 text-amb
        }`}
      >
        {current.focus ? "прицел" : "результат"}
      </span>
      <span className="text-sm font-bold text-white
        {frame + 1}/{total}. {current.title}

```

```

        slow
        ? "bg-emerald-600/20 border-emerald-500 text-slate-500"
        : "bg-slate-800 border-slate-700 text-slate-500"
      )` }
    >
    <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "0.2"}
  </button>
  <button
    onClick={() => {
      setFrame(0);
      setPlaying(false);
    }}
    className="flex items-center gap-1.5 px-3 py-3 text-xs font-bold transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" /> Сброс
  </button>

  <div className="flex items-center gap-3 ml-auto" >
    <span className="flex items-center gap-1.5">
      <span className="w-3 h-3 rounded-full" style={bg} />
    </span>
    <span className="flex items-center gap-1.5">
      <span className="w-3 h-3 rounded-full" style={border} />
    </span>
    <span className="flex items-center gap-1.5">
      <span className="w-3 h-3 rounded-full" style={text} />
    </span>
  </div>
</div>
</div>
);
};

```

```

const leftRotate = (x: SplayNode): SplayNode => {
  const y = x.right;
  if (!y) return x;
  x.right = y.left;
  y.left = x;
  return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
  if (!node) return null;
  return {
    key: node.key,
    left: cloneTree(node.left),
    right: cloneTree(node.right),
    x: node.x,
    y: node.y,
  };
};

export const SplayTreeViz: React.FC = () => {
  // Let's create an elegant initial unbalanced or semi-balanced tree
  // Values: 10, 20, 30, 40, 50, 60
  const createInitialTree = (): SplayNode => {
    let r: SplayNode | null = null;
    const initialKeys = [40, 20, 50, 10, 30, 60];
    initialKeys.forEach((key) => {
      r = insertBST(r, key);
    });
    return r!;
  };

  const [tree, setTree] = useState<SplayNode>(createInitialTree());
  const [inputValue, setInputValue] = useState<string>('');
  const [log, setLog] = useState<string[]>([
    'Дерево инициализировано. Попробуйте кликнуть на узел.',
  ]);
  const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

```

```

    } else {
      steps.push(
        `Поиск ${key}: элемент найден! Запускаем подъем
      );
    }
  }

// Now let's implement standard Splay logic recursively
const splayAction = (
  node: SplayNode | null,
  k: number,
): SplayNode | null => {
  if (!node || node.key === k) return node;

  if (k < node.key) {
    if (!node.left) return node;

    if (k < node.left.key) {
      // Zig-Zig (Left-Left)
      node.left.left = splayAction(node.left.left,
      node = rightRotate(node);
      steps.push(
        `Вращение Zig-Zig (Правый поворот родителя
      );
    } else if (k > node.left.key) {
      // Zig-Zag (Left-Right)
      node.left.right = splayAction(node.left.right
      if (node.left.right) {
        node.left = leftRotate(node.left);
        steps.push(`Компонент Zig-Zag: левый поворо
      }
    }
  }

  if (!node.left) return node;
  steps.push(
    `Финальный поворот Zig (Правое вращение корня
  );
  return rightRotate(node);
} else {

```

```
// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
  e.preventDefault();
  const val = parseInt(inputValue);
  if (isNaN(val)) return;

  // Standard BST insert, then splay!
  let newTree = cloneTree(tree);
  newTree = insertBST(newTree, val);
  const { newRoot, steps } = splayStepByStep(newTree,
  if (newRoot) {
    setTree(newRoot);
  }
  setLog([`Вставили вершину [${val}].`, ...steps]);
  setInputValue("");
  setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
  e.preventDefault();
  const val = parseInt(inputValue);
  if (isNaN(val)) return;
  handleSplay(val);
  setInputValue("");
};

const resetTree = () => {
  setTree(createInitialTree());
  setHighlightedNode(null);
  setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursive
const computeCoordinates = (
  node: SplayNode | null,
  x: number,
  y: number,
```



```

        x1={node.x}
        y1={node.y}
        x2={node.right.x}
        y2={node.right.y}
        stroke="#475569"
        strokeWidth="2"
      />,
    );
    lines = lines.concat(renderLines(node.right));
  }
  return lines;
};

const renderNodes = (node: SplayNode | null): React.R
  if (!node) return [];
  let circles: React.ReactNode[] = [];
  const isHighlighted = highlightedNode === node.key;

  circles.push(
    <g
      key={`n-${node.key}`}
      className="cursor-pointer group"
      onClick={() => handleSplay(node.key)}
      data-title={`Кликните, чтобы выполнить Splay(${
    >
    <circle
      cx={node.x}
      cy={node.y}
      r="18"
      fill={isHighlighted ? "#4f46e5" : "#1e293b"}
      stroke={isHighlighted ? "#818cf8" : "#64748b"}
      strokeWidth={isHighlighted ? "3" : "2"}
      className="transition-all duration-300 group-l
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"

```

```
</svg>
```

```
{/* Quick interactive controls */}
```

```
<div className="absolute bottom-4 left-4 right-
```

```
  <form onSubmit={handleFind} className="flex
```

```
    <input
```

```
      type="number"
```

```
      placeholder="Поиск..."
```

```
      value={inputValue}
```

```
      onChange={(e) => setInputValue(e.target
```

```
      className="w-24 bg-slate-950 text-white
```

```
00 focus:outline-none"
```

```
  />
```

```
  <button
```

```
    type="submit"
```

```
    onClick={handleFind}
```

```
    className="bg-indigo-600 hover:bg-indigo-
```

```
  >
```

```
    Поиск / Splay
```

```
</button>
```

```
<button
```

```
  type="button"
```

```
  onClick={(e) => {
```

```
    e.preventDefault();
```

```
    const val = parseInt(inputValue);
```

```
    if (!isNaN(val)) {
```

```
      let newTree = cloneTree(tree);
```

```
      newTree = insertBST(newTree, val);
```

```
      const { newRoot, steps } = splaySte
```

```
      if (newRoot) setTree(newRoot);
```

```
      setLog([`Вставили [${val}].`, ...st
```

```
      setInputValue("");
```

```
      setHighlightedNode(val);
```

```
    }
```

```
  }}
```

```
  className="bg-indigo-950 hover:bg-slate
```

```
0"
```

```
>
```

```

        </p>
        <p>
            <span className="font-semibold text-slate-300">
                оба левые или оба правые. Вращаем деда, поворачиваем
            </p>
        <p>
            <span className="font-semibold text-slate-300">
                колено. Вращаем узел в разные стороны.
            </p>
        </div>
    </div>
</div>

/* Answers Section inside the visualizer widget
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
    <h4 className="text-indigo-400 font-bold text-slate-300">
        <HelpCircle className="w-4 h-4 text-indigo-400"/>
        Разбор экзаменационных вопросов (Без нытья):
    </h4>

    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
        <div className="bg-slate-900/60 p-4 rounded-lg">
            <div>
                <p className="font-bold text-slate-300 mb-2">
                    1. Отсутствующий элемент
                </p>
                <p className="text-slate-400 leading-relaxed">
                    Допустим, мы ищем{" "}
                    <code className="text-indigo-400 font-sans">
                        его нет. Алгоритм спустится к листу, на котором
                    </code>
                    неудачей (например,{" "})
                    <code className="text-indigo-400">[20]</code>
                    <code className="text-indigo-400">[30]</code>
                    поднимет в корень{" "}
                    <strong className="text-white">
                        последний успешно просмотренный узел
                    </strong>
                    , на котором он споткнулся! Это оставляе

```

```

        всю структуру вдвое более плоской за счет
        при двойных вращениях {
        <code className="text-emerald-400 font-
        }. Дорогая операция инвестирует в дешевые
        операций. Поэтому амортизированное время
        <strong className="text-white"> $O(\log n)$ 
      </p>
    </div>
    <div className="mt-3 text-[10px] text-emerald-400">
      Каждое "растяжение" компенсируется "сжатием"
    </div>
  </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 63/79: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

```

```

interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}

```

```

const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: '🍝', color:

```

```

    counter++;

    const newPlate: Plate = {
      id: Date.now().toString(),
      ...preset,
      isDirty: true,
      animState: 'in',
    };

    setDirtyStack(prev => [...prev, newPlate]);
    addLog(`  PUSH: Положили ${newPlate.emoji} сверху с

    setTimeout(() => {
      setDirtyStack(prev => prev.map(p => p.id !== newP
    }, 500);
  }, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {
    setShake(true);
    addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${top

// Step 1: Start washing animation (sponge and soap
setDirtyStack(prev => prev.map(p => p.id !== topPla
setShowSoap(true);

setTimeout(() => {
  // Step 2: Wash complete. Plate becomes clean and

```

```

    <h3 className="font-black text-blue-400 text-
    <p className="text-xs text-slate-300 mt-1 lea
      Ты складываешь грязные тарелки друг на друга.
      А когда приходит время мыть, ты берёшь именн
      Попытаешься вытащить нижнюю — всё рухнет!
    </p>
  </div>
</div>

```

```

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap
  <button
    onClick={pushPlate}
    disabled={isWashing}
    className="flex-1 min-w-[150px] bg-gradient-t
      acity-40 disabled:cursor-not-allowed text-w
      ale-95 transition-all cursor-pointer flex i
  >
    <span> Положить грязную (PUSH)</span>
  </button>

  <button
    onClick={popPlate}
    disabled={isWashing || dirtyStack.length === 0}
    className="flex-1 min-w-[150px] bg-gradient-t
      opacity-40 disabled:cursor-not-allowed text
      ve:scale-95 transition-all cursor-pointer f
  >
    <span> Помыть верхнюю (POP)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800
      r-pointer border border-slate-700"
  >
    Сброс
  </button>

```

```

) : (
  <div className={`flex-1 flex flex-col justify`

    {/* Указатель TOP */}
    <div
      className="absolute right-8 flex items-
      style={{ bottom: `${24 + topIndex * 44}`
    >
      <span className="text-xs font-mono text
er gap-1">
        <ArrowUp className="w-3.5 h-3.5" />
        <span>ВЕРШИНА (TOP)</span>
      </span>
      <span className="text-blue-400 animate-
    </div>

    {/* Тарелки (отрисовываем в обратном поряд
    {dirtyStack.slice().reverse().map((plate,
      const idx = dirtyStack.length - 1 - rev
      const isTop = idx === topIndex;
      return (
        <div
          key={plate.id}
          className={`
            w-full max-w-[280px] h-10 rounded
shadow-md select-none transition-all duration
            ${plate.color}
            ${plate.animState === 'in' ? 'ani
            ${plate.animState === 'washing' ?
            ${isTop ? 'ring-4 ring-blue-500/5
          `}
        >
          <span className="text-lg">{plate.em
          <span className="text-slate-800 tex
          {isTop && (
            <span className="text-[9px] bg-bl
              LIFO
            </span>

```

```

        <span className="text-sm"> {plate.n
        <span className="text-[10px] text-e
      </div>
    }
  </div>
}
</div>

  { /* Столешница сушилки */ }
  <div className="w-full h-4 bg-gradient-to-r f
</div>
</div>

{ /* ЛОГ ДЕЙСТВИЙ */ }
<div className="bg-slate-900 border border-slate-
  <div className="text-[10px] font-mono text-slate
    {log.map((entry, idx) => {
      <div
        key={idx}
        className={`text-xs font-mono flex items-ce
      >
        {idx === 0 ? <span className="text-blue-500
        <span>{entry}</span>
      </div>
    })}
  </div>
</div>
);
};

```

ФАЙЛ 64/79: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useRef, useMemo } from "react"
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react"

```



```

-----
    return { s, steps, patternLength: pattern.length };
}

function generateZSteps(pattern: string, text: string)
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=$` });

        if (i <= r) {
            steps.push({ i, l, r, z: [...z], desc: `Внимание!` });
            z[i] = Math.min(r - i + 1, z[i - l]);
            steps.push({ i, l, r, z: [...z], desc: `Продолжение. Копипаста сработала.` });
        }

        steps.push({ i, l, r, zTarget: z[i], zSource: i });
        while (i + z[i] < n && s[z[i]] === s[i + z[i]])
            steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i] });
        z[i]++;
    }

    if (i + z[i] < n) {
        steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i], desc: `[i]]}' разные. Стоп.` });
    } else {
        steps.push({ i, l, r, z: [...z], desc: `Успех!` });
    }
}

```

```
// Timer loop for autoPlay
useEffect(() => {
  if (autoPlay) {
    timer.current = setInterval(() => {
      setStepIdx(prev => {
        if (prev >= steps.length - 1) { set
        return prev + 1;
      });
    }, 1200);
  }
  return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-b
      <div className="flex bg-slate-900 borde
        <button onClick={() => setMode("kmp
          className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")
          className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
          Z-ФУНКЦИЯ
        </button>
      </div>

    <div className="flex items-center gap-2
```

```

    }
    if (step.zTarget === index
        borderColor = "border-a
        bgColor = step.match ===
-900/40");
    }
}

return {
  <div key={index} className=
    {/* L/R markers for Z */
    {mode === "z" && step.l
      <div className="abs
    }}
    {mode === "z" && step.r
      <div className="abs
    }}

    {/* Index */}
    <span className={`text-

    {/* Char Box */}
    <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
      {char}
    </div>

    {/* Value array box */}
    <div className="text-[1
slate-700/50 font-mono font-bold">
      {mode === "kmp" ? s
    </div>

    {/* KMP fingers */}
    {mode === "kmp" && step
      <div className="abs
z-20">

    <span classNam

```

```

        <div className="absolute top-0 right-0 z-10
        ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_rgba(34,197,94,0.5)]
        })
        </div>
    )
    }
  }
  </div>
</div>

<div className="flex flex-col sm:flex-row gap-2" style={{
  /* Controls */
  <div className="flex bg-slate-900 border border-slate-800 rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
    <button onClick={prevStep} disabled={isFirstStep} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
      <Undo strokeWidth={3} size={16} />
    </button>
    <button onClick={playPause} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
      <button onClick={nextStep} disabled={isLastStep} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
        <SkipForward strokeWidth={3} size={16} />
      </button>
      <button onClick={reset} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
        <RefreshCw strokeWidth={3} size={16} />
      </button>
    </div>

    /* Description Log */
    <div className="flex-1 bg-slate-900/50 border border-slate-800 overflow-hidden" style={{
      <div className="absolute top-0 right-0 z-10 text-white font-mono text-sm" style={{
        <div className="text-[10px] text-slate-400" style={{
          {steps.length}}</div>
        </div>
      </div>
    </div>
  </div>

```

```
        r = i + z[i] - 1;
    }
}

</pre>
</div>
</div>
);
}
```

ФАЙЛ 65/79: `src/components/TopologicalSortViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import {
  Play,
  Pause,
  RotateCcw,
  Clock,
} from "lucide-react";

interface TNode {
  id: number;
  x: number;
  y: number;
  label: string;
  name: string;
}

interface TEdge {
  u: number;
  v: number;
}

const dagNodes: TNode[] = [
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (0" }
```

```

        visited: new Set(visited),
        fullyProcessed: new Set(fullyProcessed),
        currentNode,
        topoList: [...topoList],
        dfsStack: [...dfsStack],
    });
};

recordStep(
    "Начало топологической сортировки. Используем кла
    null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
    visited.add(u);
    dfsStack.push(u);
    recordStep(
        `DFS: зашли в вершину ${u} ("${dagNodes[u].name
        u,
    );

    for (const to of adj[u]) {
        if (!visited.has(to)) {
            dfs(to);
        } else if (!fullyProcessed.has(to)) {
            // Explaining potential cycle detected (not in
            recordStep(
                `⚠ Внимание: Рёбра в серую вершину ${to} о
                u,
            );
        }
    }
}

fullyProcessed.add(u);

```

```

    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1)
      timer = setInterval(() => {
        setCurrentStepIdx((prev) => prev + 1);
      }, 1600);
    } else {
      setIsPlaying(false);
    }
    return () => clearInterval(timer);
  }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getTopoListString = () => {
  return [...step.topoList]
    .reverse()
    .map((id) => dagNodes[id].name)
    .join(" → ");
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Clock className="w-5 h-5 text-indigo-400" />
        Топологическая сортировка (Зависимости обучен
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
          500 text-white"
      >

```

```

        refY="5"
        markerWidth="6"
        markerHeight="6"
        orient="auto-start-reverse"
    >
        <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
    </marker>
</defs>

{/* Edges */}
{dagEdges.map((edge, idx) => {
    const u = dagNodes.find((n) => n.id === e
    const v = dagNodes.find((n) => n.id === e

    const isActive =
        (step.currentNode === edge.u || step.cu
        step.visited.has(edge.v));

    return (
        <line
            key={`edge-${idx}`}
            x1={u.x}
            y1={u.y}
            x2={v.x}
            y2={v.y}
            stroke={isActive ? "#818cf8" : "#334155"}
            strokeWidth={isActive ? "3" : "2"}
            markerEnd={`url(#${isActive ? "dag-ar
            className="transition-all duration-300
        />
    );
}})

{/* Nodes */}
{dagNodes.map((node) => {
    const isCurrent = step.currentNode === no
    const isVisited = step.visited.has(node.i
    const isProcessed = step.fullyProcessed.h

```



```

        fontSize="11px"
        fontWeight="bold"
        className="pointer-events-none"
      >
        ({node.label}) {node.name.split(" "
      </text>
      <text
        x={node.x}
        y={node.y + 12}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="9px"
        className="pointer-events-none"
      >
        {node.name.split(" ").slice(1).join
      </text>
    </g>
  );
  }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5
  00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

  <div className="bg-slate-900 border border-slate-800">
    <p className="text-xs text-indigo-300 min-h-12">
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-auto">
    {/* Topological List output */}
  </div>

```

```

        </span>
        <div className="flex gap-1">
          <button
            disabled={currentStepIdx === 0}
            onClick={() => setCurrentStepIdx((prev)
            className="bg-slate-900 border border-s
t-white"
          >
            Назад
          </button>
          <button
            disabled={currentStepIdx >= steps.length
            onClick={() => setCurrentStepIdx((prev)
            className="bg-slate-900 border border-s
t-white"
          >
            Вперед
          </button>
        </div>
      </div>
    </div>
  </div>
</div>
);
};

```

ФАЙЛ 66/79: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";

import { ArticulationPointsViz } from "../ArticulationPo
import BellmanFordViz from "../BellmanFordViz";
import BfsViz from "../BfsViz";
import { DPVisualizer } from "../DPVisualizer";

```

```

    hint?: string;
    Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки – это ровно то, ч
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    ),
  },
  "heap-beam-search": {
    title: "Куча (priority queue)",
    hint: "Добавляйте гипотезы – куча сама держит лучшую
    Component: HeapViz,
  },
  "segment-trees": {
    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .
    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветится

```

```

        <>
        <ChapterImage vizType="dijkstra" />
        <DijkstraViz />
    </>
    },
},
"bellman-ford": {
    title: "Форд–Беллман",
    Component: () => (
        <>
        <ChapterImage vizType="bellman-ford" />
        <BellmanFordViz />
        </>
    ),
},
floyd: {
    title: "Флойд–Уоршелл",
    Component: () => (
        <>
        <ChapterImage vizType="floyd" />
        <FloydViz />
        </>
    ),
},
"johnson-algo": { title: "Алгоритм Джонсона", Component: () => (
"mst-kruskal": { title: "Краскал и DSU", Component: KruskalSimul
"mst-prima": { title: "Прим", Component: KruskalSimul
"mst-boruvka": { title: "Борувка", Component: KruskalSimul
"string-kmp": { title: "Префикс-функция (КМП)", Component: () => (
"string-z-func": { title: "Z-функция", Component: () => (
"aho-corasick": {
    title: "Ахо–Корасик",
    hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
    Component: () => (
        <div className="space-y-10">
            <WaterfallAnimationWidget />
            <SalmonAutomatonWidget />
        </div>
    ),
},

```

```

// Ветка H -> E -> R -> S
1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, de
2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, d
3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360,
4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460,
// Ветка H -> I -> S
5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, d
6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360,
// Ветка S -> H -> E
7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, de
8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, d
9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360,
    = 2 (HE)
};

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]:
  0: { H: 1, S: 7 },
  1: { E: 2, I: 5 },
  2: { R: 3 },
  3: { S: 4 },
  4: {},
  5: { S: 6 },
  6: {},
  7: { H: 8 },
  8: { E: 9 },
  9: {},
};

```

```

export const WaterfallAnimationWidget: React.FC = () =>
  // Переключатель режимов: 'training' (Полный BFS обхо
  const [activeMode, setActiveMode] = useState<'trainin

  // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
  const [textToScan, setTextToScan] = useState<string>{
  const [currentIndex, setCurrentIndex] = useState<numb
  const [currentNode, setCurrentNode] = useState<number>
  const [isSearchDone, setIsSearchDone] = useState<bool

```

```

let jumpType: 'normal' | 'fail' | null = null;
void jumpType;
let originalCurr = curr;

if (TRIE_TRANSITIONS[curr][char] !== undefined) {
  const nextNode = TRIE_TRANSITIONS[curr][char];
  logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}). `;
  setJumpPath({ from: curr, to: nextNode, type: 'normal' });
  curr = nextNode;
  setActiveSearchCodeLine(3);
} else {
  setActiveSearchCodeLine(2);
  let jumps = 0;
  while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
    curr = WATERFALL_NODES[curr].fail;
    jumps++;
  }
  const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
  if (originalCurr === 0) {
    logMsg += `В корне нет перехода по '${char}'. Логично.`;
    setJumpPath(null);
  } else {
    logMsg += `Поток по '${char}' у вершины #${originalCurr} не имеет  

    рыжок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}). `;
    setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
  }
  curr = nextNode;
}

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
  if (WATERFALL_NODES[temp].is terminal) {
    const word = [...currentMatches].reverse().map(m => m.word).join('');
    matchedWordsForLog.push(word);
    currentMatches = [];
  }
  temp = WATERFALL_NODES[temp].parent;
}

```

```

        просто не существует. fail[H] автоматически нап
codeLine: 5,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'H',
checkingBranchesFrom: null,
},
{
  targetNodeId: 7,
  title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
  desc: 'Обрабатываем дочернюю вершину #7 (S) на De
    ,
codeLine: 5,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'S',
checkingBranchesFrom: null,
},
{
  targetNodeId: 2,
  title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
  desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
    а-разведчик плывет в ROOT и ищет ветку по "E".
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'E',
checkingBranchesFrom: 0,
},
{
  targetNodeId: 5,
  title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
  desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
    H" и "S" не подходят. Ветки нет, fail[HI] = ROOT
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'I',

```

```

        title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
        desc: '  МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
          ба плывет в #1 (H) и ищет ветку "E". У вершины
        codeLine: 4,
        scoutPos: 1,
        inspectedParentFail: 1,
        checkTargetChar: 'E',
        checkingBranchesFrom: 1,
      },
      {
        targetNodeId: 4,
        title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
        desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
          #7 (S). fail[HERS] = #7 (S). Автомат полностью
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
      },
    ],
  ];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNode];

```

```

return (
  <div className="bg-slate-800/95 rounded-2xl p-6 border" >
    { /* Шапка и переключатель режимов */ }
    <div className="flex flex-wrap items-center justify-between" >
      <div className="flex items-center space-x-3" >
        <span className="text-3xl" > </span>
        <div>
          <h4 className="text-2xl font-bold text-white" >
            Анимация водопада: Полное обучение автомата
          </h4>
          <p className="text-sm text-slate-400" >
            Визуализация пошагового построения всех ф

```



```

    <h5 className="text-indigo-300 font-bold">
      <span> </span> Полный обход в ширину (BFS)
    </h5>
    <p className="text-xs text-slate-300 mb-3">
      Показаны все шаги очереди BFS (начиная с
      ar 8 (SHE → HE)</strong>, где наглядно видно
    </p>
    <div className="space-y-1.5 mb-6 overflow-hidden">
      {TRAINING_STEPS_INFO.map((step, idx) =>
        <button
          key={idx}
          onClick={() => setTrainStep(idx)}
          className={`w-full text-left px-3 py-2
            ${
              idx === trainStep
                ? 'bg-indigo-600 text-white font-medium'
                : 'bg-slate-800 text-slate-400'
            }`}
        >
          <span className="line-clamp-1">{step}</span>
          {idx === trainStep && <span className="font-medium">
            </button>
          </div>
        </div>

    <div className="flex gap-2">
      <button
        onClick={() => setTrainStep((prev) => Math.max(
        disabled={trainStep === 0}
        className="flex-1 bg-slate-700 hover:bg-slate-600 text-white font-medium"
      >
        Назад
      </button>
      <button
        onClick={() => setTrainStep((prev) => Math.min(
        disabled={trainStep === TRAINING_STEPS - 1}

```

```

        т, возврат в корень</span>}
      </div>
      <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 fr
      <span>4. if (trie[v][char] !== undefin
/span>
      {currentTrainInfo.codeLine === 4 && <
ан IF!</span>}
    </div>
    <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
      <span>5. else fail[u] = root; // Bero
      {currentTrainInfo.codeLine === 5 && <
root</span>}
    </div>
  </div>
</div>

<div>
  <h5 className="text-slate-200 font-bold ml
    <Lightbulb className="w-4 h-4 text-ambe
  </h5>
  <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
    {currentTrainInfo.desc}
  </div>
</div>
</div>
</div>
) : (
  /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
  <div className="grid grid-cols-1 lg:grid-cols-3
    <div className="bg-slate-900/60 p-5 rounded-x
    <div>
      <h5 className="text-white font-bold mb-2
        <span>» </span> Текст для сканирования
      </h5>
      <p className="text-xs text-slate-400 mb-4

```



```

</h5>
<div className="flex flex-wrap gap-4 text-xs">
  <span className="flex items-center gap-1 text-blue-500">
    <span className="w-3 h-0.5 bg-blue-500 inline-block"></span>
  </span>
  <span className="flex items-center gap-1 text-blue-500">
    <span className="w-3 h-0.5 border-t border-blue-500 inline-block"></span>
  </span>
</div>
</div>

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar">
  <svg viewBox="0 0 800 520" className="w-full h-full">

    {/* Горизонтальные линии и подписи уровней */}
    {[
      { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
      { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
      { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
      { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
      { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' }
    ].map((level) => {
      <g key={`depth-line-${level.depth}`}>
        <line
          x1="20"
          y1={level.y}
          x2="780"
          y2={level.y}
          stroke="#334155"
          strokeWidth="1"
          strokeDasharray="4,4"
          opacity="0.4"
        />
        <rect x="25" y={level.y - 12} width="140" height="24" />
        <text x="32" y={level.y + 4} fill="#94a3b8">
          {level.label}
        </text>
      </g>
    })}
  </svg>
</div>

```

```

        fontWeight="bold"
        textAnchor="middle"
    >
        {char}
    </text>

    {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
    {isTrainingExamined && (
        <g transform={`translate(${(fromN
.y) / 2)}`}>
            <rect x="-15" y="-12" width="30
strokeWidth="2" />
            <text x="0" y="5" fontSize="14"
                {isMatchBranch ? ' ' : ' '}
            </text>
        </g>
    )}
    </g>
    );
    });
    }}}

{/* Рендер fail-ссылок (пунктирные водопадн
Object.values(WATERFALL_NODES).map((node) :
    if (node.id === 0) return null;
    const targetNode = WATERFALL_NODES[node.f

    // В режиме обучения показываем fail-ссыл
    // Найдем индекс шага, на котором обработ
    const stepIdxForNode = TRAINING_STEPS_INF
    if (activeMode === 'training' && trainStep

        let isHighlighted = activeMode === 'search
e === 'fail';
        let isJustEstablished = activeMode === 't
-ссылку

        // Вычисляем изогнутую кривую для красоты

```

```

    {isTargetChild && (
      <circle cx={node.x} cy={node.y} r="
    )}
  )}

```

```

    {/* Основной круг вершины */}
    <circle
      cx={node.x}
      cy={node.y}
      r={isCurrent || isTargetChild ? '24' : '18'}
      fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : '#f8d7da'}
      stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : '#000000'}
      strokeWidth={isCurrent || isTargetChild ? 2 : 1}
      className="transition-all duration-300"
    />
  )}

```

```

    {/* Текст внутри вершины */}
    <text
      x={node.x}
      y={node.y + 5}
      fill={isCurrent || isTargetChild ? 'white' : 'black'}
      fontSize={isCurrent || isTargetChild ? 14 : 12}
      fontWeight="bold"
      textAnchor="middle"
    >
      {node.label}
    </text>
  )}

```

```

    {/* Метка искомой вершины в режиме обхода */}
    {isTargetChild && (
      <g transform={`translate(${node.x}, ${node.y})`}
        <rect x="-35" y="-12" width="70" height="24" fill="#f8d7da" stroke="#000000" strokeWidth="1"/>
        <text x="0" y="2" fontSize="10" fill="white" font-weight="bold">
          ИЩЕМ FAIL
        </text>
      </g>
    )}
  )}

```

```

    {/* Индикатор словарных слов */}
  )}

```

```

        </svg>
      </div>
    </div>

    { /* Список найденных совпадений (только в режиме поиска)
    {activeMode === 'search' && {
      <div className="mt-6 bg-slate-900/50 p-5 rounded" >
        <h5 className="text-white font-bold mb-3 text" >
          <span> </span> Улов лосося (Найденные слова)
        </h5>
        {foundMatches.length === 0 ? {
          <p className="text-sm text-slate-400 italic" >
            Пока ничего не поймали. Запустите шаги симуляции
          </p>
        } : {
          <div className="flex flex-wrap gap-2">
            {foundMatches.map((match, idx) => {
              <div
                key={idx}
                className="bg-emerald-950 border border-emerald-500
                text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]" >
                <CheckCircle2 className="w-4 h-4 text-emerald-500" />
                <span>{match.word}</span>
                <span className="text-[10px] bg-emerald-950/50 px-2" >
                  Индекс {match.index}
                </span>
              </div>
            })}
          </div>
        )}
      </div>
    )}
  </div>
) }
</div>
};

```

```

    <div className="flex gap-2 mb-4 overflow-x-auto" style={{
      [
        ["1d", "1. Одномерные суммы"],
        ["2d", "2. Двумерные суммы"],
      ] as const
    }).map(({key, label}) => {
      <button
        key={key}
        onClick={() => setMode(key)}
        className={`px-3 py-2 rounded-lg text-xs sm
          mode === key ? "bg-indigo-600 text-white"
        }`
      >
        {label}
      </button>
    })}
  </div>
  {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
</div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
  const pref = useMemo(
    () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]],
    []
  );
};

```

```

/** Что сейчас под курсором: префикс P[i] или элемент a[i]
const [hover, setHover] = useState<{ kind: "pref" | "a"; i: number }>({ kind: "a", i: 0 });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });

```

```

const covered =
  hover?.kind === "pref"
    ? { from: 0, to: hover.i - 1 }
    : hover?.kind === "a"
      ? { from: hover.i, to: hover.i }
      : { from: 0, to: 0 };

```



```

        >
        {v}
      </div>
    );
  }}}
</div>

{/* массив P */}
<div className="flex gap-1.5 items-center">
  <div className="w-[46px] shrink-0 text-right"
    {pref.map((v, i) => {
      const active = hover?.kind === "pref" && i === range.l;
      const isL = i === range.l;
      const isR = i === range.r + 1;
      return (
        <div
          key={i}
          onMouseEnter={() => setHover({ kind: "pref" })}
          onMouseLeave={() => setHover(null)}
          className={` ${cellBase} cursor-help $
            {active
              ? "bg-emerald-500 text-white ring-1 ring-emerald-500"
              : isR
                ? "bg-emerald-700 text-white"
                : isL
                  ? "bg-rose-700 text-white"
                  : "bg-slate-900 text-slate-300"}
            }`
          title={`P[${i}] = сумма a[0..${i} - 1]`}
        >
          {v}
        </div>
      );
    })}
  </div>
</div>
</div>

```

```

    <div className="text-xs text-slate-400 mb-2">
      Запрос суммы на отрезке (кликайте по массиву
    </div>
    <p className="font-mono text-sm sm:text-base te
      sum(a[{range.l}..{range.r}]) = P[{range.r + 1
    <span className="text-emerald-400">{pref[range.l
    <span className="text-rose-400">{pref[range.l
    <span className="text-indigo-300 font-bold">{
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
      Один вычитание вместо цикла: запрос за O(1) п
    </p>
  </div>
</div>
  );
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
  const n = A2.length;
  const m = A2[0].length;

  const P = useMemo(() => {
    const p = Array.from({ length: n + 1 }, () => Array
    for (let i = 1; i <= n; i++) {
      for (let j = 1; j <= m; j++) {
        p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i
      }
    }
    return p;
  }, [n, m]);

  const [hover, setHover] = useState<{ i: number; j: num
  const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1

  const D = P[rect.r1][rect.c1];
  const B = P[rect.r1][rect.c2 + 1];
  const Cc = P[rect.r2 + 1][rect.c1];

```



```
import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
  Play,
  RotateCcw,
  ChevronRight,
  ChevronLeft,
  Lock,
  ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);
  }
}

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],
  [39, 81, 62, 28, 51, 42, 85, 14]
```

```
const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query">
```

```
return (
```

```
  <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
```

```
    <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
```

```
      <button
```

```
        onClick={() => setTab("1d")}>
```

```
        className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400`}
```

```
        type="button" />
```

```
    </div>
```

```
      1. Сворачивание 1D
```

```
    </button>
```

```
    <button
```

```
      onClick={() => setTab("2d_build")}>
```

```
      className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400`}
```

```
      type="button" />
```

```
    </div>
```

```
      2. Сворачивание 2D (Построение)
```

```
    </button>
```

```
    <button
```

```
      onClick={() => setTab("2d_query")}>
```

```
      className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400`}
```

```
      type="button" />
```

```
    </div>
```

```
      3. Запрос 2D (Формула)
```

```
    </button>
```

```
  </div>
```

```
<AnimatePresence mode="wait">
```

```
  {tab === "1d" ? (
```

```
    <Viz1D key="1d" />
```

```
  ) : tab === "2d_build" ? (
```

```
    <Viz2DBuild key="2d_build" />
```

```
  ) : (
```

```
    <Viz2DQuery key="2d_query" />
```

```
  )}
```

```
</AnimatePresence>
```

```
</div>
```

```

        onClick={() => setStep(Math.min(maxStep, st
        className="p-2 bg-indigo-600 hover:bg-indigo
        disabled={step === maxStep}
      >
        <ChevronRight size={20} />
      </button>
      <button
        onClick={() => setStep(0)}
        className="p-2 bg-slate-800 hover:bg-slate-
      >
        <RotateCcw size={20} />
      </button>
    </div>
  </div>

<div className="bg-slate-900 rounded-xl border bo
  <div className="text-[11px] uppercase tracking-
  <div className="flex gap-1 sm:gap-1.5 overflow-
    {arr1D.map((v, idx) => {
      const inCover = !!covered && idx >= covered
      return (
        <div key={idx} className="flex flex-col i
          <div
            className={`flex items-center justify
            w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
              inCover ? "bg-indigo-500 text-white
            }`}
          >
            {v}
          </div>
          <span className="text-[9px] text-slate-
        </div>
      );
    })}
  </div>
  <div className="mt-2 text-xs min-h-[36px]">
    {hover && covered ? (
      <p className="text-slate-300">

```

```

</div>
<div className="text-center font-bold text-rap">
  2^3 (L=8)
</div>

```

```

{/* Matrix */}
{Array.from({ length: N }).map((_, i) => {
  <React.Fragment key={i}>
    <div className="text-slate-500 font-monospace">
      {i}
    </div>
    {Array.from({ length: LOG }).map((_, j) => {
      const isValid = i + (1 << j) <= N;
      const isVisible =
        isValid &&
        (j === 0 ||
          computeSteps.findIndex((s) => s.i === i + (1 << j) && s.j === j) > -1);

      // Active computing state
      let isActive = false;
      let isSrc1 = false;
      let isSrc2 = false;

      if (step > 0 && step <= maxStep) {
        const currentStep = computeSteps[step];
        if (currentStep.i === i && currentStep.j === j) {
          isActive = true;
        }
        if (currentStep.j === j + 1 && currentStep.i === i) {
          isSrc1 = true;
        }
        if (
          currentStep.j === j + 1 &&
          currentStep.i === i - (1 << (currentStep.j - j))
        ) {
          isSrc2 = true; // wait, no. Src2 is not a source
        }
        // Let's re-eval exact source:
        // We are asking if THIS cell [i][j] is a source
      }
    })}
  </div>

```

```

    })

    {/* Arrows visualization */}
    {(isSrc1 || isSrc2) && (
      <motion.svg
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        className="absolute pointer-events-none"
        style={{
          width: 100,
          height: isSrc2 ? 40 + (1 <<
          right: -50,
          top: 24,
          overflow: "visible",
        }}
      >
        <path
          d={`M 0 0 C 30 0, 30 ${isSrc1 ? "0 0" : "100 100"}
          fill="none"
          stroke={isSrc1 ? "#10b981" : "#f87171"}
          strokeWidth="3"
        />
      </motion.svg>
    )}
  </div>
);
}}
</React.Fragment>
)}}
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-800 p-4" >
  {step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];

```



```

                                highlightClass = 'bg-blue-500';
          ] md:scale-[1.08]';
                                } else if (!isTop && isLeft) {
                                    highlightClass = 'bg-emerald-500';
          9,0.3)] md:scale-[1.08]';
                                } else {
                                    highlightClass = 'bg-amber-500';
          3)] md:scale-[1.08]';
                                }
                                zIndex = 'z-10';
          }
        }

        return (
          <div
            key={idx}
            onMouseEnter={() => { if(isCurr) setCurr(r, c); }}
            onMouseLeave={() => { if(isCurr) setCurr(r, c); }}
            onClick={() => {
              if(isCurr) {
                if(locked && locked.r === r && locked.c === c) {
                  setLocked({r, c});
                }
              }
            }}
            className={`flex items-center justify-center gap-2`}
            style={{
              width: sCurr ? 'w-[clamp(24px,6.5vw,38px)]' : 'w-[clamp(20px,5.2vw,30px)]',
              height: hCurr ? 'h-[clamp(24px,6.5vw,38px)]' : 'h-[clamp(20px,5.2vw,30px)]',
              text: [clamp(8px,2vw,12px)],
            }}
          >
            {ST2D[r][c][step][step]}
          </div>
        );
      }
    }
  </div>
</div>
</React.Fragment>
)
)))

```

```

        {'}'}));
    </div>

    {activeCell ? (
      <div className="bg-slate-950 p-5 rounded
.2)] mt-2 transition-all">
        <p className="text-slate-300 mb-3 tr
        <span>Пример для ячейки:</span>
        {locked && <span className="text
ded border border-rose-500/20"><Lock size={
        </p>
        <p className="text-indigo-300 borde
        <span className="font-bold text-w
        }}[k]][k]]</span>
        <span className="text-slate-400">
        </p>

        <div className="space-y-2 pt-3 pl-2
        {(() => {
          const prevL = 1 << (k - 1);
          return (
            <>
              <div className="flex ite
                <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
                <span className="text
tiveCell.r][activeCell.c][k-1][k-1]}</span>
              </div>
              <div className="flex ite
                <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{activeC
                <span className="text
tiveCell.r][activeCell.c + prevL][k-1][k-1]
              </div>
              <div className="flex ite
                <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
                <span className="text

```

```

const [c2, setC2] = useState(6);

const H = r2 - r1 + 1;
const W = c2 - c1 + 1;
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
  <div className="flex flex-col xl:flex-row gap-6"
    <div className="flex-1 min-w-0 bg-slate-900"
      <h3 className="text-xl font-bold text-rose-500">
        <p className="text-slate-400 text-[13px] font-normal">
          еpx, r2, c2 - правый низ).</p>

      <div className="flex flex-col items-center"
        <div className="grid grid-cols-[repeat(8, 1fr)] gap-2"
          <div className="border border-rose-500/50 border-scale-105 z-10"
            <div key={idx} className={`text-[clamp(9px,2.4vw,12px)] font-normal
              y-center rounded text-[clamp(9px,2.4vw,12px)] font-normal
              0 border-rose-500/50 border scale-105 z-10`}>
              {val2D[r][c]}
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
)

```

```

        <span className="text-slate-500">{Lx}</span>
        <div className="flex gap-2">
          <input type="number" min={
14 bg-slate-900 border border-slate-700 py-1 px-2 rounded
          <input type="number" min={
14 bg-slate-900 border border-slate-700 py-1 px-2 rounded
        </div>
      </label>
    </div>
  </div>
</div>

<div className="flex-1 bg-slate-900 p-6 rounded" style={{
  <h3 className="text-xl font-bold text-emerald-400">Настройка
  <p className="text-sm font-sans text-slate-300">
    Мы знаем, что максимальная степень дв
    rounded text-white">{Lx}</span>. Аналогично
    te">{Ly}</span>.
  </p>
  <p className="text-sm font-sans text-slate-300">
    Чтобы покрыть весь прямоугольник, мы
    Name="font-mono bg-[#0a0f1e] px-1 rounded b
    роса. (Цветные прямоугольники слева).
  </p>

  <div className="bg-[#0a0f1e] rounded-xl p-4" style={{
    text-slate-300 shadow-inner overflow-x-auto
    <div className="absolute right-3 top-3" style={{
      <span className="text-slate-500">// 1.
      <span className="text-purple-400">int<
    ext-amber-300">1</span>}; <span className="
      <span className="text-purple-400">int<
    ext-amber-300">1</span>}; <span className="
      <span className="text-purple-400">int<
    "text-slate-500">// Выс: {Lx}</span><br/>
      <span className="text-purple-400">int<
    "text-slate-500">// Шир: {Ly}</span><br/><b

```

```

        let isBR = r === r2 - Lx + 1

        let color = "bg-slate-800 text-white"
        let txt = ST2D[r][c][kx][ky].

        let badge = null;
        if (isTL) { color = "bg-rose-400 text-slate-800 border-2"; badge = "TL"; }
        else if (isTR) { color = "bg-blue-300 border-2"; badge = "TR"; }
        else if (isBL) { color = "bg-emerald-300 border-2"; badge = "BL"; }
        else if (isBR) { color = "bg-amber-300 border-2"; badge = "BR"; }

        return (
            <div key={idx} className={color} duration-300 relative shrink-0 ${cx}
                {txt}
                {badge && <div className="bg-slate-700 text-white px-1 font-bold rounded"
                    </div>
                }
            )
        )
    }
}
</div>
</div>
</div>

<div className="mt-auto bg-slate-950 font-sans text-white" >
    <div className="text-center font-bold text-white" wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border-1
        <span className="text-slate-400 font-sans text-sm" >
            <span className="text-rose-400 font-sans text-sm" >
                <span className="text-blue-400 font-sans text-sm" >
                    <span className="text-emerald-400 font-sans text-sm" >

```

```

const headerMain = document.querySelector('h1');
const questionsContainer = document.getElementById('questions');
const wtfContainer = document.getElementById('wtf');

if (mode === 'wtf') {
    document.body.classList.add('wtf-mode');
    if (aside) aside.style.display = 'none';
    if (headerMain) headerMain.style.display = 'none';
    if (questionsContainer) questionsContainer.style.display = 'none';
    if (wtfContainer) wtfContainer.classList.remove('hidden');

    // Remove the URL hash to avoid scrolling
    history.pushState("", document.title, window.location.pathname);
    window.scrollTo(0, 0);
} else {
    document.body.classList.remove('wtf-mode');
    if (aside) aside.style.display = '';
    if (headerMain) headerMain.style.display = '';
    if (questionsContainer) questionsContainer.style.display = '';
    if (wtfContainer) wtfContainer.classList.add('hidden');
}

function setTheme(theme) {
    const body = document.body;
    body.classList.remove('theme-dark', 'theme-light', 'theme-terminal');
    body.classList.add('theme-' + theme);

    // Just some visual feedback on buttons
    document.querySelectorAll('[id^="theme-"]').forEach(btn => {
        btn.classList.remove('outline', 'outline-1');
    });

    if (theme === 'dark') document.getElementById('dark-button').classList.add('active');
    if (theme === 'light') document.getElementById('light-button').classList.add('active');
    if (theme === 'terminal') document.getElementById('terminal-button').classList.add('active');
}

```

```

        }
    }

    if (mobileMenuBtn && mobileDrawer) {
        mobileMenuBtn.addEventListener("click",
    }
    if (closeDrawerBtn && mobileDrawer) {
        closeDrawerBtn.addEventListener("click"
    }
    if (drawerOverlay) {
        drawerOverlay.addEventListener("click",
    }

    // Close on link click
    document.querySelectorAll("#mobile-drawer a
        link.addEventListener("click", () => {
            if (mobileDrawer && !mobileDrawer.c
                toggleDrawer();
        }
    }));
});

const observerOptions = {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: 0
};

const observer = new IntersectionObserver((
    entries.forEach(entry => {
        if (entry.isIntersecting && !document
            // Highlight active TOC link lo
            const id = entry.target.id;
            document.querySelectorAll('aside
                if (link.getAttribute('href
                    link.classList.add('bg-
                } else {
                    link.classList.remove('

```



```
@import url('https://fonts.googleapis.com/css2?
@reference "./src/index.css";
.math-font { font-family: 'Fira Code', monospace; }
.uno-card {
    width: 70px; height: 105px; border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
    align-items: center; justify-content: center; background-color: white;
    overflow: hidden; flex-shrink: 0; text-shadow: 2px 2px 0 #000, -1px -1px 0 #000;
    font-family: 'Arial Black', Impact, sans-serif; font-size: 1.2em;
}
.uno-card::before, .uno-card::after {
    content: attr(data-val); position: absolute; top: 50%; left: 50%;
    text-shadow: 1px 1px 0 #000, -1px -1px 0 #000; width: 100%; height: 100%;
}
.uno-card::before { top: 2px; left: 4px; }
.uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
.uno-card.mini { width: 45px !important; height: 60px !important; }
.uno-card.mini::before, .uno-card.mini::after {
    font-size: 0.8em;
}

/* Global formula scroll settings */
.formula-scroll {
    overflow-x: auto;
    overflow-y: hidden;
    -webkit-overflow-scrolling: touch;
    scrollbar-width: none; /* Firefox */
    -ms-overflow-style: none; /* IE and Edge */
}
.formula-scroll::-webkit-scrollbar {
    display: none;
}

mjax-container {
    max-width: 100% !important;
}
mjax-container[display="true"] {
    overflow-x: auto;
    overflow-y: hidden;
}
```

```

    .card-black { background-color: #222; } .card-b
    .card-black::before, .card-black::after { text-
    .info-block { @apply bg-slate-800/80 border-l-4
    .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
    ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
    king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
    r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 for

/* THEME OVERRIDES */
body.theme-light {
    background-color: #f8fafc !important;
    color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-light
    important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

body.theme-notebook {
    background-color: #fcf8eb !important; /* li
    color: #3b2818 !important; /* brown ink */
    background-image: repeating-linear-gradient
    background-attachment: local !important;
    font-family: "Comic Sans MS", "Caveat", "Se
}
body.theme-notebook .bg-slate-900, body.theme-n
    late-700 {

```

```

<div class="sticky top-0 z-50 w-full bg-slate-950/90
  center items-center gap-4 sm:gap-8 transition-colors">
  <div class="flex items-center gap-3">
    <span class="text-sm font-bold uppercase tracking-wider">Список тем</span>
    <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
      <button onclick="setMode('normal')" id="btn-normal"
        transition-colors">Список тем</button>
      <button onclick="setMode('wtf')" id="btn-wtf"
        transition-colors">WTF</button>
    </div>
  </div>
  <div class="flex items-center gap-3">
    <span class="text-sm font-bold uppercase tracking-wider">Темы</span>
    <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
      <button onclick="setTheme('dark')" id="btn-dark"
        transition-colors">Темная</button>
      <button onclick="setTheme('light')" id="btn-light"
        transition-colors">Светлая</button>
      <button onclick="setTheme('notebook')" id="btn-notebook"
        transition-colors">Notebook</button>
    </div>
  </div>
</div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">
  <defs>
    <marker id="arrow-yellow" markerWidth="6" markerHeight="6"
      <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-orange" markerWidth="6" markerHeight="6"
      <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-pink" markerWidth="6" markerHeight="6"
      <path d="M0,0 L6,3 L0,6 Z" fill="#f472b2" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-cyan" markerWidth="6" markerHeight="6"
      <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf" stroke="black" stroke-width="1" />
    </marker>
  </defs>
</svg>

```

```

        <path d="M 5 45 L -5 45 L -5 55 L 5 55"
        <circle cx="25" cy="50" r="15" fill="#f
    </g>
    <g id="lego-green">
        <rect x="0" y="10" width="20" height="15"
        <rect x="3" y="5" width="14" height="5"
    </g>
    <g id="lego-white">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="0" y1="2" x2="40" y2="2" stro
        <text x="20" y="14" fill="#475569" font
    </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
    0/50 z-50 hover:bg-blue-500 transition-transform
    <svg xmlns="http://www.w3.org/2000/svg" class="
        <path stroke-linecap="round" stroke-linejoin
    </svg>
</button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed ins

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-

    <!-- Левое меню (Оглавление) -->
    <aside id="mobile-drawer" class="fixed inset-y-0
        nslate-x-full transition-transform duration-300
        w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
        <div class="h-full flex flex-col p-6 overflow
            lg:rounded-2xl lg:border lg:border-slate-700
                <div class="flex justify-between items-
                    <h3 class="font-black text-transpar

```

```
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-teal-500 pl-3 font-bold text-teal-300">
    <a href="#question-3" class="text-teal-500 pl-3 font-bold text-teal-300">
      3. Тригонометрическая форма
    </a>
  </summary>
  <div class="pl-6 space-y-2 text-teal-500">
    <a href="#section-3-1" class="block text-teal-500">
    <a href="#section-3-2" class="block text-teal-500">
    <a href="#section-3-3" class="block text-teal-500">
    <a href="#section-3-4" class="block text-teal-500">
  </div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">
    <a href="#question-4" class="text-amber-500 pl-3 font-bold text-amber-300">
      4. Степени и корни
    </a>
  </summary>
  <div class="pl-6 space-y-2 text-amber-500">
    <a href="#q4-def" class="block text-amber-500">
    <a href="#q4-moivre" class="block text-amber-500">
    <a href="#q4-root" class="block text-amber-500">
    <a href="#q4-count" class="block text-amber-500">
  </div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-300">
    <a href="#question-5" class="text-fuchsia-500 pl-3 font-bold text-fuchsia-300">
      5. Группа корней из единицы
    </a>
```

```

        <a href="#q7-3" class="block">
        <a href="#q7-4" class="block">
        <a href="#q7-5" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-violet-500 pl-3 font-bold text-violet-300">
        8. Сравнимость и Поля В
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-violet-300">
        <a href="#q8-1" class="block">
        <a href="#q8-2" class="block">
        <a href="#q8-3" class="block">
        <a href="#q8-4" class="block">
        <a href="#q8-5" class="block">
        <a href="#q8-6" class="block">
        <a href="#q8-7" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-green-400">
        9. Кольцо многочленов
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-emerald-400">
        <a href="#q9-1" class="block">
        <a href="#q9-2" class="block">
        <a href="#q9-3" class="block">
        <a href="#q9-4" class="block">
    </div>
</details>
```

```

        <a href="#question-12" class=
der-blue-500 pl-3 font-bold text-blue-400">
            12. Взаимно простые мно
        </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q12-1" class="blo
        <a href="#q12-2" class="blo
        <a href="#q12-3" class="blo
еление)</a>
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-13" clas
der-emerald-500 pl-3 font-bold text-emerald
            13. Корни многочленов.
        </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q13-1" class="blo
        <a href="#q13-2" class="blo
        <a href="#q13-3" class="blo
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-14" clas
der-fuchsia-500 pl-3 font-bold text-fuchsia
            14. Кратность корня. Ко
        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2
        <a href="#q14-1" class="blo
        <a href="#q14-2" class="blo
        <a href="#q14-3" class="blo
```

```

        <span class="font-bold text-teal-500">
      </a>
      <a href="#question-18" class="block text-slate-400">
        <span class="font-bold text-blue-500">
      </a>
      <a href="#question-19" class="block text-slate-400">
        <span class="font-bold text-emerald-500">
      </a>
      <a href="#question-20" class="block text-slate-400">
        <span class="font-bold text-rose-500">
      </a>
      <a href="#question-21" class="block text-slate-400">
        <span class="font-bold text-indigo-500">
      </a>
      <a href="#question-22" class="block text-slate-400">
        <span class="font-bold text-fuchsia-500">
      </a>

      <a href="#proof-algorithms" class="block text-red-500 font-bold mt-4">
        ▲ Ликбез: Алгоритмы выживания
      </a>

    </nav>
    <script>
      document.querySelectorAll('details').forEach(detail => {
        detail.addEventListener('toggle', () => {
          if (detail.open) {
            document.querySelectorAll('details').forEach(other => {
              if (other !== detail) {
                other.removeAttribute('open')
              }
            })
          }
        })
      })
    </script>
  </body>
</html>

```



```

        if (details &&
            details.open
            document.querySelector(
                if (other
                other
            }
        }));
    }
}
});
}, {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: [0, 0.5]
});

sections.forEach(section => {
    observer.observe(section);
});
});

function fallbackModal(contentRaw,
    const modal = document.createElement('div');
    modal.style.position = 'fixed';
    modal.style.top = '0';
    modal.style.left = '0';
    modal.style.width = '100vw';
    modal.style.height = '100vh';
    modal.style.backgroundColor = 'black';
    modal.style.zIndex = '99999';
    modal.style.display = 'flex';
    modal.style.flexDirection = 'column';
    modal.style.alignItems = 'center';
    modal.style.justifyContent = 'center';

    const box = document.createElement('div');
    box.style.width = '80%';

```

```

copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
    if (navigator.clipboard) {
        navigator.clipboard.writeText(textarea.value);
        copyBtn.innerText = 'Скопировано';
    } else {
        textarea.select();
        document.execCommand('copy');
        copyBtn.innerText = 'Скопировано';
    }
    setTimeout(() => copyBtn.innerText = 'Копировать', 1000);
};

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Заккрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.remove();

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
    box.appendChild(textarea);
    btnContainer.appendChild(copyBtn);
}
btnContainer.appendChild(closeBtn);
box.appendChild(btnContainer);
modal.appendChild(box);
document.body.appendChild(modal);
}

function downloadMD() {
    let clone = document.querySelector('div[id="code"]');
    let code = clone.innerHTML;
    let blob = new Blob([code], { type: 'text/plain' });
    let url = URL.createObjectURL(blob);
    let a = document.createElement('a');
    a.href = url;
    a.download = 'code.md';
    a.click();
    URL.revokeObjectURL(url);
}

```

```
);
```

```
document.getElementById('theme-  
document.getElementById('theme-  
document.getElementById('theme-
```

```
let oldStyle = document.getElem  
if (oldStyle) oldStyle.remove()  
const style = document.createEl  
style.id = 'theme-style';
```

```
if (theme === 'dark') {  
    body.classList.add('bg-slate  
    document.getElementById('th  
} else if (theme === 'light') {  
    body.classList.add('bg-white  
    document.getElementById('th  
    style.innerHTML = `
```

```
body.bg-white section h1  
body.bg-white main { col  
body.bg-white main p {  
/* Universal text color  
body.bg-white .text-sla  
body.bg-white .text-sla  
body.bg-white .text-sla  
body.bg-white [class*="
```

```
/* Specific backgrounds  
body.bg-white aside > d  
body.bg-white [class*="
```

```
nt; }
```

```
body.bg-white [class*="
```

```
nt; }
```

```
body.bg-white [class*=""  
body.bg-white .analogy-l  
body.bg-white .img-plac  
body.bg-white .info-blo
```

```

        style.innerHTML = `
            body.bg-black .bg-slate-950 {
                background-color: t
            }
            body.bg-black section h1,
            , body.bg-black button {
                color: #4ade80 !imp
                border-color: #22c55e !imp
                font-family: monosp
            }
            body.bg-black svg path,
                stroke: #22c55e !imp
                fill: transparent !
            }
        `;
    }

    document.head.appendChild(style)
}
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И
<div class="mt-8 flex flex-col gap-4 bo
    <!-- Кнопки Конвертации / Экспорта
    <div class="flex flex-col gap-2">
        <div class="text-xs text-slate-
        <button onclick="downloadWord()
ont-bold flex items-center justify-center g
            <span> </span> Word
        </button>
        <a href="algebra.pdf" target="_l
-sm font-bold flex items-center justify-cen
            <span> </span> PDF
        </a>
        <button onclick="downloadMD()"

```

```

        <!-- Print Table of Contents -->
        <div class="hidden print:block mt-8 text-align: center; font-size: 1.2em; font-weight: bold;">
            <h2 class="font-bold text-xl mb-4">Table of Contents
            <ul class="list-disc list-inside text-base space-y-2">
                <li><a href="#question-1">1. Алгебра
                <li><a href="#question-2">2. Геометрия
                <li><a href="#question-3">3. Тригонометрия
                <li><a href="#question-4">4. Статистика
                <li><a href="#question-5">5. Графики
                <li><a href="#question-6">6. Дифференциальное исчисление
                <li><a href="#question-7">7. Алгебра
                <li><a href="#question-8">8. Калculus
                <li><a href="#question-9">9. Координаты
                <li><a href="#question-10">10. Матрицы
                <li><a href="#question-11">11. Векторы
                <li><a href="#question-12">12. Комплексные числа
                <li><a href="#question-13">13. Логика
                <li><a href="#question-14">14. Теория вероятностей
                <li><a href="#question-15">15. Статистика
                <li><a href="#question-17">17. Матрицы
                <li><a href="#question-18">18. Векторы
                <li><a href="#question-19">19. Комплексные числа
                <li><a href="#question-20">20. Логика
                <li><a href="#question-21">21. Теория вероятностей
                <li><a href="#question-22">22. Статистика
            </ul>
        </div>
    </header>

    <!-- ===== WTF CONTAINER =====>
    <div id="wtf-container" class="hidden">
        <h2 class="text-3xl font-black text-center">WTF CONTAINER

        <div class="overflow-x-auto w-full pb-4">
            <div class="grid grid-cols-3 gap-4">
                <!-- Column headers -->

```

```

        <div class="font-bold text-slate-500">
    </a>

    <!-- Row 3: НОД и НОК / Алгоритм Евклида -->
    <a href="#question-7" onclick="setMainQuestion(7)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-yellow-500">НОД и НОК / Алгоритм Евклида
        <div class="font-bold text-slate-500">НОД и НОК / Алгоритм Евклида
    </a>
    <a href="#question-12" onclick="setMainQuestion(12)">
-1-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">НОД и НОК / Алгоритм Евклида
        <div class="font-bold text-slate-500">НОД и НОК / Алгоритм Евклида
    </a>
    <!-- Empty 3rd row cell -->
    <div class="hidden md:block"></div>

    <!-- Row 4: Факторизация / Структурная теория -->
    <a href="#question-8" onclick="setMainQuestion(8)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-yellow-500">Факторизация / Структурная теория
        <div class="font-bold text-slate-500">Факторизация / Структурная теория
    </a>
    <a href="#question-11" onclick="setMainQuestion(11)">
-1-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">Факторизация / Структурная теория
        <div class="font-bold text-slate-500">Факторизация / Структурная теория
    </a>
    <a href="#question-5" onclick="setMainQuestion(5)">
-4 border-cyan-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-cyan-500">Факторизация / Структурная теория
        <div class="font-bold text-slate-500">Факторизация / Структурная теория
    </a>

    <!-- Row 5: Корни / Уравнения -->
    <div class="hidden md:block"></div>
    <a href="#question-13" onclick="setMainQuestion(13)">
-1-4 border-orange-500 hover:bg-slate-700 transition-colors

```

```

        <div class="text-xs text-blue-500">
        <div class="font-bold text-slate-700">
    </a>
    <a href="#question-19" onclick="setQuestion(19)">
- l-4 border-emerald-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-emerald-500">
        <div class="font-bold text-slate-700">
    </a>

    <a href="#question-20" onclick="setQuestion(20)">
- l-4 border-rose-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-rose-500">
        <div class="font-bold text-slate-700">
    </a>

    <a href="#question-21" onclick="setQuestion(21)">
- l-4 border-indigo-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-indigo-500">
        <div class="font-bold text-slate-700">
    </a>

    <a href="#question-22" onclick="setQuestion(22)">
- l-4 border-fuchsia-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-fuchsia-500">
        <div class="font-bold text-slate-700">
    </a>

    <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ -->
    <div class="col-span-1 md:col-span-3 lg:col-span-4"
border-b border-red-500/30 pb-2">Кластер 4:

        <a href="#proof-algorithms" onclick="setSection('proof-algorithms')">
:col-span-3 bg-red-900/20 p-4 border-l-4 border-r-4 border-t-4 border-b-4
        <div class="text-xs text-red-500">
        <div class="font-bold text-slate-700">
    </a>
</div>
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->

```

```

        <ul class="list-disc pl-5 sp-1">
            <li><span class="text-white">
                i>
                <li><span class="text-white">
                    т.д.).</li>
                    <li><span class="text-white">
                        что так и задумано: "Ой, противоречие, теорема
                    </li>
        </ul>
    </div>

```

```

<!-- Единственность -->
<div class="bg-slate-800/80 p-6">
    <h3 class="text-xl text-emerald-400">
    <p class="text-sm text-slate-400">
    <ul class="list-disc pl-5 sp-1">
        <li><span class="text-white">
            ,  $r_2$ ).</li>
            <li><span class="text-white">
            <li><span class="text-white">
                счет ограничений (размер остатка) докажи, что
            </li>
    </ul>
</div>

```

```

<!-- Индукция -->
<div class="bg-slate-800/80 p-6">
    <h3 class="text-xl text-yellow-400">
    <p class="text-sm text-slate-400">
    <ul class="list-disc pl-5 sp-1">
        <li><span class="text-white">
            валось.</li>
            <li><span class="text-white">
                Просто перепиши формулу с буквой  $k$ .</li>
            <li><span class="text-white">
                усок  $k$ , замени его на жареного карася из
            </li>
    </ul>
</div>

```

```

<!-- Двойная делимость -->

```


зложить.</p>

</div>

<div class="bg-slate-800 p-

<h4 class="text-red-400

<p class="text-sm text-

<p class="text-xs font-

. Чаще всего после этого Фейс-контроль \$p\$.

</div>

</div>

<h2 class="text-2xl font-bold b

(Правила экстрасенса)</h2>

<p class="text-sm text-slate-40

тношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>

<ul class="space-y-4 pb-10">

<li class="bg-slate-800/50 p

<div>

<h4 class="text-cyan

<p class="text-sm t

<code class="text-x

</div>

<li class="bg-slate-800/50 p

<div>

<h4 class="text-cyan

<p class="text-sm t

х двух, потом последних.

<code class="text-x

</div>


```
-----  
import { clsx, type ClassValue } from "clsx";  
import { twMerge } from "tailwind-merge";  
  
export function cn(...inputs: ClassValue[]) {  
  return twMerge(clsx(inputs));  
}
```

=====

РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

=====

ФАЙЛ 73/79: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```
/**  
 * Аудит покрытия глав пособия: у каких тем нет «объясн  
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн  
 *  
 *   node scripts/audit-coverage.mjs           # отчёт  
 *   node scripts/audit-coverage.mjs --md      # markd  
 */  
import fs from "node:fs";  
import path from "node:path";  
import { fileURLToPath } from "node:url";  
import { build } from "esbuild";  
  
const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "src");  
const TMP = path.join(ROOT, "tmp", "audit");  
  
fs.mkdirSync(TMP, { recursive: true });  
const bundle = path.join(TMP, "content.bundle.mjs");  
  
await build({  
  entryPoints: [path.join(ROOT, "src/data/content.ts")],  
  bundle: true,
```

```

-----
const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
  console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
  console.log("| --- | --- | --- | --- | --- |");
  for (const r of rows) {
    console.log(
      `| ${r.num} || "-" | ${r.title} | ${r.hasAnalogy}
      ge)) |`
    );
  }
} else {
  console.log(`Глав в пособии: ${rows.length}\n`);
  const dump = (title, list) => {
    console.log(`${title} (${list.length}):`);
    list.forEach((r) => console.log(`  * ${r.title}  [$
    console.log());
  };
  dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
  dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
  dump("НЕТ 2D (аналогия есть)", gapViz);
  console.log(`Пропущенные номера билетов 1-24: ${missi
  console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

ФАЙЛ 74/79: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

-----
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";

```

```

    const surface = m[1].toLowerCase();
    const id = map.get(surface);
    if (!id) continue;
    const ut = perTerm.get(id) ?? 0;
    const us = perSurface.get(surface) ?? 0;
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }

  if (ids.size === 0) bad.push(c.title);
  console.log(
    String(ids.size).padStart(3), "|", String(highlights).padStart(3),
    c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 4),
  );
}
console.log("\nГлав без единой подсказки:", bad.length, "шт.");
console.log("Терминов в глоссарии:", GLOSSARY.length, "шт.");
console.log("Не встретились:", GLOSSARY.filter((g) => !bad.includes(g)).length, "шт.");

```

ФАЙЛ 75/79: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

```

/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";

```

```

    doc.end();
    await new Promise((resolve, reject) => {
      stream.on("finish", resolve);
      stream.on("error", reject);
    });

    console.log("[3/3] png -> pdf");
    console.log(`      страниц: ${pages.length}`);
    console.log(`      выход:   ${path.relative(ROOT, PDF)}`);
  }

  main().catch((err) => {
    console.error("Ошибка сборки PDF:", err.message);
    process.exit(1);
  });

```

ФАЙЛ 76/79: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 1000

```

/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕГДА исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть)
 * с фоллбэком на обход файловой системы, если git недоступен
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir, } from "scripts/export/paths";

/** Расширения, которые считаем «кодом» и включаем в да

```

```
};
```

```
function listFiles() {
  let files;
  try {
    files = execFileSync("git", ["ls-files", "--cached"], {
      cwd: ROOT,
      encoding: "utf8",
    })
      .split("\n")
      .filter(Boolean);
  } catch {
    files = [];
    const walk = (dir) => {
      for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
        const abs = path.join(dir, e.name);
        const rel = path.relative(ROOT, abs).split(path.sep);
        if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
        if (e.isDirectory()) walk(abs);
        else files.push(rel);
      }
    };
    walk(ROOT);
  }

  return files
    .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d + path.sep)))
    .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
    .filter((rel) => CODE_EXT.has(path.extname(rel)))
    .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
    .sort((a, b) => {
      const ia = ORDER.indexOf(categoryOf(a));
      const ib = ORDER.indexOf(categoryOf(b));
      return ia !== ib ? ia - ib : a.localeCompare(b);
    });
}
```

```
/** Достаём заголовки глав из данных пособия (без запус
```

```
push(`Файлов исходного кода:      ${files.length}`);
push(`Суммарный объём кода:        ${humanSize(totalByteSize(files))}`);
push(`Глав/билетов в пособии:      ${chapters.length}`);
push();
```

```
push(HR);
push("                                ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
  push(`  ${String(i + 1).padStart(3, " ")} ${c.title}`);
});
push();
```

```
push(HR);
push("                                ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`  # ${cat}`);
  }
  push(`  ${String(i + 1).padStart(3, " ")} ${rel}`);
});
push();
push();
```

```
current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push(HR);
    push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
    push(HR);
    push();
  }
  push(`  ${String(i + 1).padStart(3, " ")} ${rel}`);
});
push();
```

Универсальное пособие • полный исходный код

```
-----
*   код   -> full_code_all_pages.txt      (collect-code.m
*   txt   -> page-XXXX.png                (render-pages.m
*   png   -> full_code_all_pages.pdf      (build-pdf.mjs)
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
  density: Number(process.env.EXPORT_DENSITY ?? 150),
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
  monochrome: process.env.EXPORT_GRAYSCALE !== "1",
  size: "A4",
  font: "DejaVu-Sans-Mono",
  fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
  pointsize: 8,
  /** Максимум символов в строке (дальше – жёсткий перенос) */
  cols: 138,
  /** Строк содержимого на странице (+2 служебные: шапка, подвал) */
  rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);
```



```

*/
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
  const expanded = line.replace(/\t/g, " ").replace(
    if (expanded.length <= PAGE.cols) return [expanded];
    const parts = [];
    const indent = " ".repeat(Math.min((expanded.match(/^
    let rest = expanded;
    let first = true;
    while (rest.length > 0) {
      const width = first ? PAGE.cols : PAGE.cols - inden
      parts.push((first ? "" : indent) + rest.slice(0, wi
      rest = rest.slice(width);
      first = false;
    }
    return parts;
  }

function paginate(txt) {
  const source = txt.replace(/\r\n/g, "\n").split("\n")
  const pages = [];
  for (let i = 0; i < source.length; i += PAGE.rows) {
    pages.push(source.slice(i, i + PAGE.rows));
  }
  return pages;
}

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`      отрендерено ${done} страниц\n`);
    }
  }
}

await Promise.all(Array.from({ length: concurrency }, worker));
process.stdout.write("\n");

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены ${missing.length} страниц`);

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAGE_PATH)}`);

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message);
  process.exit(1);
});

```

```
permissions:
```

```
  contents: write
```

```
jobs:
```

```
  rebuild-pdf:
```

```
    name: код → txt → png → pdf
```

```
    runs-on: ubuntu-latest
```

```
    # не реагируем на собственный коммит бота
```

```
    if: "!contains(github.event.head_commit.message, '[
```

```
  steps:
```

```
    - name: Checkout
```

```
      uses: actions/checkout@v4
```

```
      with:
```

```
        fetch-depth: 0
```

```
        persist-credentials: true
```

```
    - name: Setup Node.js
```

```
      uses: actions/setup-node@v4
```

```
      with:
```

```
        node-version: "22"
```

```
        cache: npm
```

```
    - name: Install ImageMagick + DejaVu fonts
```

```
      run: |
```

```
        sudo apt-get update -qq
```

```
        sudo apt-get install -y --no-install-recommen
```

```
        # На Ubuntu политика ImageMagick иногда запре
```

```
        # разрешаем то, что нужно пайплайну (text -> p
```

```
        for policy in /etc/ImageMagick-6/policy.xml /
```

```
          if [ -f "$policy" ]; then
```

```
            sudo sed -i 's/rights="none" pattern="\{P
```

```
          fi
```

```
        done
```

```
        convert -version
```

```
    - name: Install dependencies
```

```
      run: npm ci --no-audit --no-fund
```

- name: Upload artifacts (PNG-страницы)
uses: actions/upload-artifact@v4
with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png
 if-no-files-found: error
 retention-days: 7
- name: Commit rebuilt export back to the branch
run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action@users.noreply.github.com"
 git add -- public/export/full_code_all_pages.png
 if git diff --cached --quiet; then
 echo "Экспорт не изменился — коммит не нужен"
 exit 0
 fi
 git commit -m "chore(export): пересборка full-code-pages-png"
 git push origin "HEAD:\${GITHUB_REF_NAME}"